

Politecnico di Torino

Microelectronic Systems

DLX Microprocessor: Design & Development

Final Project Report

Master degree in Electronics Engineering
Master degree in Computer Engineering

Referents: Prof. Mariagrazia Graziano, Giovanna Turvani

Authors: Group 25
Lorenzo Pavone s362884

October 29, 2026

Contents

1 Introduction **1**
- 1.1 Starting point . . . 1
- 1.2 Development . . . 2
- 1.3 Summary of the results . . . 2
- 1.4 Coverage of the project specifications . . . 2
- 1.5 Tools . . . 2
- 1.6 Organization of the report . . . 4

2 Processor overview **5**
- 2.1 Top level . . . 5
- 2.2 An in-order, single-issue pipeline . . . 6
- 2.3 The pipeline . . . 6
- 2.4 Structure of the source files . . . 6

3 Datapath **8**
- 3.1 Overall structure . . . 8
- 3.2 Fetch . . . 8
- 3.3 Register file . . . 9
  - 3.3.1 Overview . . . 9
  - 3.3.2 Implementation . . . 9
  - 3.3.3 Integration in the datapath . . . 10
- 3.4 Immediate extension . . . 10
  - 3.4.1 Overview . . . 10
  - 3.4.2 Implementation . . . 10
  - 3.4.3 Integration in the datapath . . . 10
- 3.5 ALU . . . 11
  - 3.5.1 Overview . . . 11
  - 3.5.2 Operations . . . 11
  - 3.5.3 Integration in the datapath . . . 11
- 3.6 P4 adder . . . 12
  - 3.6.1 Overview . . . 12
  - 3.6.2 Addition and subtraction with one adder . . . 12
  - 3.6.3 Files . . . 13
- 3.7 Registers and multiplexers . . . 13
- 3.8 Write back and jump and link . . . 13

4 Control unit **14**
4.1 Overview . . . 14
4.2 Ports . . . 15
4.3 Control word table . . . 15
4.4 ALU opcode decoder . . . 15
4.5 Delaying the control signals . . . 17
4.6 Flush and stall handling . . . 17
4.7 Reset . . . 17

5 Memories **18**
5.1 Instruction memory . . . 18
5.1.1 Overview . . . 18
5.1.2 Implementation . . . 18
5.2 Data memory . . . 18
5.2.1 Overview . . . 18
5.2.2 Implementation . . . 18

6 Hazard management **19**
6.1 Branches and jumps . . . 19
6.1.1 Resolution in EX . . . 19
6.1.2 Flush . . . 20
6.1.3 Jump and link . . . 20
6.1.4 Jumps through a register . . . 20
6.2 Data hazards . . . 21
6.2.1 Stall . . . 21
6.2.2 Forwarding . . . 21
6.2.3 Load-use hazard . . . 23
6.3 Summary of the hazard cases . . . 23
6.4 Effect on performance . . . 24

7 Instruction set **25**
7.1 Basic instruction set . . . 25
7.2 Extended instruction set . . . 25

8 Verification **27**
8.1 Simulation environment . . . 27
8.1.1 Testbench . . . 27
8.1.2 Running a simulation . . . 27
8.2 Unit tests . . . 28
8.3 Test programs . . . 29
8.4 Description of the test programs . . . 29
8.4.1 `test_mem`: store and load . . . 29
8.4.2 `test_alu`: arithmetic, logic and shifts . . . 30
8.4.3 `test_cmp`: signed comparisons . . . 30
8.4.4 `test_jump` and `test_jump2`: branches and jumps . . . 30
8.4.5 `test_jal`: jump and link . . . 30
8.4.6 `test_branch`: all branches and jumps . . . 30
8.4.7 `test_loop`: a loop over memory . . . 31
8.4.8 `test_jr`: calls and returns through a register . . . 31
8.4.9 `test_lhi`: 32-bit constants . . . 31

8.4.10 `test_stalls`: data hazards . . . 31
8.5 Global test . . . 31
8.6 Test of the extended instruction set . . . 32
8.7 Waveforms . . . 32

9 Synthesis **34**
9.1 Setup . . . 34
9.2 Synthesis script . . . 34
9.3 Running the synthesis and generated files . . . 35
9.4 A synthesis pitfall: initial values of signals . . . 35
9.5 Control unit after synthesis . . . 36
9.6 Search of the maximum frequency . . . 36
9.7 Constraints and timing paths . . . 37
9.8 Synthesized design . . . 37
9.9 Critical path . . . 39
9.10 Power . . . 39

10 Physical design **40**
10.1 Design setup and configuration . . . 40
10.2 Floorplanning . . . 40
10.3 Power planning . . . 40
10.4 Standard-cell placement . . . 41
10.5 Clock tree synthesis . . . 41
10.5.1 The problem . . . 41
10.5.2 The clock tree . . . 41
10.5.3 Implementation . . . 41
10.5.4 Effects on the results . . . 42
10.6 Routing . . . 42
10.7 Filler cells . . . 42
10.8 Parasitic extraction and outputs . . . 43
10.8.1 Generated files . . . 43
10.9 Timing before and after the optimization steps . . . 43
10.10 Results . . . 44
10.11 Power . . . 45
10.12 Electromigration and thermal analysis . . . 46
10.13 Limitations . . . 46

11 Conclusions **47**
11.1 Lessons learned . . . 47
11.2 Future work . . . 47
11.3 Acknowledgements . . . 48

A Issues found in the provided material **49**

B Instruction encoding **51**

C Test programs **52**
C.1 Global test: `test_all.asm` . . . 52
C.2 Extended instruction set: `test_ext.asm` . . . 53
C.3 Jumps through a register: `test_jr.asm` . . . 54
C.4 Load high immediate: `test_lhi.asm` . . . 54

D Scripts **55**
D.1 Assembly and simulation: `build.sh` . . . 55
D.2 Synthesis: `dlx.scr` . . . 56
D.3 Place & route: `dlx_pnr.tcl` . . . 57

E VHDL sources **58**
E.1 Package . . . 58
E.2 Top level . . . 60
E.3 Top level for synthesis . . . 64
E.4 Control unit . . . 67
E.5 Datapath . . . 71
E.6 Register file . . . 78
E.7 ALU . . . 79
E.8 P4 adder . . . 81
E.8.1 P4 adder . . . 81
E.8.2 Carry generator . . . 82
E.8.3 G, PG and PG-network blocks . . . 85
E.8.4 Sum generator . . . 86
E.8.5 Carry-select block . . . 87
E.8.6 Ripple-carry adder and full adder . . . 88
E.8.7 4-bit multiplexer . . . 89
E.9 Multiplexer . . . 90
E.10 Register . . . 91
E.11 Instruction memory . . . 92
E.12 Data memory . . . 93
E.13 Testbenches . . . 93
E.13.1 Processor . . . 94
E.13.2 ALU . . . 95
E.13.3 P4 adder . . . 96
E.13.4 Register file . . . 97
E.13.5 Control unit . . . 98
E.13.6 Register . . . 101
E.13.7 Multiplexer . . . 102
E.13.8 Instruction memory . . . 102
E.13.9 Data memory . . . 103

CHAPTER 1

Introduction

This report describes the design, verification and physical implementation of a pipelined DLX microprocessor, developed as the final project of the Microelectronic Systems course.

The processor is a **scalar, single-issue, in-order** DLX with a five-stage pipeline (IF, ID, EX, MEM, WB) and a hardwired control unit. It covers all the mandatory points of the project guide (points 1 to 6, required for both the basic and the pro version: pipeline, instruction subset, datapath, synthesis, physical design and report) and adds the following pro features:

- **Extended instruction set** (point 7): 23 instructions added to the 27 of the basic set, for a total of 50: signed and unsigned comparisons, arithmetic right shift, unsigned additions and subtractions, jumps through a register (`jr`, `jalr`) and load of the upper half of a register (`lhi`).
- **ALU based on the P4 adder** (point 8): additions and subtractions are performed by the P4 adder developed in the laboratories, a sparse-tree carry generator followed by a carry-select sum generator.
- **Forwarding** (points 8 and 15): values are forwarded from the MEM and WB stages, and the register file has a write-through bypass. The only data hazard that still requires a stall is the *load-use* case, which costs one cycle.
- **Clock tree synthesis** in the physical design (point 14).
- **Automation scripts**: a single command assembles, checks and simulates a program, and the synthesis script is parametric in the clock period.

1.1 Starting point

The project started from the material provided with the course:

- a top level, `a-DLX.vhd`, containing the fetch stage (program counter, instruction register, instruction memory);
- three partial versions of the control unit, hardwired, finite state machine and microprogrammed, with their interface: the hardwired one has been chosen (chapter 4) and completed;
- a testbench, an instruction memory model, the assembler `dlxasm.pl` and the conversion script `conv2memory`;

- the components developed in the laboratories (register file, generic registers and multiplexers, ALU).

The datapath, the pipelining of the control signals and the whole hazard management were not provided and have been designed from scratch. Some problems were found in the provided files and corrected; they are listed in appendix A.

1.2 Development

The design has been grown in small steps, each one verified with its own assembly program before moving to the next:

1. datapath and first program (`addi`, `sw`, `lw`);
2. correction of the problems found in the provided material;
3. conditional branches and jumps with flush, first resolved in MEM and then moved to EX;
4. `jal`, with the return address written into `r31`;
5. stall logic for every data dependency;
6. forwarding and register file bypass, which reduced the stalls to the load-use case;
7. the first group of instructions of the pro set: comparisons, arithmetic shift, unsigned arithmetic;
8. synthesis and physical design;
9. jumps through a register (`jr`, `jalr`) and `lhi`, which complete subroutine calls and 32-bit constants;
10. the ALU based on the P4 adder of the laboratories, followed by the final synthesis and physical design.

1.3 Summary of the results

Table 1.1 collects the main figures obtained. The details are discussed in chapters 9 and 10.

1.4 Coverage of the project specifications

Table 1.2 maps the items of the project guide to the chapters where they are addressed.

1.5 Tools

- Simulation: Siemens Questa Sim 2025.3;
- Synthesis: Synopsys Design Compiler W-2024.09-SP2;
- Place & route: Cadence Innovus 23.35;
- Technology: NanGate 45 nm Open Cell Library;
- Assembler: the `dlxasm.pl` and `conv2memory` scripts provided with the project material.

Parameter	Value
Pipeline stages	5, in order, single issue
Implemented instructions	50
Branch penalty (taken branch or jump)	2 cycles
Load-use penalty	1 cycle
Other RAW hazards	0 cycles (forwarding)
Minimum clock period reached in synthesis	1.48 ns (≈676 MHz)
Clock period met in synthesis	1.7 ns (588 MHz), slack 0.00 ns
Setup slack after place & route, at 1.7 ns	+0.165 ns (≈651 MHz)
Hold slack after place & route	0.000 ns, 0 violations
Design rule violations after routing	0
Cell area after place & route	15 094 μm^2
Die size	$176.1 \times 175.3\,\mu m$
Power in synthesis, at 588 MHz	7.00 mW dynamic, 0.32 mW leakage
Power after place & route, at 588 MHz	13.18 mW (leakage 0.29 mW)

Table 1.1: Summary of the main results.

Item of the project guide	Status	Where and how
1. Five-stage pipeline (basic)	done	chapters 2–6: hardwired CU, hazards in hardware
2. Instruction set, ALU opcode as enumeration (basic)	done	chapter 7, type `aluOp`; commented test programs, appendix C
3. Datapath at RT level (basic)	done	chapter 3
4. Synthesis, frequency optimization, commented scripts (basic)	done	chapter 9, appendix D
5. Physical design and post-layout performance (basic)	done	chapter 10; EM and thermal analysis not available with the installed tools
6. Report (mandatory)	done	this document
7. Extended instruction subset (pro)	23 instructions	chapter 7; `lb`, `lbu`, `lhu`, `sb`, `mult` not implemented
8. Arithmetic structures and critical path (optional)	done	ALU with the P4 adder, section 3.6; critical path analysis, section 9.9
14. Physical design (optional): clock tree synthesis, post-layout simulation, crosstalk	partly done	clock tree synthesis done, section 10.5; post-layout simulation and crosstalk analysis not performed
15. Free features	done	forwarding with register file bypass; automated assembly and simulation script

Table 1.2: Coverage of the items of the project guide.

1.6 Organization of the report

- Chapter 2 gives the overall picture: top level, in-order pipeline, operations of each stage and organization of the source files.
- Chapter 3 describes the datapath component by component, and chapter 4 the hardwired control unit.
- Chapter 6 explains how control and data hazards are handled: flush, forwarding and the load-use stall.
- Chapter 7 lists the instruction set, basic and extended.
- Chapter 8 presents the verification flow and the test programs.
- Chapters 9 and 10 report synthesis and physical design.
- Chapter 11 draws the conclusions.

The appendices collect the problems found in the provided material, the instruction encoding, the test programs, the scripts and the VHDL sources.

CHAPTER 2

Processor overview

This chapter gives the overall picture of the processor: how it is organized, how instructions flow through the pipeline, and how the source files are structured. The details of datapath and control unit are given in chapters 3 and 4.

2.1 Top level

The processor follows a Harvard organization: instructions and data are stored in two separate memories, so that the fetch of an instruction and the memory access of a load or store never compete for the same resource. The top level entity, `DLX`, instantiates four blocks (figure 2.1):

- the **control unit** `dlx_cu`, which decodes the instruction and drives the control signals of every stage;
- the **datapath** `DataPath`, which contains the register file, the ALU, the hazard logic and all the pipeline registers from ID to WB;
- the **instruction memory** `IRAM` and the **data memory** `DRAM`, which are behavioral models used only in simulation.

The fetch stage (program counter `PC`, instruction register `IR` and the adder computing $PC + 4$) is described directly in the top level, as in the structure provided with the project. All registers use an asynchronous reset, active low, consistently with the provided testbench.

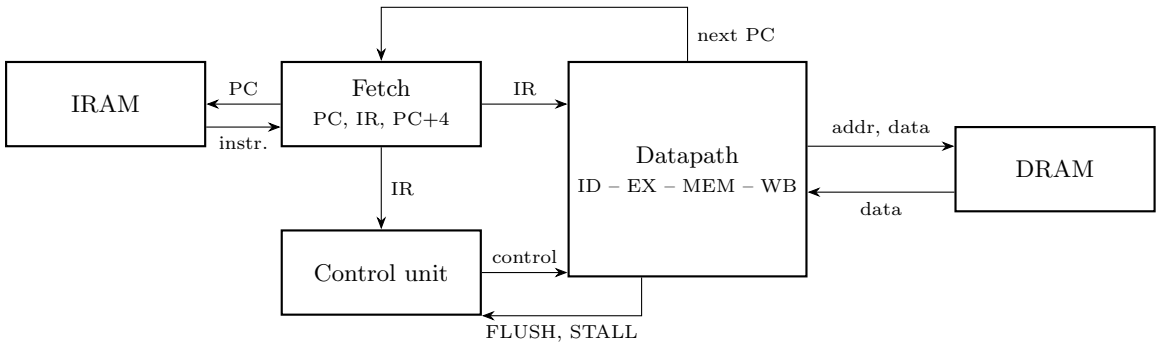


Figure 2.1: Top level organization of the DLX.

The datapath sends two signals back to the control unit: `FLUSH`, asserted when a branch or jump is taken, and `STALL`, asserted when the instruction in ID must wait for a load. All the other signals flow from the control unit to the datapath.

2.2 An in-order, single-issue pipeline

The DLX is a **scalar, single-issue, in-order** processor. At most one instruction is fetched per cycle, and instructions enter the pipeline, execute and write their results strictly in program order: there is no out-of-order execution, no instruction queue and no register renaming. This choice keeps the hardware simple and has a direct consequence on hazards:

- every instruction reads its operands in ID and writes its result in WB, always in the same stage;
- an older instruction always reaches WB before a younger one;
- therefore only **read-after-write** (RAW) dependencies can cause a hazard. Write-after-read (WAR) and write-after-write (WAW) hazards, which appear in out-of-order machines, cannot occur.

For this reason both the forwarding unit and the stall logic (chapter 6) only compare the *source* registers of an instruction with the *destinations* of the older instructions still in the pipeline.

2.3 The pipeline

The execution of an instruction is divided into five stages, each one completed in one clock cycle:

1. **Instruction Fetch (IF)**: the instruction addressed by the PC is read from the IRAM, and $PC + 4$ is computed;
2. **Instruction Decode (ID)**: the instruction is decoded by the control unit, the register file is read and the immediate is extended to 32 bits;
3. **Execute (EX)**: the ALU computes the result, the memory address or the jump target; branches are resolved;
4. **Memory (MEM)**: loads read and stores write the data memory;
5. **Write Back (WB)**: the result is written into the register file.

With five stages, up to five instructions are in execution at the same time, one per stage. In the absence of hazards one instruction is completed at every cycle: the latency of a single instruction is five cycles, but the throughput is one instruction per cycle. Table 2.1 shows what each class of instructions does in every stage.

2.4 Structure of the source files

The files follow the naming convention of the project guide: an ordering prefix reflecting the hierarchy, the name of the component and, for the leaf components, the version. The datapath components are grouped in a sub-folder, the custom types are in `000-globals.vhd`, and the testbench is in `test_bench` with the prefix `TB`.

```
Project_basic/
  000-globals.vhd                 package myTypes: constants, aluOp
  a-DLX.vhd                       top level for simulation
  a-DLX_syn.vhd                   top level for synthesis (no memories)
```

Class	IF	ID	EX	MEM	WB
R-type	fetch, $PC+4$	read `rs1`, `rs2`	A *op* B	–	write `rd`
I-type ALU	fetch, $PC+4$	read `rs1`, extend imm.	A *op* IMM	–	write `rd`
`lw`	fetch, $PC+4$	read base, extend offset	address $A+IMM$	read memory	write loaded value
`sw`	fetch, $PC+4$	read base and data	address $A+IMM$	write memory	–
`beqz`, `bnez`	fetch, $PC+4$	read tested register	target $NPC+IMM$, decision	–	–
`j`	fetch, $PC+4$	extend 26-bit offset	target $NPC+IMM$	–	–
`jal`	fetch, $PC+4$	extend 26-bit offset	target $NPC+IMM$	–	write $PC+4$ in `r31`
`jr`, `jalr`	fetch, $PC+4$	read target register	target $A+0$	–	`jalr`: write $PC+4$ in `r31`
`lhi`	fetch, $PC+4$	extend immediate	$IMM \ll 16$	–	write `rd`

Table 2.1: Operations performed in each stage by the instruction classes.

```
a.a-CU_HW.vhd                    hardwired control unit
a.b-DataPath.vhd                 datapath
a.b-DataPath.core/
  a.b.a-register_file_v1.vhd     register file with write-through bypass
  a.b.b-ALU_v1.vhd               ALU
  a.b.b.a-p4_adder.vhd           P4 adder, used by the ALU
  a.b.b.a.a-carry_generator.vhd    sparse-tree carry generator
  a.b.b.a.a.a/b/c-...              G, PG and PG-network blocks
  a.b.b.a.b-sum_generator.vhd      sum generator
  a.b.b.a.b.a-...                  carry-select block, RCA, FA, mux
  a.b.c-mux21_v1.vhd             generic 2-to-1 multiplexer
  a.b.d-register_v1.vhd          generic register with enable
a.c-IRAM.vhd                     instruction memory (simulation only)
a.d-DRAM.vhd                     data memory (simulation only)
test_bench/TB-DLX.vhd            testbench of the processor
test_bench/TB-alu.vhd, TB-p4adder.vhd, TB-regfile.vhd,
           TB-cu.vhd, TB-reg.vhd, TB-mux21.vhd,
           TB-iram.vhd, TB-dram.vhd    unit testbenches
asm/                             assembly test programs
assembler.bin/                   dlxasm.pl, conv2memory
build.sh, waves.do               assembly and simulation scripts
syn/dlx.scr                      synthesis script
pnr/DLX.globals, DLX.view,
    dlx_pnr.tcl                  place & route configuration and script
```

The VHDL files are compiled in the order of the prefixes: package, components of the datapath, datapath, memories, control unit, top level and testbench.

CHAPTER 3

Datapath

3.1 Overall structure

Figure 3.1 shows the complete pipelined datapath. The grey bars are the pipeline registers that separate the five stages; every signal that crosses a bar is stored in a register for one clock cycle. The datapath is made of the following components:

1. the fetch logic: program counter, $PC + 4$ adder and jump multiplexer `MUXJUMP`;
2. the register file, with a write-through bypass;
3. the immediate extension unit;
4. the forwarding unit, in front of the ALU (chapter 6);
5. the ALU, with its two input multiplexers `MUXA` and `MUXB`, and the P4 adder inside it;
6. the branch logic, which computes `take_branch` (chapter 6);
7. the write-back multiplexers `MUXWB` and `MUXJAL`, and the selection of `r31` for `jal`;
8. generic registers and multiplexers, used for all the pipeline registers.

The following sections describe the components in the order in which an instruction meets them.

3.2 Fetch

At every clock cycle the instruction addressed by `PC` is read from the IRAM and loaded into `IR`, while `PC` is updated with the output of `MUXJUMP`: $PC + 4$ in the normal flow, or the target computed by the ALU when `take_branch` is asserted: $PC + 4$ plus the offset for branches, `j` and `jal`, the content of a register for `jr` and `jalr`. $PC + 4$ is also propagated along the pipeline as `npc_id`, `npc_ex`, `npc_mem`, `npc_wb`: in EX it is the base for the branch target, in WB it is the return address of `jal`.

The enables of `PC`, `IR` and `npc_id` are deasserted only during a stall, so that the instruction waiting in ID and the one being fetched are kept.

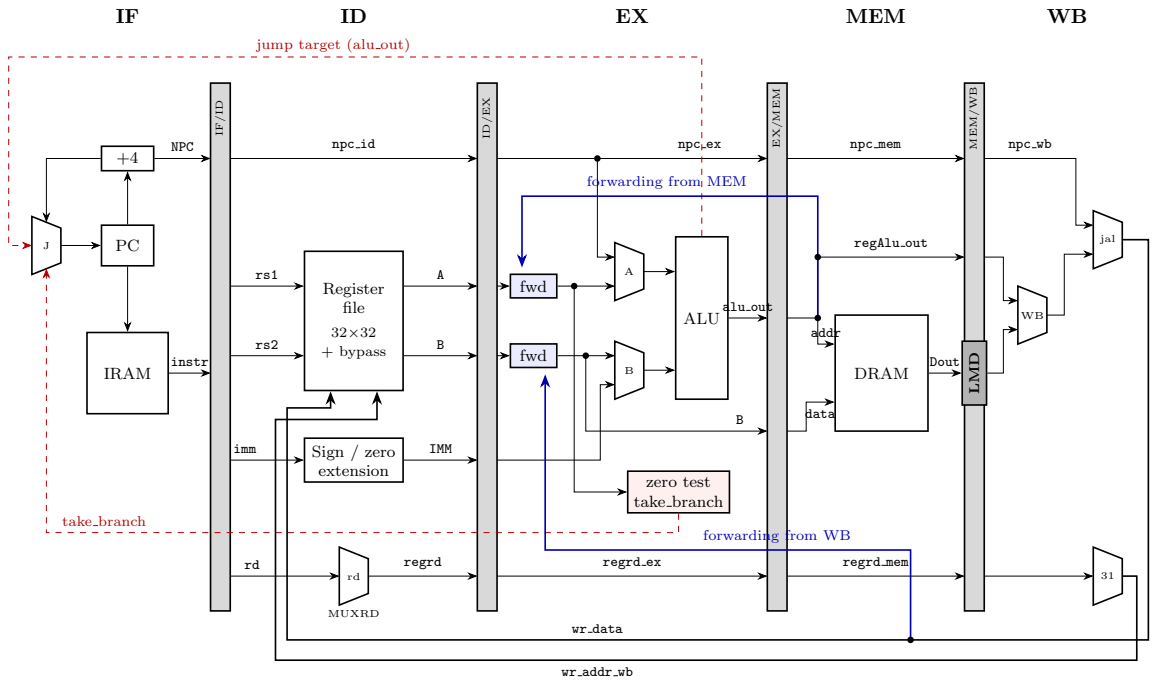


Figure 3.1: Pipelined datapath. Blue: forwarding paths. Red, dashed: branch resolution towards the PC. The muxes are labelled J (`MUXJUMP`), rd (`MUXRD`, selection of the destination register), A and B (`MUXA`, `MUXB`), WB (`MUXWB`), jal and 31 (`MUXJAL_DATA`, `MUXJAL_ADDR`).

3.3 Register file

3.3.1 Overview

The register file contains 32 registers of 32 bits, with two read ports (`OUT1`, `OUT2`) and one write port. It is read in ID and written in WB.

3.3.2 Implementation

Writing is synchronous: on the rising edge of the clock, if `WR` is asserted, `DATAIN` is stored in the register addressed by `ADD_WR`. Reading is combinational, with two particular cases (figure 3.2):

- register `r0` always reads zero, as required by the DLX architecture, even if an instruction tries to write it;
- when the register being read is the one being written in the same cycle, the output is `DATAIN` instead of the stored value. Without this *write-through bypass*, the new value would be stored only at the end of the cycle, too late for the instruction in ID.

```
OUT1 <= (others => '0') when ADD_RD1 = "00000" else
        DATAIN          when (WR = '1' and ADD_RD1 = ADD_WR) else
        REGISTERS(to_integer(unsigned(ADD_RD1)));
```

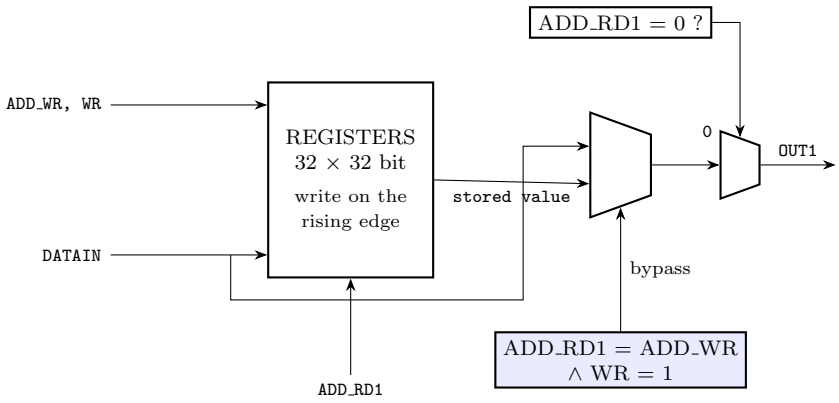


Figure 3.2: Read port of the register file, with the r0 check and the write-through bypass. The second port is identical.

3.3.3 Integration in the datapath

The read addresses come directly from the `rs1` and `rs2` fields of `IR`. The write address and data come from WB (`wr_addr_wb`, `wr_data`), after the `jal` multiplexers. The reset, active low, clears all the registers.

3.4 Immediate extension

3.4.1 Overview

The immediate of the instruction is brought to 32 bits in ID (figure 3.3):

- the 16-bit immediate of I-type instructions is extended with its sign bit, or with zeros for the logical and unsigned instructions (`is_uimm`);
- the 26-bit offset of `j` and `jal` is sign-extended;
- a multiplexer selects one of the two, according to the opcode.

3.4.2 Implementation

The extension is described with concurrent assignments in the datapath. The fill value of the 16-bit immediate depends on the opcode:

```
fill_i <= (others => '0') when (opcode = ITYPE_ANDI  or opcode = ITYPE_ORI   or
           opcode = ITYPE_XORI  or opcode = ITYPE_ADDUI or opcode = ITYPE_SUBUI or
           opcode = ITYPE_SLTUI or opcode = ITYPE_SGTUI or opcode = ITYPE_SLEUI or
           opcode = ITYPE_SGEUI)
          else (others => immediate_i(15));
```

3.4.3 Integration in the datapath

The selected value is stored in the `IMM` register at the end of ID and reaches `MUXB` in EX, where it replaces the second register operand for I-type instructions, loads, stores, branches and jumps.

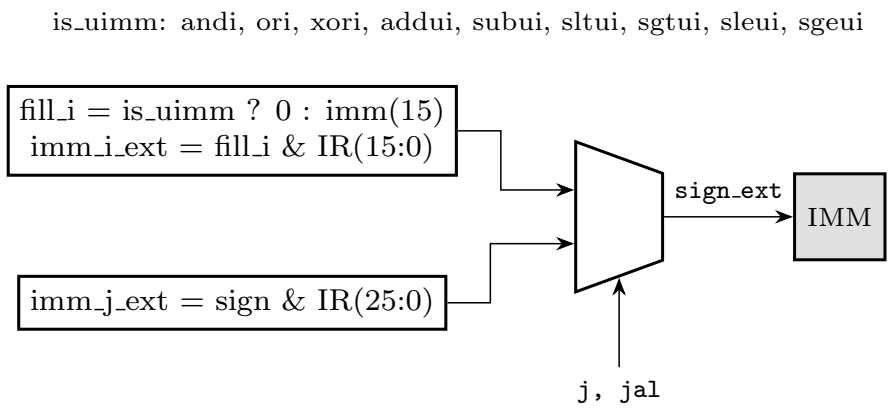


Figure 3.3: Immediate extension unit.

3.5 ALU

3.5.1 Overview

The ALU receives two 32-bit operands and the operation `FUNC`, of the enumerated type `aluOp` defined in `000-globals.vhd`, as required by the project specifications. Additions and subtractions are performed by a structural **P4 adder** (section 3.6); all the other operations are described behaviorally, in a single combinational process with a `case` on the operation.

3.5.2 Operations

Table 3.1 lists the operations and how they are implemented.

Operation	Implementation	Used by
`ADDS`	P4 adder, `Cin` = 0	add, addu, addi, addui, lw, sw, branch and jump targets
`SUBS`	P4 adder, `not DATA2`, `Cin` = 1	sub, subu, subi, subui
`ANDS`, `ORS`, `XORS`	bitwise	and, or, xor and immediates
`LLS`, `LRS`	`sll`, `srl` on `bit_vector`, amount `DATA2(4:0)`	sll, srl and immediates
`ARS`	`sra` on `bit_vector`	sra, srai
`SEQS`, `SNES`	equality, result 1 or 0	seq, sne and immediates
`SLTS`, `SGTS`, `SLES`, `SGES`	comparison of `signed(...)`	slt, sgt, sle, sge and immediates
`SLTUS`, `SGTUS`, `SLEUS`, `SGEUS`	comparison of `unsigned(...)`	sltu, sgtu, sleu, sgeu and immediates
`LHIS`	`DATA2(15:0)` in the upper half, zeros below	lhi
`NOP`	output zero	nop, bubbles

Table 3.1: ALU operations.

The comparisons produce a 32-bit word whose least significant bit is the result, so that they can be written into a register as any other ALU result. They are written as two assignments, all zeros followed by the least significant bit, which keeps the description free of aggregates whose size depends on the generic.

3.5.3 Integration in the datapath

The first operand comes from `MUXA` (register or $PC+4$), the second from `MUXB` (register or immediate), both after the forwarding logic. The result is the value written back, the address of loads and stores, and the target of every branch and jump, which reaches the PC through `MUXJUMP`.

3.6 P4 adder

3.6.1 Overview

The adder of the ALU is the P4 adder developed in the laboratories, the structure of the Pentium 4 adder. It is made of two parts (figure 3.4):

- a **carry generator**, a sparse tree of PG and G blocks that computes in parallel only the carries at the boundaries of the 4-bit blocks, $C_4, C_8, \ldots, C_{28}$, in a logarithmic number of levels;
- a **sum generator**, made of eight carry-select blocks of 4 bits. Each block computes its sum twice with two small ripple-carry adders, assuming an incoming carry of 0 and of 1, and a multiplexer selects the correct one as soon as the carry generator provides the real carry.

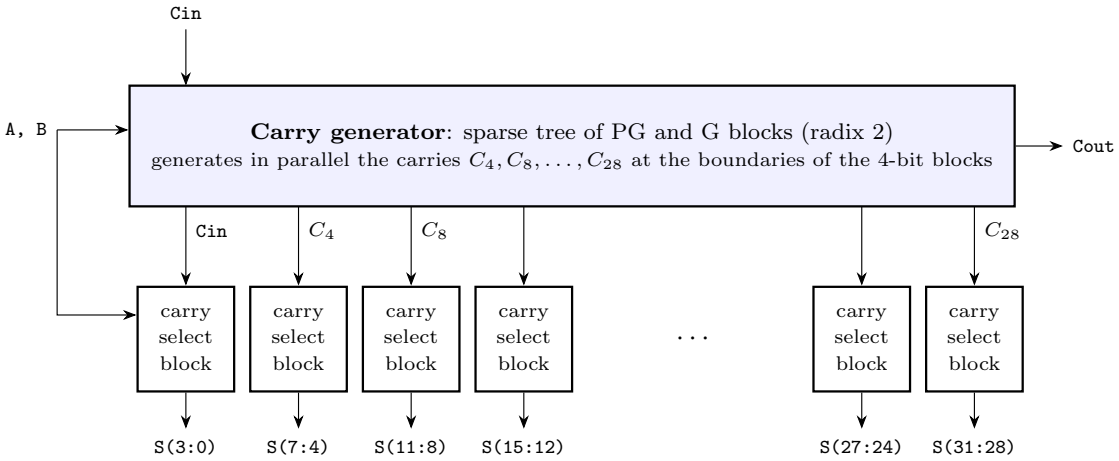


Figure 3.4: Structure of the P4 adder.

The two parts work in parallel: while the carry tree is computing the block carries, the carry-select blocks are already computing both possible sums, so the delay grows with the logarithm of the width instead of linearly as in a ripple-carry adder.

3.6.2 Addition and subtraction with one adder

A single P4 adder performs both operations. In two's complement $-B = \overline{B} + 1$, hence

$$A - B = A + \overline{B} + 1.$$

For a subtraction the ALU inverts the second operand and sets the carry input to 1; for an addition it passes the operand unchanged with carry input 0:

```
cin_p4 <= '1' when FUNC = SUBS else '0';          -- +1 for the subtraction
b_p4   <= not DATA2 when FUNC = SUBS else DATA2;  -- A - B = A + not(B) + 1

ADDER: P4_ADDER
  generic map (NBIT => N)
  port map (A => DATA1, B => b_p4, Cin => cin_p4, S => sum_p4, Cout => cout_p4);
```

In the `case` of the ALU, both `ADDS` and `SUBS` return `sum_p4`. The carry output is not used, since the DLX has no flags.

3.6.3 Files

The adder keeps the hierarchy of the laboratory: `P4_ADDER` instantiates `CARRY_GENERATOR` (with the `G_BLOCK`, `PG_BLOCK` and `PG_NET_BLOCK` components) and `SUM_GENERATOR` (with `CARRY_SELECT_BLOCK`, `RCA`, `FA` and `MUX21_4BIT`). The files are in `a.b-DataPath.core` with the prefix `a.b.b.a`, as subcomponents of the ALU.

3.7 Registers and multiplexers

All the pipeline registers are instances of the same generic component `REG`, with parametric width, an enable and an asynchronous active-low reset that clears the content. The multiplexers are instances of the generic 2-to-1 `MUX21_GENERIC`.

Table 3.2 lists the signals propagated along the pipeline beside the data registers `A`, `B`, `IMM`, `ALU_OUT`, `LMD`.

Signal	Path	Purpose
`regrd_ex/mem/wb`	ID → WB	destination register
`npc_id/ex/mem/wb`	IF → WB	$PC + 4$: branch target and return address of `jal`
`rs1_ex`, `rs2_ex`	ID → EX	source registers, for the forwarding logic
`regB_mem`	EX → MEM	data to be stored by `sw`
`is_jump_ex`	ID → EX	unconditional jump
`is_load_ex`	ID → EX	load in EX, for the load-use stall
`is_jal_ex/mem/wb`	ID → WB	selection of return address and `r31`

Table 3.2: Signals propagated along the pipeline.

3.8 Write back and jump and link

`MUXWB` selects between the ALU result and the value loaded from memory. Two further multiplexers handle `jal` and `jalr`: `MUXJAL_DATA` replaces the data with the return address `npc_wb`, and `MUXJAL_ADDR` forces the destination to `r31`; both are driven by `is_jal_wb`. The registers `npc_mem` and `npc_wb` are enabled only when the instruction in the previous stage is a `jal` or a `jalr`: in this way the return address cannot be overwritten by the instructions flushed after the jump.

CHAPTER 4

Control unit

4.1 Overview

Among the three possible implementations proposed in the project guide (hardwired, finite state machine, microprogrammed), a **hardwired** control unit has been chosen. In a pipeline every stage receives a new instruction at every cycle, so the control unit has no internal state to follow, as an FSM would, and no sequence of micro-instructions to execute, as a microprogrammed unit would: it only has to translate each instruction into its control signals and deliver them to the right stage at the right time. A lookup table indexed by the opcode, followed by a chain of registers, does exactly this with the least logic.

Figure 4.1 shows its structure. It has three parts:

1. the **control word table**, which translates the opcode into 15 control bits;
2. the **ALU opcode decoder**, which translates opcode and function field into the ALU operation;
3. the **pipeline of control words**, which delivers each bit to its stage, and inserts bubbles for flushes and stalls.

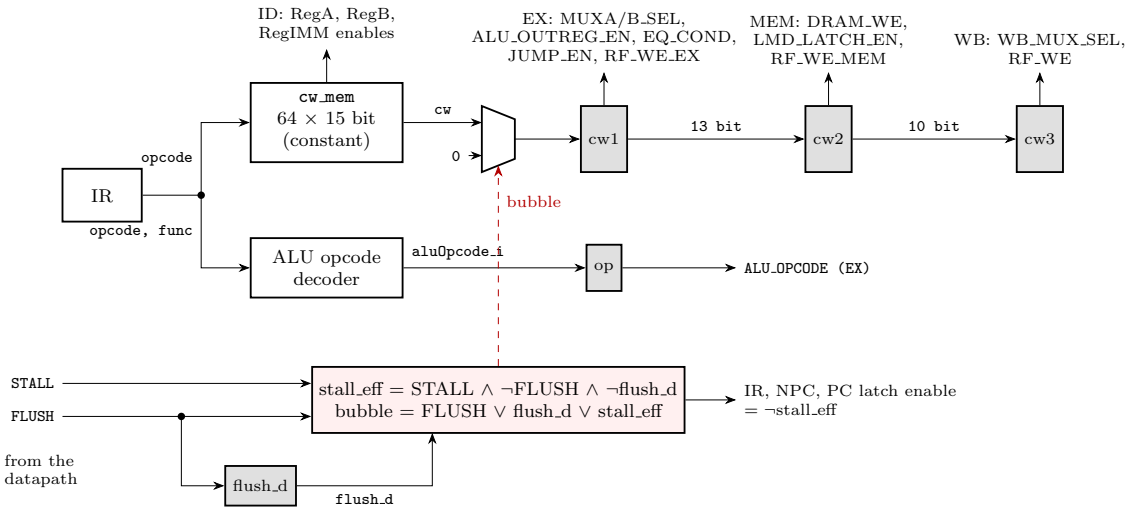


Figure 4.1: Structure of the hardwired control unit.

4.2 Ports

Table 4.1 describes the interface of the control unit.

Port	Direction	Description
`Clk`, `Rst`	in	clock; asynchronous reset, active low
`IR_IN`	in	instruction in ID (32 bits)
`FLUSH`	in	a branch or jump is taken in EX
`STALL`	in	load-use hazard detected in ID
`IR_LATCH_EN`, `NPC_LATCH_EN`, `PC_LATCH_EN`	out	fetch enables, low during a stall
`RegA_LATCH_EN`, `RegB_LATCH_EN`, `RegIMM_LATCH_EN`	out	ID: load the operand registers
`MUXA_SEL`, `MUXB_SEL`	out	EX: ALU operand selection
`ALU_OUTREG_EN`, `EQ_COND`, `JUMP_EN`	out	EX: ALU output register, branch type, jump enable
`ALU_OPCODE`	out	EX: ALU operation, type `aluOp`
`DRAM_WE`, `LMD_LATCH_EN`	out	MEM: memory write, load data register
`WB_MUX_SEL`, `RF_WE_WB`	out	WB: write-back source, register file write
`RF_WE_EX`, `RF_WE_MEM`	out	write enables of the instructions in EX and MEM, for the hazard logic

Table 4.1: Ports of the control unit.

4.3 Control word table

The opcode of the instruction in `IR` indexes a table of 64 control words of 15 bits, `cw_mem`, declared as a VHDL *constant*. All R-type instructions share opcode `0x00` and therefore the same control word; they are distinguished only by the ALU operation. Table 4.2 shows the meaning of the bits.

Bit	Signal	Stage
14, 13	`IR_LATCH_EN`, `NPC_LATCH_EN`	IF
12 – 10	`RegA_LATCH_EN`, `RegB_LATCH_EN`, `RegIMM_LATCH_EN`	ID
9 – 6	`MUXA_SEL`, `MUXB_SEL`, `ALU_OUTREG_EN`, `EQ_COND`	EX
5 – 2	`DRAM_WE`, `LMD_LATCH_EN`, `JUMP_EN`, `PC_LATCH_EN`	MEM
1, 0	`WB_MUX_SEL`, `RF_WE`	WB

Table 4.2: Structure of the control word.

Table 4.3 shows the control words of the instruction classes. `j` and `bnez` share the same word: the two are distinguished in the datapath, where `is_jump` marks the unconditional jumps. `jr` and `jalr` differ from `j` and `jal` only in `MUXA_SEL`, which selects the register instead of $PC + 4$, so that the ALU computes the target as the register content plus a zero immediate. `lhi` has the same control word as the arithmetic I-type instructions: the difference is only in the ALU operation.

4.4 ALU opcode decoder

A combinational process translates the instruction into the ALU operation: for R-type instructions it decodes the 11-bit function field, for the others the opcode (table 4.4). Loads, stores, branches and jumps use `ADDS`, to compute an address or a target; unrecognized codes produce `NOP`. The result is delayed by one register, so that it reaches the ALU when the instruction is in EX.

Class	Control word	Hex
R-type	111100110000101	7985
I-type arithmetic/logic	111010010000101	7485
lw	111010010010111	7497
sw	111110010100100	7CA4
beqz	111011011001100	76CC
bnez, j	111011010001100	768C
jal	110011010001101	668D
jr	111010010001100	748C
jalr	111010010001101	748D
lhi	111010010000101	7485
nop, bubble	000000000000000	0000

Table 4.3: Control words of the instruction classes.

Instructions	Decoded from	ALU operation
add, addu, addi, addui	func / opcode	ADDS
lw, sw, beqz, bnez, j, jal, jr, jalr	opcode	ADDS (address or target)
sub, subu, subi, subui	func / opcode	SUBS
and, or, xor and immediates	func / opcode	ANDS, ORS, XORS
sll, srl, sra and immediates	func / opcode	LLS, LRS, ARS
set-on-condition, signed	func / opcode	SEQS . . . SGES
set-on-condition, unsigned	func / opcode	SLTUS . . . SGEUS
lhi	opcode	LHIS
nop, unknown codes	opcode	NOP

Table 4.4: ALU opcode decoding.

4.5 Delaying the control signals

In a pipeline every stage holds a different instruction, so every stage needs the control word of its *own* instruction. The word read from the table drives the ID stage directly, and is then shifted along three registers:

- `cw1`, 15 bits, drives the EX stage;
- `cw2`, 13 bits, drives the MEM stage;
- `cw3`, 10 bits, drives the WB stage.

The registers are progressively narrower because the bits already consumed are not propagated. In this way each control signal reaches its stage together with the data of the same instruction.

With respect to the original layout of the control word, three signals are no longer taken from the table: the fetch enables and `PC_LATCH_EN` are generated by the stall logic, and `JUMP_EN` is read from `cw1` instead of `cw2`, because branches are resolved in EX. Two further outputs, `RF_WE_EX` and `RF_WE_MEM`, export the write enable of the instructions in EX and MEM to the hazard logic of the datapath.

4.6 Flush and stall handling

The control unit receives `FLUSH` and `STALL` from the datapath and reacts as follows:

- a **bubble** is inserted in EX by loading zeros into `cw1` instead of the control word of the instruction in ID. A zero control word deasserts every enable: the instruction keeps flowing through the pipeline but writes neither the register file nor the memory. This happens when `FLUSH` is asserted, in the following cycle (`flush_d`), and during a stall;
- during a **stall**, the enables of `IR`, `NPC` and `PC` are deasserted, so that the waiting instruction stays in ID and the next one stays in IF;
- a stall is ignored when a branch is taken in the same or in the previous cycle: `stall_eff = STALL and not FLUSH and not flush_d`. The instruction asking for the stall is on the wrong path and is being flushed, and the PC must be free to load the target.

```
if FLUSH = '1' or flush_d = '1' or stall_eff = '1' then
  cw1 <= (others => '0');                    -- bubble in EX
else
  cw1 <= cw;
end if;
cw2 <= cw1(CW_SIZE-1-2 downto 0);            -- older instructions proceed
cw3 <= cw2(CW_SIZE-1-5 downto 0);
```

4.7 Reset

At reset the control word registers are cleared, so that no stage can write the register file or the memory before the first instruction arrives. The instruction register is also cleared: since opcode `0x00` is the R-type row of the table, the first cycles after reset execute an R-type instruction with all fields at zero, which writes `r0` and has therefore no effect.

CHAPTER 5

Memories

The two memories are behavioral models used only in simulation: as required by the project guide, they are excluded from synthesis and connected to the processor through the ports of the synthesis top level (chapter 9). Both are word-organized, with 32-bit words, and are addressed with byte addresses: the two least significant bits are discarded, so that only word-aligned accesses are meaningful.

5.1 Instruction memory

5.1.1 Overview

`IRAM` holds 48 words. At reset it reads the program from the file `test.asm.mem`, one hexadecimal word per line, produced by the assembler and the conversion script.

5.1.2 Implementation

The read is combinational: the instruction addressed by the PC is available in the same cycle and is loaded into `IR` at the next rising edge. The index is `Addr(31 downto 2)`, i.e. the byte address divided by four, modulo the memory depth. The provided model used the PC directly as index, so that only one instruction out of four was read; this and the reload of the program at every reset have been corrected (appendix A).

5.2 Data memory

5.2.1 Overview

`DRAM` holds 64 words. It is read by loads and written by stores in the MEM stage.

5.2.2 Implementation

Reading is combinational, writing is synchronous and enabled by `DRAM_WE`. The address is the ALU result registered in `regAlu_out`, and the data to be written is `regB_mem`. A store at address 4 writes the second word: this convention is used by all the test programs.

Known limitations. Both memories are arrays of `integer`, as in the provided material. They work correctly, but converting a word whose most significant bit is set produces warnings in simulation. A description based on `std_logic_vector` would remove them.

CHAPTER 6

Hazard management

A pipeline executes several instructions at the same time, one per stage, and in general this creates three kinds of hazards. *Structural hazards* cannot occur in this design, because every stage has its own resources: instructions and data reside in separate memories, $PC + 4$ is computed by a dedicated adder and not by the ALU, every instruction spends a single cycle in EX, and the register file has two read ports and one write port. *Control hazards* appear when a branch or jump changes the flow of the program after some instructions have already been fetched. *Data hazards* appear when an instruction needs a register that a previous instruction has not written yet.

Since the pipeline is in order (section 2.2), the only data hazards are read-after-write: an instruction can only need a value that an *older* instruction has not written yet. WAR and WAW hazards cannot happen, because every instruction reads in ID and writes in WB, in program order.

6.1 Branches and jumps

6.1.1 Resolution in EX

Branches and jumps are resolved in the EX stage. In the same cycle the ALU computes the target $PC + 4 + imm$, and the condition is evaluated:

$$cond = is_jump_ex \ \vee \ (EQ_COND = regA_zero)$$
$$take_branch = JUMP_EN \wedge cond$$

`regA_zero` tells whether the tested register is zero, and `EQ_COND` is 1 for `beqz` and 0 for `bnez`: the branch is taken when the two coincide. Unconditional jumps (`j`, `jal`, `jr`, `jalr`) are marked by `is_jump_ex` and are always taken. `JUMP_EN` comes from the control word and guarantees that a non-branch instruction never changes the program flow, even if its first operand happens to be zero. When `take_branch` is asserted, `MUXJUMP` loads the target into the PC. Figure 6.1 shows the logic.

The zero test uses `fwdA`, the forwarded value of the tested register: a branch can therefore depend on the instruction immediately before it, as in a loop closed by `subi` and `bnez`, without any stall.

In a first version the decision was taken in MEM, using a registered copy of the condition. That left three wrong-path instructions in the pipeline, the oldest of which already too close to write back to be cancelled in time. Moving the decision to EX reduced the instructions to cancel to two.

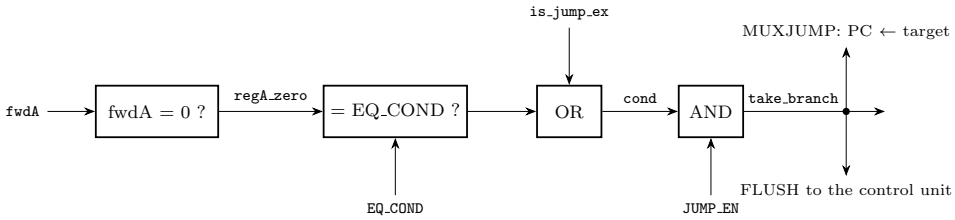


Figure 6.1: Branch decision in the EX stage.

6.1.2 Flush

When the jump decision is taken, the instruction in ID and the one being fetched belong to the wrong path. They are cancelled by replacing their control word with zeros: the instructions keep flowing through the pipeline, but with all the enables deasserted they write neither the register file nor the memory. The control unit receives `take_branch` as `FLUSH`, keeps a delayed copy `flush_d`, and inserts a bubble in both cycles (chapter 4). Table 6.1 shows the resulting timing.

Instruction	c1	c2	c3	c4	c5	c6
`beqz r0, l1`	IF	ID	**EX**	MEM	WB	
`addi r27, r0, ##99`		IF	ID	flushed		
`addi r27, r0, ##98`			IF	flushed		
`l1: bnez r25, l2`				IF	ID	EX

Table 6.1: A taken branch, from `test_all.asm`: decided in EX (c3), the target is fetched in c4 and the two wrong-path instructions become bubbles, so `r27` is never written.

The first cycle cancels the instruction that was in ID, the second one cancels the instruction that was in IF. The branch itself, already in EX, and the older instructions are not affected. A taken branch or jump therefore costs two cycles.

6.1.3 Jump and link

`jal` uses the same mechanism to jump, and in addition writes its return address into `r31`. The return address is `npc_ex` captured when the `jal` is in EX, and is propagated to WB through `npc_mem` and `npc_wb`, which are enabled only by the `jal` itself. In WB, `MUXJAL_DATA` and `MUXJAL_ADDR` select the return address as data and `r31` as destination.

6.1.4 Jumps through a register

`jr` and `jalr` jump to the address contained in a register, as needed to return from a subroutine (`jr r31`) or to call a routine whose address is computed at run time. They reuse the same path as the other jumps: the ALU computes the target as the register content plus a zero immediate, `MUXJUMP` loads it into the PC and the two following instructions are flushed. `jalr` also writes its return address into `r31`, exactly as `jal`.

Since the register is read through `fwdA`, the forwarding unit works for these jumps too: a `jr` can use a register written by the instruction immediately before it, and a `jr` after a load of the same register is stalled for one cycle as any other load-use dependency. A call with `jal` followed by a return with `jr r31` therefore costs two flushed instructions each, with no stall.

6.2 Data hazards

6.2.1 Stall

The first solution implemented was a pure stall. The instruction in ID compares its source registers with the destinations of the instructions in EX, MEM and WB; if a match is found with an instruction that actually writes the register file, the instruction waits. During a stall the control unit:

- deasserts the enables of `PC`, `IR` and `npc_id`, so that the waiting instruction stays in ID and the following one stays in IF;
- inserts a bubble in EX by zeroing `cw1`;
- lets the older instructions proceed normally.

A stall is ignored when a branch is being taken in the same cycle (`stall_eff = STALL and not FLUSH and not flush_d`): the instruction in ID belongs to the wrong path and will be flushed anyway, and the PC must be free to load the target.

With this solution every dependency between adjacent instructions costs up to three cycles. The values, however, are already available inside the pipeline long before they reach the register file. This motivated the introduction of forwarding.

6.2.2 Forwarding

Three mechanisms cover the three possible distances between the instruction that produces a value and the one that uses it (figure 6.3).

Distance	Producer when consumer in EX	Value taken from	Mechanism
1	MEM	`regAlu_out`	forwarding from MEM
2	WB	`wr_data`	forwarding from WB
3	WB, while the consumer is still in ID	`DATAIN` of the RF	register file bypass

Table 6.2: Forwarding sources as a function of the distance between producer and consumer.

Register file bypass. When the instruction in WB writes the same register that the instruction in ID is reading, the register file returns the incoming value instead of the old content (figure 3.2 in chapter 3). This covers distance 3 without any additional logic in EX.

Forwarding unit. The source register numbers are propagated to EX (`rs1_ex`, `rs2_ex`) and compared with the destinations of the instructions in MEM and WB. The most recent value has priority:

```
fwdA <= regAlu_out when (rs1_ex /= "00000" and rs1_ex = regrd_mem
and RF_WE_MEM = '1') else
        wr_data    when (rs1_ex /= "00000" and rs1_ex = wr_addr_wb and RF_WE
= '1') else
        regA_out;
```

Figure 6.2 shows the corresponding hardware. `fwdB` is identical with `rs2_ex` and `regB_out`.

Table 6.3 shows an example: the `add` uses `r2`, produced by the instruction immediately before it, and `r1`, produced two instructions before. Both values reach the ALU in the same cycle, from MEM and from WB, and no cycle is lost.

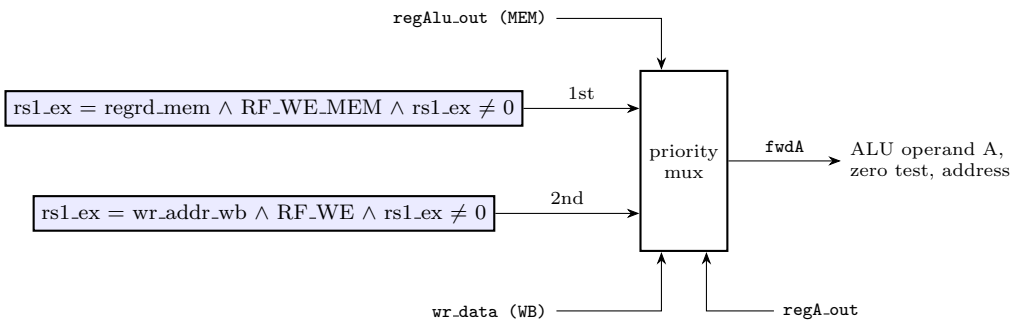


Figure 6.2: Forwarding unit for the first operand.

Instruction	c1	c2	c3	c4	c5	c6
`addi r1, r0, ##12`	IF	ID	EX	MEM	**WB**	
`addi r2, r0, ##5`		IF	ID	EX	**MEM**	WB
`add r3, r1, r2`			IF	ID	**EX**	MEM

Table 6.3: Forwarding without stalls, from the first instructions of `test_all.asm`: in c5 `r2` comes from MEM (`regAlu_out`) and `r1` from WB (`wr_data`).

The forwarded values replace the register operands in three places: the ALU inputs, the zero test of conditional branches, and the data written to memory by `sw`. The comparisons ignore `r0`, and use the write enables of the producers, so that instructions that do not write the register file, such as `sw` and branches, never cause forwarding.

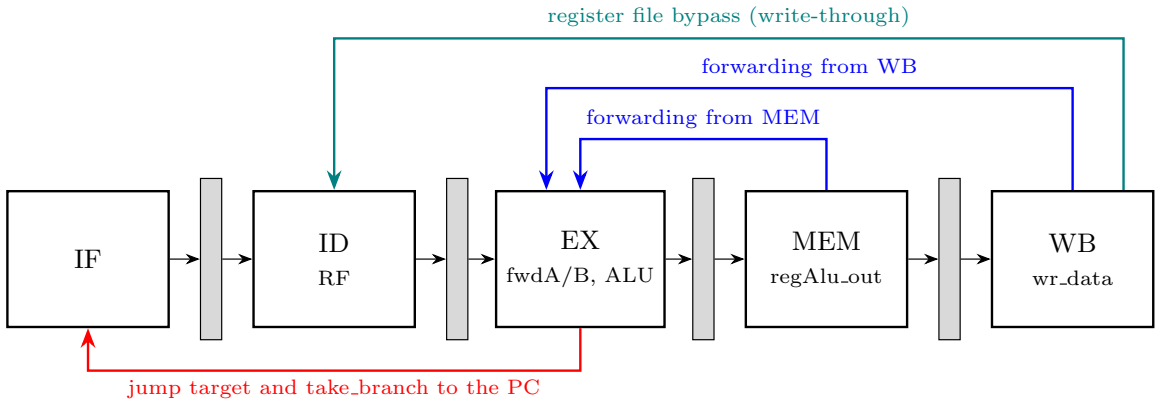


Figure 6.3: Forwarding paths (blue), register file bypass (green) and branch resolution in EX (red).

6.2.3 Load-use hazard

The only case not covered by forwarding is a load followed by an instruction that uses the loaded register. When the consumer is in EX the load is in MEM, and `regAlu_out` contains the address, not the data, which is being read from the memory in that same cycle. The stall logic is therefore reduced to this single case:

```
stall_int <= '1' when is_load_ex = '1' and
      ((rs1 /= "00000" and rs1 = regrd_ex) or
       (rs2 /= "00000" and rs2 = regrd_ex))
   else '0';
```

Figure 6.4 shows the hazard detection logic, and table 6.4 the execution of a load followed by a dependent instruction.

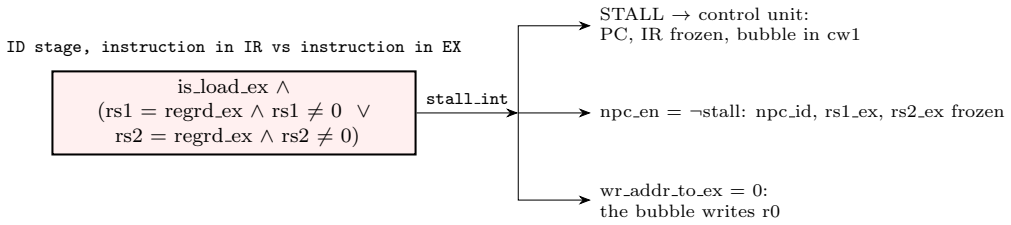


Figure 6.4: Load-use hazard detection, in ID.

Instruction	c1	c2	c3	c4	c5	c6
`lw r24, 0(r0)`	IF	ID	EX	MEM	WB	
bubble				EX	MEM	WB
`add r24, r24, r1`		IF	ID	ID (stall)	EX	MEM

Table 6.4: Load-use hazard, from `test_all.asm`: the `add` stays in ID for one more cycle while a bubble fills EX (c4), then takes `r24` from WB through the forwarding (c5).

After one stall cycle the load is in WB and the data reaches the consumer through the forwarding from WB. The bubble inserted during the stall receives `r0` as destination (`wr_addr_to_ex`), so that it can trigger neither a new stall nor a wrong forwarding.

An alternative would be to forward the memory output directly from MEM, removing the stall completely. This option was tried and discarded: it puts the memory access, the forwarding multiplexer and the ALU in the same cycle, lengthening the critical path considerably. Moreover, in simulation the memory model answers instantaneously, which would hide the real cost of this path.

6.3 Summary of the hazard cases

Table 6.5 lists the action taken by the pipeline for every combination of producer and distance from the consumer. The same actions apply whatever the use of the operand: ALU input, zero test of a branch, or data of a store.

The `jal` row deserves a comment: the two instructions that follow a `jal` are always flushed, so the first instruction of the called routine reaches ID when the `jal` is already in WB, and reads `r31` through the register file bypass.

Two cases are handled conservatively, at the cost of an occasional extra cycle:

Producer	Distance 1	Distance 2	Distance 3	≥ 4
ALU instruction	fwd from MEM	fwd from WB	RF bypass	RF
`lw`	1 stall, then fwd from WB	fwd from WB	RF bypass	RF
`jal`, `jalr` (`r31`)	flushed	flushed	RF bypass	RF
`sw`, branches, `j`, `nop`	no register written: no hazard			

Table 6.5: Action of the pipeline for each data hazard. Distance 1 means that the consumer immediately follows the producer.

- a `sw` that stores the result of the `lw` immediately before it is stalled, although the data would be needed only in MEM. A forwarding path from WB to the data input of the memory would remove this stall;
- the stall logic compares the `rs2` field for every format, and in I-type instructions that field is the destination. An I-type instruction whose destination equals the register loaded by the preceding `lw` is therefore stalled without need. The result is still correct.

6.4 Effect on performance

The test program `test_stalls` contains a store that depends on the instruction immediately before it, and a subtraction that depends on the load immediately before it. Table 6.6 reports, for the three versions of the hazard logic, the time at which the last instruction writes its result, with the reset released at 2 ns and a clock period of 2 ns in simulation.

Hazard handling	Last write back	Cycles
Stall only	31 ns	14
Stall + register file bypass	27 ns	12
Forwarding + load-use stall	21 ns	9

Table 6.6: Execution time of `test_stalls`, counted from the first instruction in ID. The results are identical in the three cases.

The same program, with the same results, completes in 9 cycles instead of 14. The price is paid in hardware and in clock period: the forwarding multiplexers lie on the critical path of the processor, as shown in chapter 9.

CHAPTER 7

Instruction set

7.1 Basic instruction set

Table 7.1 lists the 27 instructions of the basic version, with the encoding of appendix B of the DLX documentation. R-type instructions have opcode `0x00` and are identified by the function field. The instruction formats are summarized in appendix B.

Type	Instruction	Opcode	Func
R	`add`, `sub`	`0x00`	`0x20`, `0x22`
	`and`, `or`, `xor`	`0x00`	`0x24`, `0x25`, `0x26`
	`sll`, `srl`	`0x00`	`0x04`, `0x06`
	`sne`, `sle`, `sge`	`0x00`	`0x29`, `0x2C`, `0x2D`
I	`addi`, `subi`	`0x08`, `0x0A`	–
	`andi`, `ori`, `xori`	`0x0C`, `0x0D`, `0x0E`	–
	`slli`, `srli`	`0x14`, `0x16`	–
	`snei`, `slei`, `sgei`	`0x19`, `0x1C`, `0x1D`	–
	`lw`, `sw`	`0x23`, `0x2B`	–
	`beqz`, `bnez`	`0x04`, `0x05`	–
J	`j`, `jal`	`0x02`, `0x03`	–
–	`nop`	`0x15`	–

Table 7.1: Basic instruction set (27 instructions).

7.2 Extended instruction set

The pro version adds 23 instructions (table 7.2), for a total of 50. They reuse the existing datapath almost entirely: each of them has a row in the control unit table and a case in the ALU opcode decoder; nine of them use an ALU operation of their own, and the two jumps through a register are recognized by the jump logic of the datapath.

Jumps through a register. `jr rs1` sets the PC to the content of `rs1`, `jalr rs1` does the same and saves the return address in `r31`. Together with `jal`, they make subroutine calls and returns possible:

Group	R-type (func)	I-type (opcode)	ALU operation
Signed comparisons	`seq 0x28`, `slt 0x2A`, `sgt 0x2B`	`seqi 0x18`, `slti 0x1A`, `sgti 0x1B`	`SEQS`, `SLTS`, `SGTS`
Unsigned comparisons	`sltu 0x3A`, `sgtu 0x3B`	`sltui 0x3A`, `sgtui 0x3B`	`SLTUS`, `SGTUS`
	`sleu 0x3C`, `sgeu 0x3D`	`sleui 0x3C`, `sgeui 0x3D`	`SLEUS`, `SGEUS`
Arithmetic shift	`sra 0x07`	`srai 0x17`	`ARS`
Unsigned add/sub	`addu 0x21`, `subu 0x23`	`addui 0x09`, `subui 0x0B`	`ADDS`, `SUBS`
Jumps through a register	–	`jr 0x12`, `jalr 0x13`	`ADDS` (target)
Load high immediate	–	`lhi 0x0F`	`LHIS`

Table 7.2: Extended instruction set (23 instructions).

`jal` calls a routine and `jr r31` returns to the instruction after the call. Their implementation is described in section 6.1.

Load high immediate. `lhi rd, imm` loads the 16-bit immediate into the upper half of `rd` and clears the lower half. Since immediates are 16 bits wide, `lhi` followed by `ori` is the way to build any 32-bit constant:

```
lhi r1, ##0x1234   then   ori r1, r1, ##0x5678   →   r1 = 0x12345678
```

Signed and unsigned comparisons. The two families differ only in the interpretation of the operands: the signed versions compare `signed(DATA1)` and `signed(DATA2)`, the unsigned ones `unsigned(...)`. With -3 (`0xFFFFFFFD`) and 5, `slt` returns 1 while `sltu` returns 0, because `0xFFFFFFFD` is a very large unsigned number. This case is checked explicitly by the test programs.

Arithmetic shift. `sra` shifts right replicating the sign bit, so that a negative number stays negative: $-3 \gg 1$ gives -2 (`0xFFFFFFFE`), while the logical shift `srl` would give `0x7FFFFFFE`. It is implemented with the `sra` operator on `bit_vector`, in the same style as the existing logical shifts.

Unsigned additions. In two's complement addition and subtraction produce the same bits regardless of the interpretation of the operands, and the processor does not handle overflow exceptions. `addu` and `subu` are therefore mapped on the existing `ADDS` and `SUBS`.

Zero extension of the immediate. According to the DLX specification, the logical instructions with immediate and the unsigned ones use a 16-bit unsigned immediate. The extension unit fills the upper half with zeros for `andi`, `ori`, `xori`, `addui`, `subui`, `sltui`, `sgtui`, `sleui`, `sgeui`, and with the sign bit for all the other instructions. With an immediate whose bit 15 is set, such as `#32768`, `ori` produces `0x00008000` and not `0xFFFF8000`.

Since the highest opcode used is now `0x3D`, the control unit table has been enlarged to 64 entries.

CHAPTER 8

Verification

8.1 Simulation environment

The verification follows the flow of figure 8.1: an assembly program is assembled and converted into the memory file read by the IRAM, then simulated with Questa Sim; the result is checked on the final content of the register file and of the data memory, and, when needed, on the waveforms.

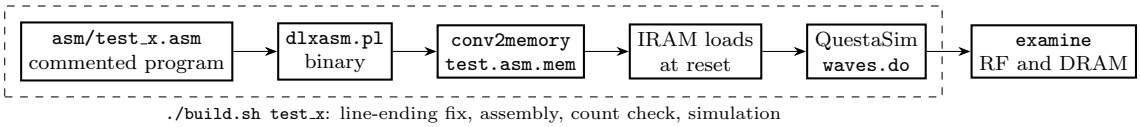


Figure 8.1: Verification flow.

8.1.1 Testbench

The testbench `TB-DLX` instantiates the `DLX`, which contains the two memories, and generates a clock with a period of 2 ns and an active-low reset. The original reset sequence, with two pulses, has been simplified to a single pulse released at 2 ns, so that the program runs from the first cycles without being interrupted; the double pulse remains useful to check that the processor restarts correctly after a reset during execution. The program to be executed is loaded by the IRAM from the file `test.asm.mem` at reset.

8.1.2 Running a simulation

The steps to prepare and run a program have been collected in a small script, `build.sh`. It converts the line endings of the source file (files saved with Windows or old Mac line endings were read by the assembler as a single line), assembles it with `dlxasm.pl`, converts the binary with `conv2memory`, compares the number of non-empty source lines with the number of words in the memory file as a quick sanity check, and finally launches the simulation with a predefined set of waveforms. Labels and comment lines are counted as source lines, so a difference is reported as a warning and is expected for programs with labels.

A complete run is started with a single command, from the project directory:

```
./build.sh test_all
```

The script prints the two numbers and the first lines of the memory file, so that a wrong or stale program is detected before the simulation starts. Instructions must start at the beginning of the line: with the provided assembler, an indented program produces an extra zero word at the beginning of the memory image, which shifts every address by four bytes (appendix A). The waveform setup `waves.do` groups the signals by pipeline stage (IF, ID, EX, MEM, WB) and shows the opcode of the instruction in each stage, which makes stalls and flushes visible at a glance.

The correctness of every program is checked at the end of the simulation by reading the register file and the data memory:

```
examine -radix hex /tb_dlx/U1/DP_I/REG_FILE/REGISTERS
examine /tb_dlx/U1/DRAM_I/DRAM_mem
```

and, for the behaviour of the pipeline, by inspecting the waveforms of the signals of each stage.

8.2 Unit tests

Before being connected in the pipeline, each component has its own testbench, in the folder `test_bench`. Each one applies a sequence of inputs chosen to exercise the behaviour the processor relies on, with the expected values written in comments next to every step and checked on the waveforms (table 8.1).

Testbench	Component	What is checked
`TB-alu`	ALU	all 20 operations of `aluOp`; signed and unsigned comparisons between -3 and 5; logical and arithmetic right shift of -3; `lhi`
`TB-p4adder`	P4 adder	addition and subtraction as done by the ALU (inverted operand, carry input 1); carries crossing one, several and all the 4-bit blocks; carry output and signed overflow
`TB-regfile`	register file	reset; write and read back on both ports; `r0` always zero; write-through bypass in the same cycle; no write and no bypass with `WR` = 0
`TB-cu`	control unit	control signals of every instruction class, `jr`, `jalr` and `lhi` included in ID, EX, MEM and WB; ALU operation decoded from opcode and function field; bubbles inserted by `FLUSH` (two cycles) and by `STALL`, with the fetch enables deasserted
`TB-reg`	pipeline register	load only with `EN` = 1, content kept with `EN` = 0 (stall), asynchronous active-low reset
`TB-mux21`	multiplexer	selection of A with `SEL` = 1 and of B with `SEL` = 0
`TB-iram`	instruction memory	program loaded from `test.asm.mem` during the active-low reset; byte addresses divided by four; addresses wrapping around the 48 words; reload at a second reset
`TB-dram`	data memory	synchronous write and combinational read; negative values; byte addresses with the two least significant bits discarded; no write with `DRAM_WE` = 0

Table 8.1: Unit testbenches.

The datapath is not tested alone: it needs all the control signals of the control unit cycle by cycle, which is exactly what the complete processor provides. It is verified by the integration tests of the following sections, run with `TB-DLX`.

8.3 Test programs

Each test program targets one group of instructions or one mechanism of the pipeline, and is written so that a wrong behaviour leaves a visible trace in the final state (table 8.2). For instance, the instructions that must be cancelled by a flush all write the same register with a recognizable value: if that register is not zero at the end, a flush has failed.

Program	Property proven	Final check
`test_mem`	store and load, also of a negative value; load-use stall	DRAM = 7, −2, 16; r4 = E
`test_alu`	arithmetic, logic and shifts, R-type and immediate	r3–r15
`test_cmp`	signed comparisons ($-3 \geq 5$ false, $-3 \leq 5$ true)	r4–r12
`test_jump`	`beqz` taken, two instructions flushed	r3–r5 = 0, r6 = 7
`test_jump2`	`bnez` taken and `j`	r2, r3, r5, r6 = 0
`test_jal`	`jal`: jump, flush, return address	r31 = 8
`test_branch`	every branch and jump, taken and not taken	r10–r17 = 0, r31 = 0x38
`test_loop`	a loop with loads, accumulation and stores	DRAM[5] = 15
`test_jr`	`jr`, `jalr`: call and return, target from forwarding	r1–r4, r9 = 0, r31 = 0x14
`test_lhi`	`lhi`, 32-bit constants, forwarding of the result	r1 = 0x12345678
`test_stalls`	forwarding at distance 1, load-use stall	r1, r3, r4, DRAM[0]
`test_ext`	comparisons, shifts and unsigned arithmetic of the extended set	r4–r18 = 1
`test_all`	everything above in one program, with a loop	all 32 registers, DRAM[0–2]

Table 8.2: Test programs: property proven and final check. The execution time of `test_stalls` is analyzed in table 6.6.

The early versions of the programs contained `nop` instructions between dependent instructions, to avoid hazards before the stall logic existed. After the introduction of forwarding the programs have been rewritten without them: the only `nop` left are at the end, to let the last instructions complete.

Every program ends either with a self-loop (`done: j done`) or with a few `nop`. This matters because the instruction memory is addressed modulo its 48 words: after the last instruction the processor reads empty memory and, at word 48, starts the program again. For most programs a second execution writes the same values, but a program with a `jal` would rewrite `r31` at every pass, so `test_jal` and `test_branch` end with a self-loop, which makes the final state independent of the length of the simulation.

8.4 Description of the test programs

Following the project guide, for each program the operations used, the property proven and the commented final state are reported. Register values are in hexadecimal, as shown by `examine -radix hex`.

8.4.1 `test_mem`: store and load

- **Operations**: `addi`, `sw`, `lw`, `add`, `sub`.
- **Property**: a value written to memory is read back unchanged, also when it is negative; a load followed by an instruction that uses its result (`lw r3` then `add r4, r3, r3`) is handled by the load-use stall; a computed value is stored again.

- **Final state**: r1 = `7` and r2 = `FFFFFFFE` (-2) are stored in DRAM[0] and DRAM[1] and loaded back into r3 = `7` and r5 = `FFFFFFFE`; r4 = `E` ($7+7$, after the stall); r6 = `10` ($14-(-2)=16$), stored in DRAM[2]. Memory: 7, -2, 16.

8.4.2 `test_alu`: arithmetic, logic and shifts

- **Operations**: `add`, `sub`, `and`, `or`, `xor`, `sll`, `srl` and the immediate versions `addi`, `subi`, `andi`, `ori`, `xori`, `slli`, `srli`, on the operands 12 and 5.
- **Property**: every arithmetic, logic and shift operation, R-type and immediate, returns the expected result, with operands produced by the instructions immediately before it.
- **Final state**: r1 = `C`, r2 = `5`; r3 = `11` ($12+5$), r4 = `C` ($17-5$), r5 = `4` ($12 \wedge 5$), r6 = `D` ($12 \vee 5$), r7 = `9` ($12 \oplus 5$); r8 = `A` ($12-2$), r9 = `4` ($12 \wedge 6$), r10 = `F` ($12 \vee 3$), r11 = `3` ($12 \oplus 15$); r12 = `A0` ($5 \ll 5$), r13 = `5` (`A0` $\gg 5$), r14 = `28` ($5 \ll 3$), r15 = `A` (`28` $\gg 2$).

8.4.3 `test_cmp`: signed comparisons

- **Operations**: `sge`, `sle`, `sne` and the immediate versions `sgei`, `slei`, `snei`, with a negative operand.
- **Property**: the comparisons are signed, so that $-3 \geq 5$ is false and $-3 \leq 5$ is true; the result, 1 or 0, is written into the destination register.
- **Final state**: r3 = `FFFFFFFD` (-3); r4 = `1` ($12 \geq 5$), r5 = `0` ($12 \leq 5$), r6 = `1` ($12 \neq 5$), r7 = `0` ($-3 \geq 5$), r8 = `1` ($-3 \leq 5$), r9 = `0` ($12 \neq 12$), r10 = `1` ($12 \geq 12$), r11 = `1` ($-3 \leq -3$), r12 = `0` ($5 \neq 5$).

8.4.4 `test_jump` and `test_jump2`: branches and jumps

- **Operations**: `beqz` taken, `bnez` taken, `j`.
- **Property**: a taken branch changes the flow, and the two instructions that follow it are cancelled.
- **Final state**: in `test_jump`, r3–r5 = `0` because the instructions writing them were flushed, while r6 = `7` is written at the target. In `test_jump2`, the flushed registers r2, r3, r5, r6 are `0`, while r4 = `1` and r7 = `2` are written on the taken paths.

8.4.5 `test_jal`: jump and link

- **Operations**: `jal`, `addi`.
- **Property**: `jal` jumps, flushes the two following instructions and writes the return address into `r31`.
- **Final state**: r2 = r3 = `0` (flushed), r5 = `3` (written in the called routine), r31 = `8`, the address of the instruction after the `jal`, which is at address 4.

8.4.6 `test_branch`: all branches and jumps

- **Operations**: `beqz` and `bnez`, taken and not taken, `j`, `jal`.
- **Property**: every taken branch or jump cancels the two instructions that follow it, while a branch not taken continues in sequence.

- **Final state**: r1 = 3; r3 = 1, written because `beqz r1` is not taken; r4 = 2, written in the called routine; r10–r17 = 0, because every instruction writing them follows a taken branch or jump; r31 = 38, the address after the `jal`, which is at address `0x34`.

8.4.7 `test_loop`: a loop over memory

- **Operations**: `addi`, `sw`, `lw`, `add`, `subi`, `bnez`.
- **Property**: a loop reads five words from memory and accumulates them; inside the loop a load is followed by an instruction that uses it (load-use stall), and `bnez` depends on the `subi` immediately before it (forwarding to the zero test).
- **Final state**: DRAM[0–4] = 1, 2, 3, 4, 5 and DRAM[5] = 15, their sum; r4 = `F`, r3 = `0`, r2 = `14`, the address after the last word read.

8.4.8 `test_jr`: calls and returns through a register

- **Operations**: `jal`, `jr`, `jalr`, `addi`, `j`.
- **Property**: a routine called with `jal` returns with `jr r31` to the instruction after the call; `jalr` jumps to an address computed by the instruction immediately before it, which reaches the jump through forwarding; all the instructions after the jumps are flushed.
- **Final state**: r1 = `A` (the routine is executed once), r2 = `B` (the return reached the instruction after the call), r3 = `3` (`jalr` reached the second routine), r4 = `4` (second return), r9 = `0` (every instruction writing r9 is flushed), r10 = `30`, the address 48 of the second routine, r31 = `14`, the return address of `jalr` (20).

8.4.9 `test_lhi`: 32-bit constants

- **Operations**: `lhi`, `ori`, `addi`, `add`.
- **Property**: `lhi` loads the upper half and clears the lower one; together with `ori` it builds a 32-bit constant, with the value forwarded from `lhi` to the instruction immediately after it.
- **Final state**: r1 = `12345678`, r2 = `80000000`, r3 = `80000001`, r4 = `00010000`, r5 = `12355678`.

8.4.10 `test_stalls`: data hazards

- **Operations**: `addi`, `sw` depending on the instruction before it, `lw` followed by a `subi` that uses the loaded value.
- **Property**: forwarding at distance 1 needs no stall; the load-use case costs exactly one stall cycle. This is the program chosen to underline pipeline stalls, as asked by point 1 of the guide.
- **Final state**: r1 = `C`, r3 = `7`, r4 = `C`, DRAM[0] = 12. The last write back happens at 21 ns with forwarding, against 31 ns with stalls only (table 6.6).

8.5 Global test

`test_all` (appendix C) is the reference program. It contains exactly 48 instructions, the capacity of the IRAM, and exercises:

- all the arithmetic, logic and shift instructions, each one depending on the previous results, so that most operands come from the forwarding paths;
- the comparisons, including the signed comparison between -3 and 5;
- stores and loads, with a load immediately followed by an instruction that uses its result;
- three taken branches and one not taken, and a `jal`;
- a loop whose `bnez` depends on the instruction immediately before it.

At the end of the program the processor executes `j done` on itself, so that the final state remains stable. Table 8.3 compares the expected values with the ones obtained in simulation.

Registers	Expected	Obtained
r1 – r7	`C, 5, 11, C, 4, D, 9`	identical
r8 – r15	`A0, 5, A, 4, F, 3, 28, A`	identical
r16	`FFFFFFFD`	identical
r17 – r23	`1` (seven comparisons true)	identical
r24, r25	`1D`, `FFFFFFFD`	identical
r26	`1` (branch not taken)	identical
r27	`0` (all flushed instructions cancelled)	identical
r28 – r30	`7, 0, 6`	identical
r31	`98` (return address of `jal`)	identical
DRAM[0 – 2]	17, -3, 29	identical

Table 8.3: Final state of `test_all`.

8.6 Test of the extended instruction set

`test_ext` checks the new comparisons, shifts and unsigned operations. The operands are chosen so that a wrong interpretation of the sign or a wrong extension of the immediate gives a different result (table 8.4). All the values obtained in simulation match the expected ones.

Instruction	Expected	Value with the wrong interpretation
`slt r7, r3, r2` ($-3 < 5$)	1	0 if unsigned
`sltu r8, r2, r3` ($5 <$ `0xFFFFFFFD`)	1	0 if signed
`sra r19, r3, r2` ($-3 \gg 5$)	`FFFFFFFF`	`07FFFFFF` if logical
`srai r20, r3, #1` ($-3 \gg 1$)	`FFFFFFFE`	`7FFFFFFE` if logical
`addui r23, r2, #40000`	`9C45`	`FFFF9C45` if sign-extended
`ori r25, r0, #32768`	`8000`	`FFFF8000` if sign-extended

Table 8.4: Selected results of `test_ext`.

8.7 Waveforms

Figures 8.2 and 8.3 show two significant moments of the execution.

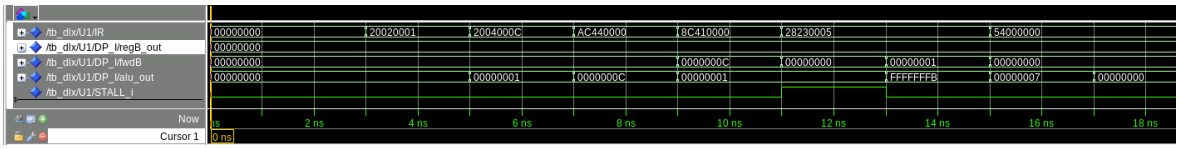


Figure 8.2: Forwarding and load-use stall in `test_stalls`: `fwdB` differs from `regB_out` while the `sw` is in EX, and `STALL` is high for one cycle after the `lw`.

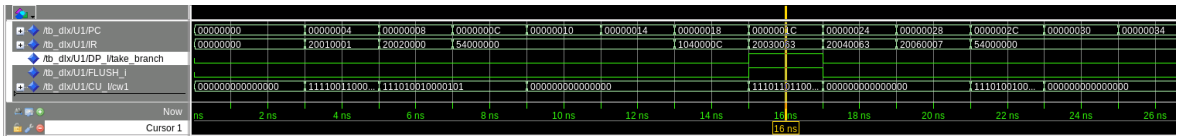


Figure 8.3: Flush after a taken branch: `FLUSH` is high, `cw1` is zero for two cycles and the PC is loaded with the target.

CHAPTER 9

Synthesis

9.1 Setup

The synthesis of both the datapath and the control unit, with a final optimization for frequency and commented scripts, is point 4 of the project guide, mandatory for every version. Figure 9.1 summarizes the implementation flow described in this chapter and in the next one.

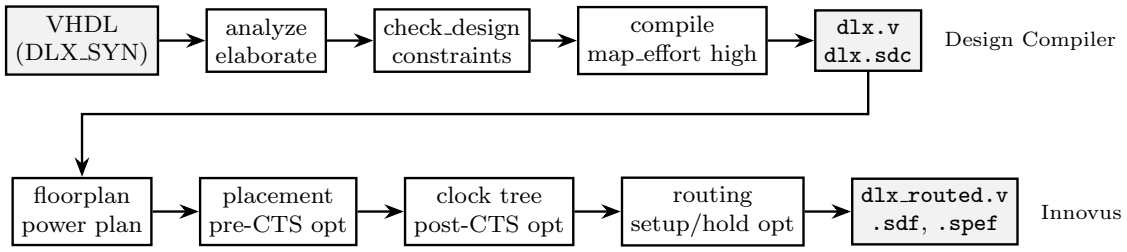


Figure 9.1: Synthesis and physical design flow.

The synthesis has been performed with Synopsys Design Compiler, mapping on the typical corner of the NanGate 45 nm Open Cell Library with the wire load model `5K_hvratio_1_4`.

Synthesis top level. The instruction and data memories are behavioral models that read their content from a file, and they are not meant to be synthesized. A dedicated top level, `DLX_SYN`, has been written for synthesis: it is identical to `DLX`, but the two memories are removed and their interfaces become ports of the entity (`IRAM_ADDR`, `IRAM_IN`, `DRAM_ADDR`, `DRAM_DIN`, `DRAM_IN`, `DRAM_WE`). The synthesized block therefore contains control unit, datapath, PC and IR.

9.2 Synthesis script

The complete, commented script is reported in appendix D. It is parametric in the target clock period, so that the same script is used for all the runs:

```
for P in 10 1.0 1.4 1.5 1.6; do
  dc_shell -x "set PERIOD $P; source dlx.scr" | tee syn_$P.log
done
```

Its steps are the following:

1. **Analysis** (`analyze`) of the VHDL sources, read directly from the project directory, so that synthesis and simulation always use the same files. The package is analyzed first, then the components, the datapath, the control unit and the synthesis top level. The memories are not in the list.
2. **Elaboration** (`elaborate`) of `DLX_SYN`, followed by `check_design`, which reports undriven nets, unconnected ports and constant outputs. This check was added after the problem described in section 9.4.
3. **Constraints**: a clock of period `PERIOD` on the `Clk` port (`create_clock`), which bounds every register-to-register path; the same value as maximum delay between inputs and outputs (`set_max_delay`); the wire load model `5K_hvratio_1_4` of the laboratories.
4. **Compilation** with `compile -map_effort high`.
5. **Reports**: worst path, ten worst endpoints, total area, area per hierarchical block, power. Each file carries the period in its name.
6. **Outputs for place & route**: `change_names -rules verilog` makes the names compatible with Innovus; then the hierarchical Verilog netlist and the SDC constraints are written.

9.3 Running the synthesis and generated files

The synthesis is run in batch mode from the `syn` directory, one run per clock period:

```
cd syn
dc_shell -x "set PERIOD 1.6; source dlx.scr" | tee syn_1.6.log
```

The list of analyzed files includes the components of the P4 adder, in bottom-up order, before the ALU. Each run produces the following files, all carrying the period in their name:

- `syn_<P>.log`: the complete log of Design Compiler;
- `reports/check_<P>.rpt`: the result of `check_design`;
- `reports/timing_<P>.rpt` and `reports/timing_top10_<P>.rpt`: worst path and ten worst endpoints;
- `reports/area_<P>.rpt` and `reports/area_hier_<P>.rpt`: total area and area per block;
- `reports/power_<P>.rpt`: power estimate;
- `netlist/dlx_<P>.v` and `netlist/dlx_<P>.sdc`: netlist and constraints for place & route.

9.4 A synthesis pitfall: initial values of signals

The first complete run of the flow revealed a problem that simulation could not show. The table of control words of the control unit was declared as a *signal* with an initial value:

```
signal cw_mem : mem_array := ( 0 => "111100110000101", ... );
```

In simulation the initial value is used and everything works. Design Compiler, instead, ignores initial values of signals, because real hardware has no "time zero" in which they can be imposed. Since nothing else drives `cw_mem`, the table was synthesized as all zeros, and in the netlist every control output of the control unit was tied to a constant:

```
assign RegA_LATCH_EN = 1'b0;
assign RF_WE_WB      = 1'b0;
assign DRAM_WE       = 1'b0;   ...
```

The problem was not visible in the synthesis reports. Design Compiler optimizes each level of hierarchy separately and does not propagate constants across the boundary between control unit and datapath, so the datapath remained complete and the area looked reasonable. It emerged during place & route: Innovus works on the flattened netlist, found that the registers A and B were never loaded and that the register file was never written, and removed 4 106 cells, among which 1 045 flip-flops, during the *Simplify Netlist* optimization. The layout was almost empty.

The fix is to declare the table as a *constant*, which is the correct description of a fixed lookup table and is synthesized as combinational logic. Since then, `check_design` is part of the synthesis script, and the netlist is checked for constant outputs of the control unit before moving to place & route. All the results that follow refer to the corrected design.

9.5 Control unit after synthesis

The synthesized control unit contains 21 flip-flops. The control word registers would require 38 bits, but Design Compiler removed the bits that are never read: in EX only 9 bits of `cw1` are used (the five EX signals plus those propagated to MEM), in MEM only 4 bits of `cw2`, in WB only the 2 bits of `cw3`. The remaining flip-flops store the ALU operation (5) and `flush_d` (1). This also shows that the control word could be made shorter, since several of its bits are no longer used after the introduction of the stall and flush logic.

9.6 Search of the maximum frequency

A first run with a loose constraint (10 ns) gives the reference, not optimized, circuit. The clock period has then been reduced progressively. Table 9.1 reports the results.

Target period [ns]	Slack [ns]	Critical path [ns]	Area [μm^2]	Dynamic power [mW]	Leakage [μW]
10.0	+6.81	3.19	16 055	1.18	311.7
1.7	0.00	1.70	16 423	7.00	320.7
1.6	−0.02	1.62	16 297	7.41	315.8
1.5	−0.11	1.61	16 580	7.92	322.0
1.4	−0.11	1.51	16 742	8.49	325.9
1.0	−0.48	1.48	17 337	11.95	340.2

Table 9.1: Synthesis results as a function of the target clock period.

The results show a typical behaviour of the synthesis tool: the circuit obtained depends on the constraint. With a tighter target, Design Compiler uses larger and faster cells and restructures the logic more aggressively; with a looser one it stops as soon as the constraint is close. For this reason the fastest circuit (1.48 ns) was obtained with an unreachable target of 1.0 ns, while relaxing the target produced slower circuits.

Two values are therefore significant:

- the **maximum frequency reached**, about $1/1.48\,ns \approx 676$ MHz, obtained by over-constraining the synthesis;
- the **frequency met with non-negative slack**, $1/1.7\,ns \approx 588$ MHz, which is the netlist used for place & route.

With respect to the reference circuit, whose critical path is 3.19 ns (313 MHz), the optimization makes the processor about 2.2 times faster, for an area increase of 8% in the most aggressive case and of 2.3% for the circuit met at 1.7 ns. The area changes little because a large part of it is sequential: the 1 491 flip-flops occupy about $7\,100\,\mu m^2$, more than 40% of the total, and they can be neither restructured nor made faster by the synthesis tool.

9.7 Constraints and timing paths

The timing paths of a synchronous design fall into four groups, and each group is bounded by a different constraint:

- **register to register**: bounded by the clock (`create_clock`), which requires every path between two flip-flops to fit in one period, minus the setup time;
- **input to register** and **register to output**: also bounded by the clock, but only relative to the clock edge, since no input or output delays are specified; in this design they are the paths from and to the memory interface;
- **input to output**, purely combinational: not touched by the clock, and bounded by `set_max_delay` with the same value as the period.

With a loose constraint, Design Compiler maps the logic with small cells and minimal area. When the constraint tightens, it reduces the delay of the critical paths in three ways: it replaces cells with stronger versions of the same function (higher drive strength, e.g. `X2`, `X4`), inserts buffers to split large fanouts, and restructures the logic, including choosing faster architectures for the DesignWare adders, subtractors and comparators. This is why the area grows only on the paths that need it.

9.8 Synthesized design

Figure 9.2 shows the top level of the synthesized netlist: the control unit and the datapath are both synthesized, as required by point 4 of the guide, and remain separate hierarchical blocks.

Table 9.2 reports the area of the main blocks in the netlist synthesized at 1.7 ns (`report_area -hierarchy`).

The register file alone takes 56.5% of the processor. Half of its area is sequential, the 1 024 flip-flops ($4\,631\,\mu m^2$), and half is combinational ($4\,650\,\mu m^2$): the two 32-to-1 read multiplexers, 32 bits wide, the write decoder and the bypass logic. In a real design a register file of this size would be a full-custom memory macro rather than standard cells.

Inside the ALU, the P4 adder keeps the structure described in the VHDL: the synthesis tool can resize its cells but not change its architecture. The shifters and the comparators, described behaviorally, are mapped by Design Compiler on the components of its DesignWare library, one per operator. In a 32-bit pipeline register, $170\,\mu m^2$ out of 235 are the 32 flip-flops; the rest is the enable logic.

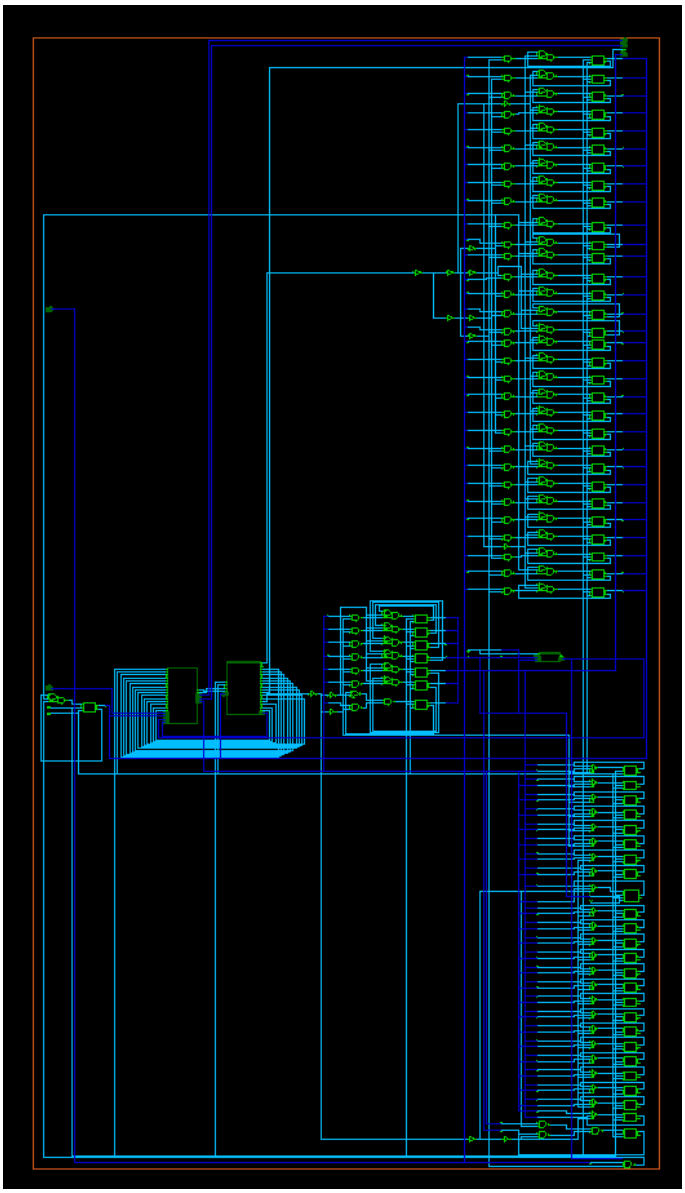


Figure 9.2: Top level of the synthesized DLX, as shown by Design Vision. The large block is the datapath (`DP_I`), with most of the pins: control signals, instruction, program counter and the data memory ports. The small block is the control unit (`CU_I`), which receives the instruction and the `FLUSH` and `STALL` signals and drives the control signals. The remaining cells form the fetch stage: PC, IR and the $PC + 4$ adder.

Block	Area [μm^2]	Share
`DLX_SYN`, total	16 423.4	100%
control unit (`CU_I`)	250.0	1.5%
register file	9 280.5	56.5%
ALU	2 645.4	16.1%
of which P4 adder	919.8	5.6%
each 32-bit pipeline register	≈ 235	1.4%
each 32-bit 2-to-1 multiplexer	≈ 65	0.4%

Table 9.2: Area of the main blocks after synthesis at 1.7 ns.

9.9 Critical path

In the optimized runs the critical path ends in the program counter. At 1.6 ns it starts in the WB stage:

```
is_jal_wb / regrd_wb   (WB)  registers that select the destination
  -> wr_addr_wb              destination in WB (r31 for jal and jalr)
  -> forwarding comparison   wr_addr_wb = rs1_ex ?            (EX)
  -> fwdA                    forwarding multiplexer
  -> ALU / regA_zero         target computation, zero test
  -> take_branch             jump decision
  -> MUXJUMP -> PC_reg       next PC                          (IF)
```

Depending on the run, the timing report names as startpoint `regrd_wb` or `is_jal_wb`: both feed the multiplexer that forms `wr_addr_wb`, so they are the same path. With the tightest constraint (1.0 ns) the path starts instead from `rs2_ex`, the other side of the same comparison, and follows the same chain. This is the longest chain the pipeline can produce: a value selected in WB crosses the forwarding logic, enters the ALU or the zero test of a branch, decides whether to jump and selects the next PC, all in the same cycle. The path is the direct consequence of two design choices, forwarding and branch resolution in EX, which both reduce the number of lost cycles but lengthen the cycle. Resolving branches in MEM would shorten this path at the cost of one more flushed instruction per taken branch.

The cells of the P4 adder do not appear on the critical path at 1.6 ns: the frequency is limited by the control of the pipeline, not by the arithmetic.

9.10 Power

The power reported by Design Compiler (table 9.1) is estimated with the default switching activity, at the frequency of the constraint. For the netlist at 1.7 ns it is 7.00 mW of dynamic power and 0.32 mW of leakage.

The dynamic power grows almost exactly in proportion to the frequency: 1.18 mW at 100 MHz and 7.00 mW at 588 MHz correspond to 11.8 and 11.9 μW/MHz, i.e. to an energy of about 11.9 pJ per clock cycle in both cases. This is the expected behaviour of the dynamic power, $P_{dyn} = \alpha\, C\, V_{DD}^2\, f$, when the switched capacitance does not change much. The leakage, instead, depends on the cells and not on the frequency, and grows only slightly (+9% between the reference and the 1.0 ns circuit) because faster circuits use larger cells.

CHAPTER 10

Physical design

The synthesized netlist must be placed and routed (point 5 of the project guide). The place & route has been performed with Cadence Innovus on the netlist synthesized at 1.7 ns, following the flow of the course laboratory (figure 9.1). All the steps are collected in the script `dlx_pnr.tcl` (appendix D), run from the Innovus shell with `source dlx_pnr.tcl`.

10.1 Design setup and configuration

The design is imported with `init_design`, reading the configuration file `DLX.globals`, adapted from the laboratory:

- the netlist `dlx.v` and the top cell `DLX_SYN`;
- the LEF file of the NanGate 45 nm library, with the physical description of cells and metal layers;
- the power nets `vdd` and `gnd`;
- the multi-mode multi-corner file `DLX.view`, which loads the constraints `dlx.sdc` written by Design Compiler and defines three corners: typical, worst (slow library, high temperature) and best (fast library, low temperature). Setup and hold are analyzed on the typical corner, as in the laboratory.

10.2 Floorplanning

The floorplan fixes the shape of the chip: aspect ratio 1.0, core utilization 0.6, 5 μm margins between core and die. A utilization of 0.6 leaves 40% of the core free for the buffers inserted by the optimizations and for routing.

10.3 Power planning

Power rings surround the core on metal 9 (horizontal) and metal 10 (vertical), with width, spacing and offset of 0.8 μm. Vertical stripes on metal 10, every 20 μm, distribute the supply across the core. Finally `sroute` connects the supply rails of the standard-cell rows to rings and stripes.

10.4 Standard-cell placement

The cells and the I/O pins are placed with `placeDesign`, limiting the routing to metal 6. The pins had to be placed explicitly (`-place_global_place_io_pins true`): without this option the ports of the design remained unplaced. After placement, a first timing analysis and an optimization (`optDesign -preCTS`) are run.

10.5 Clock tree synthesis

10.5.1 The problem

The processor contains 1 491 flip-flops, and all of them must receive the clock edge at the same instant. In synthesis the clock is *ideal*: Design Compiler assumes that it reaches every flip-flop instantly and simultaneously, at no cost. In reality the clock enters the block from a single port, which cannot drive 1 491 inputs: the edge would be very slow, and it would reach the flip-flops close to the port much earlier than the distant ones.

10.5.2 The clock tree

The clock tree synthesis (CTS) builds the real distribution network: a tree of buffers in which the clock port drives a few buffers, each of them drives a few more, down to the leaves, which are the flip-flops (figure 10.1). The tool chooses number, size and position of the buffers, and balances the branches so that all the paths from the port to the flip-flops have about the same delay.

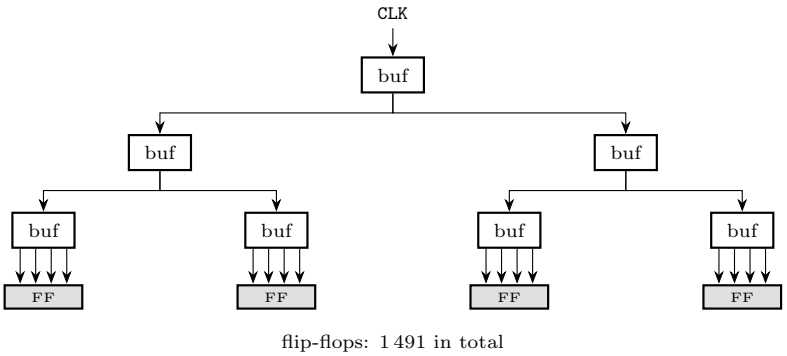


Figure 10.1: Principle of a clock tree: the clock is distributed through levels of buffers, balanced so that it reaches all the flip-flops at the same time.

10.5.3 Implementation

The step was absent from the laboratory script, written for a purely combinational adder, and has been added after the pre-CTS optimization, using the CCOpt engine of Innovus:

```
create_ccopt_clock_tree_spec               # derive the tree from the constraints
set_ccopt_property target_max_trans 0.05   # 50 ps maximum transition
set_ccopt_property target_skew 0.02        # 20 ps maximum skew
clock_opt_design                           # build and optimize the tree
timeDesign -postCTS -outDir timing_postCTS
optDesign -postCTS
```

The two targets are the quality parameters of the tree:

- the **skew** is the difference between the arrival times of the clock at the earliest and at the latest flip-flop. It must be small, because if two registers receive the clock at different instants, the time available for the data between them changes: a positive skew eats the setup margin of some paths, a negative one the hold margin of others;
- the **transition** is the time the clock edge takes to rise or fall. Slow edges make the flip-flops less precise and increase the short-circuit power.

10.5.4 Effects on the results

The clock tree is visible in three results of this chapter:

- **timing**: after the CTS the setup slack remains positive (+0.090 ns, table 10.1), although the clock now reaches the flip-flops through a real network with its own delay;
- **hold**: the clock reaches the registers about 0.2 ns late through the tree, while the data from the memory ports are assumed to change at the clock edge; this is the origin of the hold violations on the input paths (section 10.13);
- **power**: the tree switches at every cycle, with the maximum activity, and is one of the main reasons why the dynamic power almost doubles from synthesis to layout (section on power).

This covers the clock tree synthesis requested by point 14 of the project guide. The other two activities of the same point, a simulation of the netlist after place & route with the extracted delays and a crosstalk analysis, have not been performed: the timing analysis was run without signal-integrity effects, i.e. without the coupling capacitances between adjacent wires.

10.6 Routing

NanoRoute connects all the nets, up to metal 6, with a global and a detailed routing step. The timing is then analyzed with the real wires, and a post-route optimization is run for setup and then for hold. The hold fixing inserts delay buffers on the paths that are too fast, and therefore needs free space in the rows (section 10.7).

10.7 Filler cells

Filler cells contain no logic: they fill the gaps left in the standard-cell rows, so that the supply rails and the doped wells of the silicon are continuous, as required to manufacture the chip. Two details of their insertion turned out to matter.

The list of cells. The command of the laboratory script, `addFiller -prefix FILL`, does not list the filler cells to use: Innovus reports *No filler cell provided* and inserts nothing, without stopping the script. The cells must be listed explicitly; in the NanGate library they are `FILLCELL_X1` to `FILLCELL_X32`.

The position in the flow. Fillers occupy all the free space of the rows: after their insertion the occupation is 100%. If they are inserted before the hold optimization, there is no room left for the delay buffers, and the hold violations cannot be fixed. In this design, with the fillers already in place, the post-route hold optimization left 468 violating paths; after removing the fillers (`deleteFiller`), the same optimization fixed all of them. The fillers are therefore inserted at the very end, after the last optimization, followed by `ecoRoute` to fix the conflicts with the existing wires:

```
optDesign -postRoute
optDesign -postRoute -hold
addFiller -cell {FILLCELL_X32 FILLCELL_X16 FILLCELL_X8
                 FILLCELL_X4 FILLCELL_X2 FILLCELL_X1} -prefix FILL
ecoRoute -target
```

About 14 200 filler cells are inserted, bringing the row occupation from 55.1% to 100%. The timing, checked again after the insertion, is unchanged, as expected for cells without logic.

10.8 Parasitic extraction and outputs

The resistances and capacitances of the routed wires are extracted (`extractRC`) and saved; the connectivity is verified, and the reports, the routed netlist and the delay file are written.

10.8.1 Generated files

- `dlx_routed.v`: the netlist after place & route, including clock-tree buffers, optimization cells and fillers.
- `dlx.spef`: the Standard Parasitic Exchange Format file, which lists for every net the resistances and capacitances of the real wires extracted from the layout. It is the base for an accurate timing and power analysis.
- `dlx_routed.sdf`: the Standard Delay Format file with the delay of every cell and net, which can be back-annotated in a post-layout simulation of the netlist.

10.9 Timing before and after the optimization steps

The project guide asks for the performance before and after the optimization steps. The timing has therefore been analyzed (`timeDesign`) after every main step of the flow, at the clock period of 1.7 ns; table 10.1 reports the worst negative slack (WNS) and the number of violating paths.

Step	WNS [ns]	Violating paths
Setup		
After placement, before optimization	−0.433	105
After pre-CTS optimization and clock tree synthesis	+0.090	0
After routing, before optimization	+0.165	0
After post-route optimization and fillers	+0.165	0
Hold		
Post-route hold optimization, fillers in place	−0.205	468
Post-route hold optimization, fillers removed	0.000	0

Table 10.1: Slack along the physical design flow, clock period 1.7 ns.

The table shows the role of each optimization:

- **After placement** the netlist misses the target by 0.433 ns on 105 paths. The cells chosen by Design Compiler were sized for the wire load model; with the cells placed, Innovus estimates the wires from their real positions, and the longest paths no longer fit in the period.
- The **pre-CTS optimization** resizes cells and inserts buffers on the critical paths, and recovers all the violations: after the clock tree synthesis the slack is positive, +0.090 ns, although the clock now reaches the flip-flops through a real tree.
- **After routing** the slack improves further to +0.165 ns, and the post-route optimization keeps it. The estimate before routing was pessimistic, as is the wire load model in synthesis.
- The **hold** check is closed last. All its violations are on paths starting from the input ports (section 10.13); they are fixed by delay buffers, once the space occupied by the fillers is released.

10.10 Results

Check	WNS	TNS	Violating paths
Setup, all paths	+0.165 ns	0	0
Setup, register to register	+0.165 ns	0	0
Hold, all paths	0.000 ns	0	0
Hold, register to register	+0.071 ns	0	0
Design rules (max transition, capacitance, fanout, length)		0 violations	

Table 10.2: Timing after place & route, clock period 1.7 ns.

The design meets the 1.7 ns constraint after layout with a setup slack of +0.165 ns, which corresponds to an effective critical path of about 1.54 ns, or 651 MHz. The slack after layout is better than the one obtained in synthesis (0.00 ns). This is explained by two factors: the wire load model used in synthesis is a statistical estimate, pessimistic for a block of this size; and Innovus re-optimizes the critical paths knowing the real placement and the real wires.

Block	Cells	Area [μm^2]	Share
`DLX_SYN` (total)	7 787	15 094.2	100%
Control unit	159	246.1	1.6%
Datapath	7 334	14 258.4	94.5%
of which ALU	1 791	2 081.7	13.8%
of which P4 adder	476	541.8	3.6%
of which carry generator	163	170.8	1.1%

Table 10.3: Cell area after place & route, fillers excluded.

The hardwired control unit occupies less than 2% of the area: the cost of the processor is in the datapath, and in particular in the storage elements. The P4 adder takes less area after place & route (541.8 μm^2) than after synthesis (919.8 μm^2), because Innovus resizes its cells to the smallest ones that still meet the timing, now that the real delays are known; the carry generator, the fast part of the adder, takes less than a third of it.

The die measures $176.1 \times 175.3\,\mu m$, i.e. about $30\,900\,\mu m^2$ ($0.031\,\text{mm}^2$). Removing the $5\,\mu m$ margins, the core is $166.1 \times 165.3\,\mu m$, about $27\,500\,\mu m^2$: with the requested utilization of 0.6 this corresponds to the $16\,400\,\mu m^2$ of cells of the synthesized netlist, as expected.

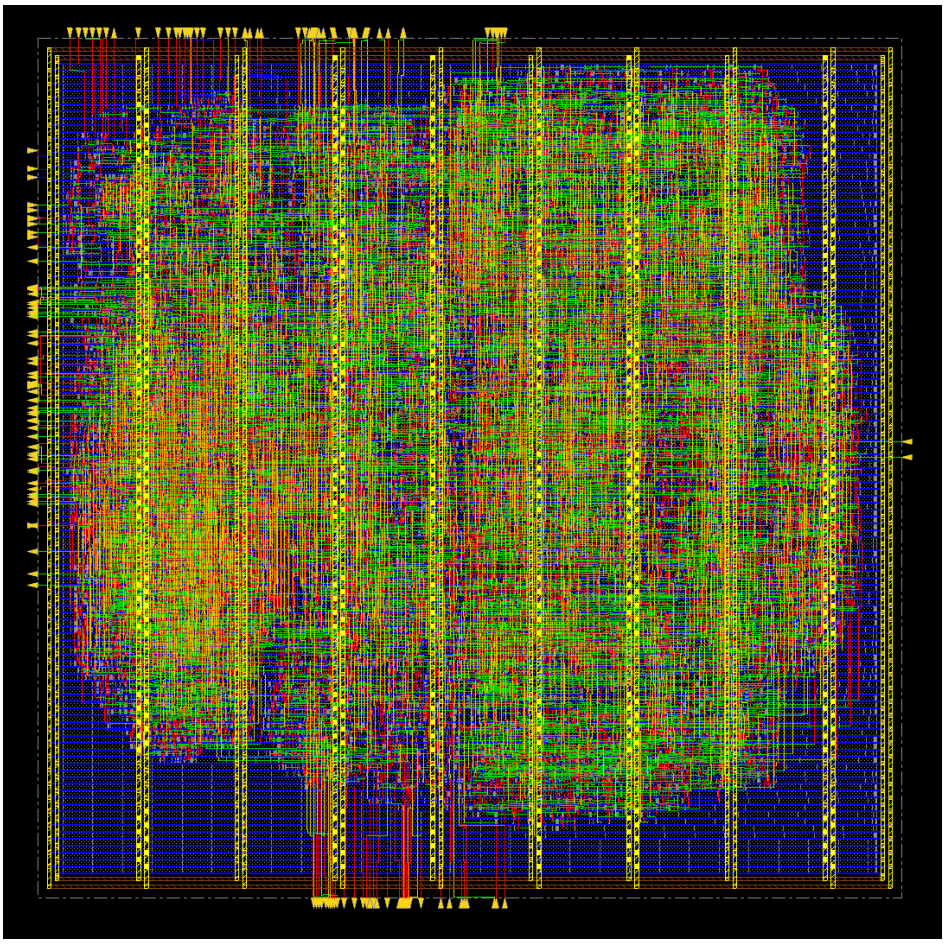


Figure 10.2: Layout of the DLX after place & route, with the filler cells.

10.11 Power

Table 10.4 reports the power estimated by Innovus on the routed design, compared with the estimate of Design Compiler for the same netlist.

The leakage is practically unchanged, since it depends only on the cells. The dynamic power, instead, almost doubles. The main reasons are:

- the **clock tree**: in synthesis the clock is ideal and has no cost, while after clock tree synthesis the buffers that distribute the clock to 1 491 flip-flops switch at every cycle, with the maximum activity;
- the **real interconnect**: the switching power charges the capacitance of the wires, which in synthesis is only estimated by the wire load model;
- the **cells added** by the optimizations, including the buffers inserted to fix the hold violations.

	Synthesis [mW]	Place & route [mW]
Dynamic power (internal + switching)	7.00	12.89
of which internal	–	8.18 (62.0%)
of which switching	–	4.71 (35.8%)
Leakage power	0.32	0.29 (2.2%)
Total	7.32	13.18

Table 10.4: Power at 588 MHz, after synthesis and after place & route. Innovus splits the dynamic power into internal power, dissipated inside the cells, and switching power, needed to charge the load capacitances.

The two tools also use default switching activities that are not identical, so the comparison is indicative. In both cases the leakage is a small fraction of the total, about 2%, as expected for a circuit working at several hundred MHz.

10.12 Electromigration and thermal analysis

The project guidelines ask also for electromigration (EM) and thermal information. As stated in the laboratory instructions, the rail analysis (IR drop and electromigration) could not be run with the installed version of Innovus and of the technology library, and it has not been performed. The power figures of the previous section are the available indication of the thermal load: about 13 mW on a die of 0.031 mm^2, i.e. a power density of about 43 W/cm^2. The value is high because the die is very small, but the absolute power is tiny, and in a real chip this block would be surrounded by much larger memories and I/O, spreading the heat over a much larger area.

10.13 Limitations

The memories are outside the implemented block, and no input or output delays have been specified for the ports that connect to them. Innovus therefore assumes that the inputs coming from the memories are stable at the beginning of the clock period, while the clock reaches the flip-flops through the clock tree with some delay. This is the origin of the hold violations on the input paths found during the flow. In a complete chip the memories would be clocked by the same clock tree, and the timing of the interface should be described with `set_input_delay` and `set_output_delay` constraints derived from the memory characteristics.

CHAPTER 11

Conclusions

A five-stage pipelined DLX has been designed, verified and implemented down to the layout. The processor executes 50 instructions, uses the P4 adder of the laboratories for additions and subtractions, resolves control hazards with a two-cycle flush and data hazards with forwarding, stalling only in the load-use case. After place & route it meets a clock period of 1.7 ns (588 MHz), with a margin that corresponds to about 650 MHz, without setup, hold or design rule violations, on an area of about 15 100 μm^2 and with a power of 13 mW.

11.1 Lessons learned

The work highlighted some general aspects of processor design:

- every reduction of the number of lost cycles has a cost in cycle time: forwarding and branch resolution in EX remove stalls and flushed instructions, but together they form the critical path of the processor;
- a design that is correct in simulation is not necessarily correct after synthesis, as the initial value of the control table showed. Checking the synthesized netlist, and not only the reports, is necessary;
- the critical path of a pipelined processor is not necessarily in the arithmetic: here it lies in the control of the pipeline, while the P4 adder of the ALU does not appear on it;
- the order of the steps of the physical flow matters: filler cells inserted before the last optimization leave no room for the hold-fixing buffers.

11.2 Future work

- **Shorter control word**: several bits are no longer used after the introduction of the stall and flush logic and could be removed from the table.
- **Lower branch penalty** (point 10 of the guide): branches and jumps are already handled correctly, resolved in EX with a two-cycle flush (section 6.1). The penalty could be reduced by resolving them in ID, with a dedicated comparator and adder and forwarding towards ID, down to one cycle; a branch history table predicting the outcome would avoid most of the remaining flushes. Both would have to be evaluated against their effect on the critical path, which already goes through the branch decision.

- **Remaining instructions**: byte and half-word loads and stores (`lb`, `lbu`, `lhu`, `sb`), which require byte selection in the memory stage, and `mult`.
- **Interface constraints**: input and output delays describing the timing of the memories, for a more realistic analysis of the ports.

11.3 Acknowledgements

I thank Prof. Mariagrazia Graziano and Giovanna Turvani for the material of the course and of the laboratories, on which this project is based.

APPENDIX A

Issues found in the provided material

During the development some problems were found in the material provided with the project. They are reported here together with the adopted solution.

Component	Problem	Solution
IRAM, addressing	The PC was used directly as index of the memory array, while the PC is a byte address and every location contains a 32-bit word: only one instruction out of four was read.	The index is `Addr(31 downto 2)`, i.e. the address divided by four.
Lab components	Several files in the working folder declared the same entity: the compiler kept the last one analyzed, which was not always the intended version.	Duplicate files removed; one file per entity, named after the convention of the project guide.
`conv2memory`	The script uses `od` without the `-v` option: consecutive identical lines, such as several `nop`, are replaced by a single `*`, and the program loaded in memory is shorter than the source.	`od -v` is used, which prints every line.
Register file and register of the laboratories	The reset was active high, while the testbench and the rest of the design use an active-low reset: the register file was cleared during normal operation.	The reset of both components has been made active low.
Source files	Files saved with Windows (CR+LF) or old Mac (CR only) line endings: the shell scripts did not start, and an assembly file with CR-only endings was read as a single instruction.	`build.sh` normalizes the line endings before assembling.

Component	Problem	Solution
Place & route script of the laboratory	`addFiller` without the list of filler cells: no filler inserted, with only an error message in the log. Moreover, fillers inserted before the post-route optimization leave no room for the hold-fixing buffers.	Cells listed explicitly with `-cell`; fillers inserted after the last optimization, followed by `ecoRoute`.
Assembler `dlxasm.pl`	With indented instructions the binary starts with an extra zero word, so the program is loaded 4 bytes later than assumed by the assembler. PC-relative branches and jumps (`beqz`, `bnez`, `j`, `jal`) still work, but absolute addresses computed by the program, such as a `jalr` target, are off by four bytes. The extra word is harmless when executed, since its destination is `r0`.	Assembler not modified: the test programs are written with the instructions starting from the first column, and `build.sh` checks that the number of generated instructions matches the source.

Table A.1: Problems found and adopted solutions.

Moreover, the data and instruction memories are described as arrays of `integer`. The conversion of words with the most significant bit set produces warnings in simulation (*MSB is not zero*), even if the stored values are correct. A description based on `std_logic_vector` would remove them.

APPENDIX B

Instruction encoding

All instructions are 32 bits long and the opcode always occupies bits 31–26. Table B.1 shows the three formats, as decoded by the datapath.

Format	31–26	25–21	20–16	15–11	10–0
R	opcode	`rs1`	`rs2`	`rd`	`func`
I	opcode	`rs1`	`rd`	immediate (16 bits)	
J	opcode	offset (26 bits)			

Table B.1: Instruction formats.

Consequently the destination of an I-type instruction is taken from bits 20–16, the same field used as second source by R-type instructions; `MUXRD` selects between bits 15–11 (R-type) and 20–16 (I-type). For `jal` the destination is forced to `r31` in write back.

The opcodes and function codes of all the implemented instructions are listed in tables 7.1 and 7.2 of chapter 7. As an example, `addi r1, r0, ##12` is encoded as `0x2001000C`: opcode `001000`, `rs1` = 0, `rd` = 1, immediate `0x000C`.

APPENDIX C

Test programs

C.1 Global test: `test_all.asm`

```
; test_all: every basic instruction in one program, with forwarding,
; load-use stall, flushes after branches and jumps, jal and a loop.
; Each instruction that must be cancelled by a flush writes r27:
; r27 = 0 at the end proves that every flush worked.
addi r1, r0, #12      ; r1 = 0x0C
addi r2, r0, #5       ; r2 = 0x05
add  r3, r1, r2       ; r3 = 0x11   r2 forwarded from MEM, r1 from WB
sub  r4, r3, r2       ; r4 = 0x0C
and  r5, r1, r2       ; r5 = 0x04
or   r6, r1, r2       ; r6 = 0x0D
xor  r7, r1, r2       ; r7 = 0x09
sll  r8, r2, r2       ; r8 = 0xA0
srl  r9, r8, r2       ; r9 = 0x05
subi r10, r1, #2      ; r10 = 0x0A
andi r11, r1, #6      ; r11 = 0x04
ori  r12, r1, #3      ; r12 = 0x0F
xori r13, r1, #15     ; r13 = 0x03
slli r14, r2, #3      ; r14 = 0x28
srli r15, r14, #2     ; r15 = 0x0A
addi r16, r0, #-3     ; r16 = 0xFFFFFFFD
sge  r17, r1, r2      ; 12 >= 5  -> 1
sle  r18, r2, r1      ; 5 <= 12  -> 1
sne  r19, r1, r2      ; 12 != 5  -> 1
sle  r20, r16, r2     ; -3 <= 5  -> 1   (it would be 0 with an unsigned comparison)
slei r21, r16, #-3    ; -3 <= -3 -> 1
sgei r22, r1, #12     ; 12 >= 12 -> 1
snei r23, r2, #4      ; 5 != 4   -> 1
sw   0(r0), r3        ; DRAM[0] = 17
sw   4(r0), r16       ; DRAM[1] = -3
lw   r24, 0(r0)       ; r24 = 17
add  r24, r24, r1     ; r24 = 0x1D   load-use: one stall cycle
lw   r25, 4(r0)       ; r25 = 0xFFFFFFFD
sw   8(r0), r24       ; DRAM[2] = 29
beqz r0, l1           ; taken
addi r27, r0, #99     ; flushed
addi r27, r0, #98     ; flushed
l1:
bnez r25, l2          ; taken
addi r27, r0, #99     ; flushed
addi r27, r0, #98     ; flushed
l2:
beqz r1, l3           ; NOT taken
addi r26, r0, #1      ; executed -> r26 = 1
l3:
jal  func             ; r31 = 0x98, the return address
addi r27, r0, #99     ; flushed
addi r27, r0, #98     ; flushed
func:
```

```
addi r28, r0, #7       ; r28 = 7
addi r29, r0, #3       ; loop counter
loop:
add  r30, r30, r29     ; r30 = 3 + 2 + 1 = 6
subi r29, r29, #1
bnez r29, loop         ; depends on the subi just before: forwarding to the zero test
done:
j    done              ; stops the processor
nop
nop
```

C.2 Extended instruction set: `test_ext.asm`

```
; test_ext: the instructions of the extended (pro) set: signed and unsigned
; comparisons, arithmetic right shifts, unsigned add and subtract.
; Every comparison is true, so r4..r18 = 1 at the end.
addi  r1, r0, #12        ; r1 = 12
addi  r2, r0, #5         ; r2 = 5
addi  r3, r0, #-3        ; r3 = -3 (0xFFFFFFFD)
seq   r4, r1, r1         ; 12 == 12              -> 1
slt   r5, r2, r1         ; 5 < 12                -> 1
sgt   r6, r1, r2         ; 12 > 5                -> 1
slt   r7, r3, r2         ; -3 < 5 signed         -> 1  (unsigned it would be 0)
sltu  r8, r2, r3         ; 5 < 0xFFFFFFFD        -> 1  (signed it would be 0)
sgtu  r9, r3, r2         ; 0xFFFFFFFD > 5        -> 1  (signed it would be 0)
sleu  r10, r2, r2        ; 5 <= 5                -> 1
sgeu  r11, r3, r1        ; 0xFFFFFFFD >= 12      -> 1
seqi  r12, r2, #5        ; 5 == 5                -> 1
slti  r13, r3, #0        ; -3 < 0                -> 1
sgti  r14, r1, #-1       ; 12 > -1               -> 1
sltui r15, r2, #6        ; 5 < 6                 -> 1
sgtui r16, r3, #100      ; 0xFFFFFFFD > 100      -> 1
sleui r17, r2, #5        ; 5 <= 5                -> 1
sgeui r18, r1, #12       ; 12 >= 12              -> 1
sra   r19, r3, r2        ; -3 >> 5 arithmetic    -> 0xFFFFFFFF
srai  r20, r3, #1        ; -3 >> 1 arithmetic    -> 0xFFFFFFFE
addu  r21, r1, r2        ; 12 + 5                -> 0x11
subu  r22, r1, r2        ; 12 - 5                -> 0x07
addui r23, r2, #40000    ; 5 + 0x9C40            -> 0x9C45 (with sign extension: 0xFFFF9C45)
subui r24, r1, #2        ; 12 - 2                -> 0x0A
ori   r25, r0, #32768    ; 0 | 0x8000            -> 0x8000 (with sign extension: 0xFFFF8000)
nop
nop
nop
nop
nop
```

C.3 Jumps through a register: `test_jr.asm`

```
addi r1, r0, #5       ; r1 = 5
jal  func             ; call func, r31 = 8
addi r2, r1, #1       ; after the return: r2 = 11
addi r10, r0, #48     ; r10 = address of func2
jalr r10              ; call through register, r31 = 20
addi r4, r0, #4       ; after the return: r4 = 4
j    done
addi r9, r0, #99     ; flushed
func:
addi r1, r1, #5       ; r1 = 10
jr   r31              ; return to 8
addi r9, r0, #99      ; flushed
addi r9, r0, #98      ; flushed
func2:
addi r3, r0, #3       ; r3 = 3
jr   r31              ; return to 20
addi r9, r0, #99      ; flushed
addi r9, r0, #98      ; flushed
done:
j    done
nop
nop
```

C.4 Load high immediate: `test_lhi.asm`

```
lhi  r1, #4660        ; r1 = 12340000
ori  r1, r1, #22136   ; r1 = 12345678, r1 from forwarding
lhi  r2, #32768       ; r2 = 80000000
addi r3, r2, #1       ; r3 = 80000001
lhi  r4, #1           ; r4 = 00010000
add  r5, r1, r4       ; r5 = 12355678
nop
nop
nop
nop
```

APPENDIX D

Scripts

D.1 Assembly and simulation: `build.sh`

```
#!/bin/bash
# Assembles a test program, loads it into the instruction memory and starts
# the simulation in QuestaSim. Run ./compile.sh once before this script.
# usage: ./build.sh test_alu        (program asm/test_alu.asm, without .asm)

PROG=$1
if [ -z "$PROG" ]; then
  echo "usage: ./build.sh <program_name>    (without .asm)"
  exit 1
fi

if [ ! -f "asm/$PROG.asm" ]; then
  echo "ERROR: asm/$PROG.asm does not exist"
  exit 1
fi

# normalize Windows (CR+LF) and old Mac (CR) line endings, then assemble the
# program and convert the binary into the text file read by the IRAM
tr '\r' '\n' < "asm/$PROG.asm" > test.asm
./assembler.bin/dlxasm.pl test.asm > /dev/null || { echo "ERROR: assembler failed"; exit 1; }
./assembler.bin/conv2memory test.asm.exe > test.asm.mem
sed -i '/^$/d' test.asm.mem          # remove empty lines from the memory file

# quick check: non-empty source lines vs words in the memory file
# (labels and comment lines are counted too, so a difference is not always an error)
ASM=$(grep -c . "asm/$PROG.asm")
MEM=$(wc -l < test.asm.mem)
echo "program: $PROG"
echo "asm lines: $ASM    instructions in .mem: $MEM"
[ "$ASM" -ne "$MEM" ] && echo ">>> WARNING: the two numbers differ (labels and comments are
    counted as lines)"
echo "first instructions:"
head -10 test.asm.mem

# start the simulation with the waveform setup
vsim -gui -voptargs=+acc -do waves.do work.cfg_tb &
```

D.2 Synthesis: `dlx.scr`

```
##############################################################
# DLX synthesis script - Synopsys Design Compiler
#
# Usage, from the syn directory:
#   dc_shell -x "set PERIOD 1.6; source dlx.scr" | tee syn_1.6.log
#
# The memories (IRAM, DRAM) are NOT synthesized: the top level
# DLX_SYN contains control unit, datapath, PC and IR only, and the
# memory interfaces are ports of the entity.
##############################################################

# target clock period in ns (default value if not given from outside)
if {![info exists PERIOD]} { set PERIOD 1.6 }

# 1. Analysis of the VHDL sources. They are read directly from the
#    project directory, so synthesis and simulation use the same files.
#    Order: package, components, datapath, control unit, top level.
analyze -library WORK -format vhdl { \
    ../000-globals.vhd \
    ../a.b-DataPath.core/a.b.a-register_file_v1.vhd \
    ../a.b-DataPath.core/a.b.b.a.b.a.a.a-fa.vhd \
    ../a.b-DataPath.core/a.b.b.a.b.a.a-rca.vhd \
    ../a.b-DataPath.core/a.b.b.a.b.a.b-mux21_4bit.vhd \
    ../a.b-DataPath.core/a.b.b.a.b.a-carry_select_block.vhd \
    ../a.b-DataPath.core/a.b.b.a.b-sum_generator.vhd \
    ../a.b-DataPath.core/a.b.b.a.a.a-g_block.vhd \
    ../a.b-DataPath.core/a.b.b.a.a.b-pg_block.vhd \
    ../a.b-DataPath.core/a.b.b.a.a.c-pg_net_block.vhd \
    ../a.b-DataPath.core/a.b.b.a.a-carry_generator.vhd \
    ../a.b-DataPath.core/a.b.b.a-p4_adder.vhd \
    ../a.b-DataPath.core/a.b.b-ALU_v1.vhd \
    ../a.b-DataPath.core/a.b.c-mux21_v1.vhd \
    ../a.b-DataPath.core/a.b.d-register_v1.vhd \
    ../a.b-DataPath.vhd \
    ../a.a-CU_HW.vhd \
    ../a-DLX_syn.vhd }

# 2. Elaboration of the synthesis top level and design check
#    (undriven nets, unconnected ports, constant outputs).
elaborate DLX_SYN -architecture dlx_rtl -library WORK
check_design > reports/check_${PERIOD}.rpt

# 3. Constraints.
#    Clock on the Clk port: all register-to-register paths must fit
#    in one period. The same bound is applied to any purely
#    combinational path from the inputs to the outputs.
create_clock -name "CLK" -period $PERIOD Clk
set_max_delay $PERIOD -from [all_inputs] -to [all_outputs]
#    Wire load model used in the laboratories (NanGate 45 nm).
set_wire_load_model -name 5K_hvratio_1_4

# 4. Mapping on the technology library with high effort.
compile -map_effort high

# 5. Reports: worst path, the 10 worst endpoints, total area,
#    area per block (control unit vs datapath), power.
report_timing                          > reports/timing_${PERIOD}.rpt
report_timing -max_paths 10 -nworst 1 > reports/timing_top10_${PERIOD}.rpt
report_area                            > reports/area_${PERIOD}.rpt
report_area -hierarchy                 > reports/area_hier_${PERIOD}.rpt
report_power                           > reports/power_${PERIOD}.rpt

# 6. Outputs for place & route: names compatible with Verilog,
#    hierarchical netlist and timing constraints.
change_names -hierarchy -rules verilog
write -hierarchy -f verilog -output netlist/dlx_${PERIOD}.v
write_sdc netlist/dlx_${PERIOD}.sdc

quit
```

D.3 Place & route: dlx_pnr.tcl

```
# DLX place & route, based on lab 6

# 1. configuration
source DLX.globals
init_design
puts "CHECK init: [llength [dbGet top.insts]] instances"

# 2. floorplan: aspect ratio 1.0, utilization 0.6, 5 um margins
floorPlan -r 1.0 0.6 5 5 5 5
saveDesign dlx_floorplan.enc

# 3. power rings (M9 horizontal, M10 vertical)
addRing -nets {vdd gnd} -type core_rings -follow core -layer {top M9 bottom M9 left M10 right M10}
      -width 0.8 -spacing 0.8 -offset 0.8

# 4. power stripes
addStripe -nets {vdd gnd} -layer M10 -direction vertical -width 0.8 -spacing 0.8 -
     set_to_set_distance 20 -start_from left -start_offset 15

# 5. standard cell power routing
sroute -nets {vdd gnd}

# 6. placement
setPlaceMode -fp false -maxRouteLayer 6 -place_global_place_io_pins true
placeDesign
saveDesign dlx_placed.enc

# 8. pre-CTS timing and optimization
setDelayCalMode -siAware false
timeDesign -preCTS -outDir timing_preCTS
optDesign -preCTS
puts "CHECK preCTS: [llength [dbGet top.insts]] instances"

# 9. clock tree synthesis (not needed for the combinational adder)
create_ccopt_clock_tree_spec
set_ccopt_property target_max_trans 0.05
set_ccopt_property target_skew 0.02
clock_opt_design
timeDesign -postCTS -outDir timing_postCTS

# 10. post-CTS optimization, then filler cells
optDesign -postCTS

# 11. routing
setNanoRouteMode -quiet -routeTopRoutingLayer 6
routeDesign -globalDetail
timeDesign -postRoute -outDir timing_postRoute

# 12. post-route optimization: setup and hold
optDesign -postRoute
optDesign -postRoute -hold
addFiller -cell {FILLCELL_X32 FILLCELL_X16 FILLCELL_X8 FILLCELL_X4 FILLCELL_X2 FILLCELL_X1} -
     prefix FILL
ecoRoute -target
timeDesign -postRoute       -outDir timing_final
timeDesign -postRoute -hold -outDir timing_final_hold
saveDesign dlx_routed.enc

# 13. parasitic extraction
setExtractRCMode -engine postRoute -effortLevel low
extractRC
rcOut -spef dlx.spef

# 14. checks and deliverables
verifyConnectivity -type all
report_power  > dlx_power_pnr.rpt
report_area   > dlx_area_pnr.rpt
report_timing > dlx_timing_pnr.rpt
saveNetlist dlx_routed.v
write_sdf   dlx_routed.sdf
```

APPENDIX E

VHDL sources

E.1 Package

```
library ieee;
use ieee.std_logic_1164.all;

package myTypes is

    -- ALU operation types
    type aluOp is (NOP, ADDS, SUBS, ANDS, ORS, XORS, LLS, LRS, ARS,SGES, SLES, SNES,
         SEQS, SLTS, SGTS,SGEUS, SLEUS, SLTUS, SGTUS, LHIS);

    -- Control Unit input field sizes
    constant OP_CODE_SIZE : integer := 6;   -- OPCODE field size
    constant FUNC_SIZE    : integer := 11;  -- FUNC field size

    constant numBit       : integer := 32;  --ALU size


    -- R-Type instructions (FUNC field)
    constant RTYPE_ADD : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000100000"; -- R[RC] = R[
         RA] + R[RB]
    constant RTYPE_SUB : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000100010"; -- R[RC] = R[
         RA] - R[RB]
    constant RTYPE_AND : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000100100"; -- R[RC] = R[
         RA] AND R[RB]
    constant RTYPE_OR  : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000100101"; -- R[RC] = R[
         RA] OR R[RB]
    constant RTYPE_XOR : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000100110"; -- R[RC] = R[
         RA] XOR R[RB]
    constant RTYPE_SLL : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000000100"; -- R[RC] = R[
         RA] SLL R[RB]
    constant RTYPE_SRL : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000000110"; -- R[RC] = R[
         RA] SRL R[RB]
    constant RTYPE_SNE : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000101001"; -- R[RC] = (R[
         RA] /= R[RB])
    constant RTYPE_SLE : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000101100"; -- R[RC] = (R[
         RA] <= R[RB])
    constant RTYPE_SGE : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000101101"; -- R[RC] = (R[
         RA] >= R[RB])
    constant RTYPE_SRA  : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000000111"; -- 0x07
    constant RTYPE_ADDU : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000100001"; -- 0x21
    constant RTYPE_SUBU : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000100011"; -- 0x23
    constant RTYPE_SEQ  : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000101000"; -- 0x28
    constant RTYPE_SLT  : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000101010"; -- 0x2A
    constant RTYPE_SGT  : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000101011"; -- 0x2B
    constant RTYPE_SLTU : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000111010"; -- 0x3A
    constant RTYPE_SGTU : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000111011"; -- 0x3B
    constant RTYPE_SLEU : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000111100"; -- 0x3C
    constant RTYPE_SGEU : std_logic_vector(FUNC_SIZE - 1 downto 0) := "00000111101"; -- 0x3D
    -- Base OPCODE for Register-to-Register operations
```

```
    constant RTYPE     : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "000000";

    -- I-Type & J-Type instructions (OPCODE field)
    constant ITYPE_NOP  : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "010101"; -- No operation
    constant ITYPE_J    : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "000010"; -- PC = Target
    constant ITYPE_JAL  : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "000011"; -- R[31] = PC +
         4; PC = Target
    constant ITYPE_BEQZ : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "000100"; -- if (R[RA] ==
         0) PC = PC + Offset
    constant ITYPE_BNEZ : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "000101"; -- if (R[RA] /=
         0) PC = PC + Offset
    constant ITYPE_JR   : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "010010"; -- PC = R[RA]
    constant ITYPE_JALR : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "010011"; -- R[31] = PC +
         4; PC = R[RA]
    constant ITYPE_LHI  : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "001111"; -- R[RB] = Imm
        << 16
    constant ITYPE_ADDI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "001000"; -- R[RA] = R[RB
        ] + INP1
    constant ITYPE_SUBI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "001010"; -- R[RA] = R[RB
        ] - INP1
    constant ITYPE_ANDI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "001100"; -- R[RA] = R[RB
        ] AND INP1
    constant ITYPE_ORI  : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "001101"; -- R[RA] = R[RB
        ] OR INP1
    constant ITYPE_XORI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "001110"; -- R[RA] = R[RB
        ] XOR INP1
    constant ITYPE_SLLI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "010100"; -- R[RA] = R[RB
        ] SLL INP1
    constant ITYPE_SRLI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "010110"; -- R[RA] = R[RB
        ] SRL INP1
    constant ITYPE_SNEI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "011001"; -- R[RA] = (R[
        RB] /= INP1)
    constant ITYPE_SLEI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "011100"; -- R[RA] = (R[
        RB] <= INP1)
    constant ITYPE_SGEI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "011101"; -- R[RA] = (R[
        RB] >= INP1)
    constant ITYPE_LW   : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "100011"; -- R[RA] = MEM[
        R[RB] + INP1]
    constant ITYPE_SW   : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "101011"; -- MEM[R[RB] +
        INP1] = R[RA]
    constant ITYPE_ADDUI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "001001"; -- 0x09
    constant ITYPE_SUBUI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "001011"; -- 0x0B
    constant ITYPE_SRAI  : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "010111"; -- 0x17
    constant ITYPE_SEQI  : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "011000"; -- 0x18
    constant ITYPE_SLTI  : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "011010"; -- 0x1A
    constant ITYPE_SGTI  : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "011011"; -- 0x1B
    constant ITYPE_SLTUI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "111010"; -- 0x3A
    constant ITYPE_SGTUI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "111011"; -- 0x3B
    constant ITYPE_SLEUI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "111100"; -- 0x3C
    constant ITYPE_SGEUI : std_logic_vector(OP_CODE_SIZE - 1 downto 0) := "111101"; -- 0x3D
end myTypes;
```

E.2 Top level

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use work.myTypes.all;

entity DLX is
  generic (
    IR_SIZE      : integer := 32;       -- Instruction Register Size
    PC_SIZE      : integer := 32       -- Program Counter Size
    );       -- ALU_OPC_SIZE if explicit ALU Op Code Word Size
  port (
    Clk : in std_logic;
    Rst : in std_logic);                -- Active Low
end DLX;


architecture dlx_rtl of DLX is

 ---------------------------------------------------------------------
 -- Components Declaration
 ---------------------------------------------------------------------

  --Instruction Ram
  component IRAM
    generic (
        RAM_DEPTH : integer := 48;
        I_SIZE : integer := 32)
    ;
    port (
      Rst  : in  std_logic;
      Addr : in  std_logic_vector(PC_SIZE - 1 downto 0);
      Dout : out std_logic_vector(IR_SIZE - 1 downto 0)
    );
  end component;

  -- Data Ram
 component DRAM is
    generic (
        RAM_DEPTH : integer := numBit*2;
        D_SIZE : integer := numBit
    );
    port (
        Clk     : in std_logic;
        Rst     : in  std_logic;
    DRAM_WE     : in  std_logic;
        Din     : in std_logic_vector(D_SIZE - 1 downto 0);
        addr    : in  std_logic_vector(D_SIZE - 1 downto 0);
        Dout    : out std_logic_vector(D_SIZE - 1 downto 0)
        );
 end component;

  -- Datapath
 component DataPath is

    generic (N : integer := numBit);
    port(
    Clk                 : IN  std_logic;  -- Clock
    Rst                 : IN  std_logic;  -- Reset:Active-Low
    DIN         : IN std_logic_vector(N -1 downto 0); -- DRAM data
    IR          : IN std_logic_vector(N -1 downto 0);   --instruction register content to be
         decoded
    NPC_IN          : IN std_logic_vector(N -1 downto 0);
    --fetch phase done in the DLX file
    --decode
    RegA_LATCH_EN       : IN std_logic;  -- Register A Latch Enable
        RegB_LATCH_EN       : IN std_logic;  -- Register B Latch Enable
        RegIMM_LATCH_EN     : IN std_logic;  -- Immediate Register Latch Enable
    --execute
    MUXA_SEL            : IN std_logic;  -- MUX-A Sel
        MUXB_SEL            : IN std_logic;  -- MUX-B Sel
```

```
      ALU_OUTREG_EN      : IN std_logic;  -- ALU Output Register Enable
  EQ_COND            : IN std_logic;  -- Branch if (not) Equal to Zero
  ALU_OPCODE         : IN aluOp; -- choose between implicit or exlicit coding, like
       std_logic_vector(ALU_OPC_SIZE -1 downto 0);
  RF_WE_EX       : IN std_logic;
  --memory
      LMD_LATCH_EN       : IN std_logic;  -- LMD Register Latch Enable
      JUMP_EN            : IN std_logic;  -- JUMP Enable Signal for PC input MUX
      RF_WE_MEM      : IN std_logic;
  --writeback
  WB_MUX_SEL         : IN std_logic;  -- Write Back MUX Sel
      RF_WE              : IN std_logic;  -- Register File Write Enable


  DADDR          : OUT std_logic_vector(N -1 downto 0); --DRAM address
  DOUT           : OUT std_logic_vector(N -1 downto 0); --DRAM data
  PC_OUT         : OUT std_logic_vector(N -1 downto 0); --Next PC value
  FLUSH          : OUT std_logic;
  STALL          : OUT std_logic
  );
end component;

 -- Control Unit
 component dlx_cu
 generic (
  MICROCODE_MEM_SIZE :     integer := 64;  -- Microcode Memory Size
  FUNC_SIZE          :     integer := 11;  -- Func Field Size for R-Type Ops
  OP_CODE_SIZE       :     integer := 6;  -- Op Code Size
  -- ALU_OPC_SIZE       :     integer := 6;  -- ALU Op Code Word Size
  IR_SIZE            :     integer := 32;  -- Instruction Register Size
  CW_SIZE            :     integer := 15);  -- Control Word Size
 port (
  Clk                : in  std_logic;  -- Clock
  Rst                : in  std_logic;  -- Reset:Active-Low
  STALL          : IN std_logic;
  -- Instruction Register
  IR_IN              : in  std_logic_vector(IR_SIZE - 1 downto 0);

  -- IF Control Signal
  IR_LATCH_EN        : out std_logic;  -- Instruction Register Latch Enable
  NPC_LATCH_EN       : out std_logic;
                                   -- NextProgramCounter Register Latch Enable
  -- ID Control Signals
  RegA_LATCH_EN      : out std_logic;  -- Register A Latch Enable
  RegB_LATCH_EN      : out std_logic;  -- Register B Latch Enable
  RegIMM_LATCH_EN    : out std_logic;  -- Immediate Register Latch Enable

  -- EX Control Signals
  MUXA_SEL           : out std_logic;  -- MUX-A Sel
  MUXB_SEL           : out std_logic;  -- MUX-B Sel
  ALU_OUTREG_EN      : out std_logic;  -- ALU Output Register Enable
  EQ_COND            : out std_logic;  -- Branch if (not) Equal to Zero
  RF_WE_EX           : out std_logic;  -- for managing stalls
  -- ALU Operation Code
  ALU_OPCODE         : out aluOp; -- choose between implicit or exlicit coding, like
       std_logic_vector(ALU_OPC_SIZE -1 downto 0);
  FLUSH          : IN std_logic;
  -- MEM Control Signals
  DRAM_WE            : out std_logic;  -- Data RAM Write Enable
  LMD_LATCH_EN       : out std_logic;  -- LMD Register Latch Enable
  JUMP_EN            : out std_logic;  -- JUMP Enable Signal for PC input MUX
  PC_LATCH_EN        : out std_logic;  -- Program Counte Latch Enable
  RF_WE_MEM          : out std_logic;  -- for managing stalls

  -- WB Control signals
  WB_MUX_SEL         : out std_logic;  -- Write Back MUX Sel
  RF_WE_WB             : out std_logic);  -- Register File Write Enable
 end component;


 ---------------------------------------------------------------
 -- Signals Declaration
 ---------------------------------------------------------------
```

```
-- Instruction Register (IR) and Program Counter (PC) declaration
signal IR : std_logic_vector(IR_SIZE - 1 downto 0);
signal PC : std_logic_vector(PC_SIZE - 1 downto 0);

-- Instruction Ram Bus signals
signal IRam_DOut : std_logic_vector(IR_SIZE - 1 downto 0);

-- Datapath Bus signals
signal PC_BUS : std_logic_vector(PC_SIZE -1 downto 0);

-- Control Unit Bus signals
signal IR_LATCH_EN_i : std_logic;
signal NPC_LATCH_EN_i : std_logic;
signal RegA_LATCH_EN_i : std_logic;
signal RegB_LATCH_EN_i : std_logic;
signal RegIMM_LATCH_EN_i : std_logic;
signal EQ_COND_i : std_logic;
signal JUMP_EN_i : std_logic;
signal ALU_OPCODE_i : aluOp;
signal MUXA_SEL_i : std_logic;
signal MUXB_SEL_i : std_logic;
signal ALU_OUTREG_EN_i : std_logic;
signal DRAM_WE_i : std_logic;
signal LMD_LATCH_EN_i : std_logic;
signal PC_LATCH_EN_i : std_logic;
signal WB_MUX_SEL_i : std_logic;
signal RF_WE_i : std_logic;
signal RF_WE_MEM_i: std_logic;
signal RF_WE_EX_i: std_logic;
signal FLUSH_i : std_logic;
signal STALL_i : std_logic;

-- Data Ram Bus signals
signal NPC       : std_logic_vector(PC_SIZE - 1 downto 0);
signal DRAM_Dout : std_logic_vector(IR_SIZE - 1 downto 0);
signal DP_DADDR  : std_logic_vector(IR_SIZE - 1 downto 0);
signal DP_DOUT   : std_logic_vector(IR_SIZE - 1 downto 0);

begin  -- DLX



  NPC <= PC + 4;

  -- purpose: Instruction Register Process
  -- type   : sequential
  -- inputs : Clk, Rst, IRam_DOut, IR_LATCH_EN_i
  -- outputs: IR_IN_i

  --FETCH PHASE outside datapath so IR and PC processes must be declared  here

  IR_P: process (Clk, Rst)
  begin  -- process IR_P
    if Rst = '0' then                 -- asynchronous reset (active low)
      IR <= (others => '0');
    elsif Clk'event and Clk = '1' then  -- rising clock edge
      if (IR_LATCH_EN_i = '1') then
        IR <= IRam_DOut;
      end if;
    end if;
  end process IR_P;


  -- purpose: Program Counter Process
  -- type   : sequential
  -- inputs : Clk, Rst, PC_BUS
  -- outputs: IRam_Addr

  PC_P: process (Clk, Rst)
  begin  -- process PC_P
    if Rst = '0' then                 -- asynchronous reset (active low)
      PC <= (others => '0');
    elsif Clk'event and Clk = '1' then  -- rising clock edge
      if (PC_LATCH_EN_i = '1') then
```

```
      PC <= PC_BUS;
    end if;
  end if;
end process PC_P;

-- Control Unit Instantiation
CU_I: dlx_cu
  port map (
      Clk             => Clk,
      Rst             => Rst,
  STALL       => STALL_i,
      IR_IN           => IR,
      IR_LATCH_EN     => IR_LATCH_EN_i,
      NPC_LATCH_EN    => NPC_LATCH_EN_i,
      RegA_LATCH_EN   => RegA_LATCH_EN_i,
      RegB_LATCH_EN   => RegB_LATCH_EN_i,
      RegIMM_LATCH_EN => RegIMM_LATCH_EN_i,
      MUXA_SEL        => MUXA_SEL_i,
      MUXB_SEL        => MUXB_SEL_i,
      ALU_OUTREG_EN   => ALU_OUTREG_EN_i,
      EQ_COND         => EQ_COND_i,
  RF_WE_EX    => RF_WE_EX_i,
      ALU_OPCODE      => ALU_OPCODE_i,
  FLUSH           => FLUSH_i,
      DRAM_WE         => DRAM_WE_i,
      LMD_LATCH_EN    => LMD_LATCH_EN_i,
      JUMP_EN         => JUMP_EN_i,
      PC_LATCH_EN     => PC_LATCH_EN_i,
  RF_WE_MEM       => RF_WE_MEM_i,
      WB_MUX_SEL      => WB_MUX_SEL_i,
      RF_WE_WB        => RF_WE_i

);
-- Instruction Ram Instantiation
IRAM_I: IRAM
  port map (
      Rst  => Rst,
      Addr => PC,
      Dout => IRam_DOut
);


DP_I: DataPath
port map(
  Clk    => Clk,
  Rst    => Rst,
  DIN    => DRAM_Dout,
  IR     => IR,
  NPC_IN => NPC,
  RegA_LATCH_EN   => RegA_LATCH_EN_i,
  RegB_LATCH_EN   => RegB_LATCH_EN_i,
  RegIMM_LATCH_EN => RegIMM_LATCH_EN_i,
  MUXA_SEL        => MUXA_SEL_i,
  MUXB_SEL        => MUXB_SEL_i,
  ALU_OUTREG_EN   => ALU_OUTREG_EN_i,
  EQ_COND         => EQ_COND_i,
  ALU_OPCODE      => ALU_OPCODE_i,
  RF_WE_EX        => RF_WE_EX_i,
  LMD_LATCH_EN    => LMD_LATCH_EN_i,
  JUMP_EN         => JUMP_EN_i,
  RF_WE_MEM       => RF_WE_MEM_i,
  WB_MUX_SEL      => WB_MUX_SEL_i,
  RF_WE           => RF_WE_i,
  DADDR       => DP_DADDR,
  DOUT        => DP_DOUT,
  PC_OUT      => PC_BUS,
  FLUSH       => FLUSH_i,
  STALL           => STALL_i
);


DRAM_I: DRAM
port map (
  Clk     => Clk,
```

```
      Rst     => Rst,
      DRAM_WE => DRAM_WE_i,
      Din     => DP_DOUT,
      addr    => DP_DADDR,
      Dout    => DRAM_Dout
    );

end dlx_rtl;
```

E.3 Top level for synthesis

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use work.myTypes.all;

-- DLX top level for synthesis: same as DLX but without IRAM and DRAM,
-- which are left outside as ports.

entity DLX_SYN is
  generic (
    IR_SIZE : integer := 32;
    PC_SIZE : integer := 32);
  port (
    Clk       : in  std_logic;
    Rst       : in  std_logic;                                    -- active low
    IRAM_ADDR : out std_logic_vector(PC_SIZE - 1 downto 0);       -- to instruction memory
    IRAM_IN   : in  std_logic_vector(IR_SIZE - 1 downto 0);       -- from instruction memory
    DRAM_ADDR : out std_logic_vector(31 downto 0);                -- to data memory
    DRAM_DIN  : out std_logic_vector(31 downto 0);                -- data written to memory
    DRAM_IN   : in  std_logic_vector(31 downto 0);                -- data read from memory
    DRAM_WE   : out std_logic);
end DLX_SYN;


architecture dlx_rtl of DLX_SYN is

  -- Datapath
  component DataPath is
    generic (N : integer := numBit);
    port (
      Clk             : IN  std_logic;
      Rst             : IN  std_logic;
      DIN             : IN  std_logic_vector(N-1 downto 0);
      IR              : IN  std_logic_vector(N-1 downto 0);
      NPC_IN          : IN  std_logic_vector(N-1 downto 0);
      RegA_LATCH_EN   : IN  std_logic;
      RegB_LATCH_EN   : IN  std_logic;
      RegIMM_LATCH_EN : IN  std_logic;
      MUXA_SEL        : IN  std_logic;
      MUXB_SEL        : IN  std_logic;
      ALU_OUTREG_EN   : IN  std_logic;
      EQ_COND         : IN  std_logic;
      ALU_OPCODE      : IN  aluOp;
      RF_WE_EX        : IN  std_logic;
      LMD_LATCH_EN    : IN  std_logic;
      JUMP_EN         : IN  std_logic;
      RF_WE_MEM       : IN  std_logic;
      WB_MUX_SEL      : IN  std_logic;
      RF_WE           : IN  std_logic;
      DADDR           : OUT std_logic_vector(N-1 downto 0);
      DOUT            : OUT std_logic_vector(N-1 downto 0);
      PC_OUT          : OUT std_logic_vector(N-1 downto 0);
      FLUSH           : OUT std_logic;
      STALL           : OUT std_logic);
  end component;

  -- Control Unit
  component dlx_cu
    generic (
```

```
      MICROCODE_MEM_SIZE : integer := 64;
      FUNC_SIZE          : integer := 11;
      OP_CODE_SIZE       : integer := 6;
      IR_SIZE            : integer := 32;
      CW_SIZE            : integer := 15);
    port (
      Clk              : in  std_logic;
      Rst              : in  std_logic;
      STALL            : in  std_logic;
      IR_IN            : in  std_logic_vector(IR_SIZE - 1 downto 0);
      IR_LATCH_EN      : out std_logic;
      NPC_LATCH_EN     : out std_logic;
      RegA_LATCH_EN    : out std_logic;
      RegB_LATCH_EN    : out std_logic;
      RegIMM_LATCH_EN  : out std_logic;
      MUXA_SEL         : out std_logic;
      MUXB_SEL         : out std_logic;
      ALU_OUTREG_EN    : out std_logic;
      EQ_COND          : out std_logic;
      RF_WE_EX         : out std_logic;
      ALU_OPCODE       : out aluOp;
      FLUSH            : in  std_logic;
      DRAM_WE          : out std_logic;
      LMD_LATCH_EN     : out std_logic;
      JUMP_EN          : out std_logic;
      PC_LATCH_EN      : out std_logic;
      RF_WE_MEM        : out std_logic;
      WB_MUX_SEL       : out std_logic;
      RF_WE_WB         : out std_logic);
  end component;

  -- fetch registers
  signal IR : std_logic_vector(IR_SIZE - 1 downto 0);
  signal PC : std_logic_vector(PC_SIZE - 1 downto 0);
  signal NPC : std_logic_vector(PC_SIZE - 1 downto 0);
  signal PC_BUS : std_logic_vector(PC_SIZE - 1 downto 0);

  -- control signals
  signal IR_LATCH_EN_i, NPC_LATCH_EN_i, PC_LATCH_EN_i : std_logic;
  signal RegA_LATCH_EN_i, RegB_LATCH_EN_i, RegIMM_LATCH_EN_i : std_logic;
  signal MUXA_SEL_i, MUXB_SEL_i, ALU_OUTREG_EN_i, EQ_COND_i : std_logic;
  signal ALU_OPCODE_i : aluOp;
  signal DRAM_WE_i, LMD_LATCH_EN_i, JUMP_EN_i : std_logic;
  signal WB_MUX_SEL_i, RF_WE_i, RF_WE_EX_i, RF_WE_MEM_i : std_logic;
  signal FLUSH_i, STALL_i : std_logic;

  -- memory buses
  signal IRam_DOut : std_logic_vector(IR_SIZE - 1 downto 0);
  signal DRAM_Dout : std_logic_vector(31 downto 0);
  signal DP_DADDR  : std_logic_vector(31 downto 0);
  signal DP_DOUT   : std_logic_vector(31 downto 0);

begin

  NPC <= PC + 4;

  -- memory interface: the memories are outside the synthesized block
  IRAM_ADDR <= PC;
  IRam_DOut <= IRAM_IN;
  DRAM_ADDR <= DP_DADDR;
  DRAM_DIN  <= DP_DOUT;
  DRAM_Dout <= DRAM_IN;
  DRAM_WE   <= DRAM_WE_i;

  IR_P: process (Clk, Rst)
  begin
    if Rst = '0' then
      IR <= (others => '0');
    elsif Clk'event and Clk = '1' then
      if IR_LATCH_EN_i = '1' then
        IR <= IRam_DOut;
      end if;
    end if;
  end process IR_P;
```

```
  PC_P: process (Clk, Rst)
  begin
    if Rst = '0' then
      PC <= (others => '0');
    elsif Clk'event and Clk = '1' then
      if PC_LATCH_EN_i = '1' then
        PC <= PC_BUS;
      end if;
    end if;
  end process PC_P;

  CU_I: dlx_cu
    port map (
      Clk              => Clk,
      Rst              => Rst,
      STALL            => STALL_i,
      IR_IN            => IR,
      IR_LATCH_EN      => IR_LATCH_EN_i,
      NPC_LATCH_EN     => NPC_LATCH_EN_i,
      RegA_LATCH_EN    => RegA_LATCH_EN_i,
      RegB_LATCH_EN    => RegB_LATCH_EN_i,
      RegIMM_LATCH_EN  => RegIMM_LATCH_EN_i,
      MUXA_SEL         => MUXA_SEL_i,
      MUXB_SEL         => MUXB_SEL_i,
      ALU_OUTREG_EN    => ALU_OUTREG_EN_i,
      EQ_COND          => EQ_COND_i,
      RF_WE_EX         => RF_WE_EX_i,
      ALU_OPCODE       => ALU_OPCODE_i,
      FLUSH            => FLUSH_i,
      DRAM_WE          => DRAM_WE_i,
      LMD_LATCH_EN     => LMD_LATCH_EN_i,
      JUMP_EN          => JUMP_EN_i,
      PC_LATCH_EN      => PC_LATCH_EN_i,
      RF_WE_MEM        => RF_WE_MEM_i,
      WB_MUX_SEL       => WB_MUX_SEL_i,
      RF_WE_WB         => RF_WE_i);

  DP_I: DataPath
    port map (
      Clk              => Clk,
      Rst              => Rst,
      DIN              => DRAM_Dout,
      IR               => IR,
      NPC_IN           => NPC,
      RegA_LATCH_EN    => RegA_LATCH_EN_i,
      RegB_LATCH_EN    => RegB_LATCH_EN_i,
      RegIMM_LATCH_EN  => RegIMM_LATCH_EN_i,
      MUXA_SEL         => MUXA_SEL_i,
      MUXB_SEL         => MUXB_SEL_i,
      ALU_OUTREG_EN    => ALU_OUTREG_EN_i,
      EQ_COND          => EQ_COND_i,
      ALU_OPCODE       => ALU_OPCODE_i,
      RF_WE_EX         => RF_WE_EX_i,
      LMD_LATCH_EN     => LMD_LATCH_EN_i,
      JUMP_EN          => JUMP_EN_i,
      RF_WE_MEM        => RF_WE_MEM_i,
      WB_MUX_SEL       => WB_MUX_SEL_i,
      RF_WE            => RF_WE_i,
      DADDR            => DP_DADDR,
      DOUT             => DP_DOUT,
      PC_OUT           => PC_BUS,
      FLUSH            => FLUSH_i,
      STALL            => STALL_i);

end dlx_rtl;
```

E.4 Control unit

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;
use work.myTypes.all;
--use ieee.numeric_std.all;
--use work.all;

entity dlx_cu is

  generic (
    MICROCODE_MEM_SIZE :     integer := 64;  -- Microcode Memory Size
    FUNC_SIZE          :     integer := 11;  -- Func Field Size for R-Type Ops
    OP_CODE_SIZE       :     integer := 6;  -- Op Code Size
    -- ALU_OPC_SIZE       :     integer := 6;  -- ALU Op Code Word Size
    IR_SIZE            :     integer := 32;  -- Instruction Register Size
    CW_SIZE            :     integer := 15);  -- Control Word Size
  port (
    Clk                : in  std_logic;  -- Clock
    Rst                : in  std_logic;  -- Reset:Active-Low
    STALL          : in std_logic;
    -- Instruction Register
    IR_IN              : in  std_logic_vector(IR_SIZE - 1 downto 0);

    -- IF Control Signal
    IR_LATCH_EN        : out std_logic;  -- Instruction Register Latch Enable
    NPC_LATCH_EN       : out std_logic;
                                         -- NextProgramCounter Register Latch Enable
    -- ID Control Signals
    RegA_LATCH_EN      : out std_logic;  -- Register A Latch Enable
    RegB_LATCH_EN      : out std_logic;  -- Register B Latch Enable
    RegIMM_LATCH_EN    : out std_logic;  -- Immediate Register Latch Enable

    -- EX Control Signals
    MUXA_SEL           : out std_logic;  -- MUX-A Sel
    MUXB_SEL           : out std_logic;  -- MUX-B Sel
    ALU_OUTREG_EN      : out std_logic;  -- ALU Output Register Enable
    EQ_COND            : out std_logic;  -- Branch if (not) Equal to Zero
    RF_WE_EX           : out std_logic;  -- for managing stalls
    -- ALU Operation Code
    ALU_OPCODE         : out aluOp; -- choose between implicit or exlicit coding, like
         std_logic_vector(ALU_OPC_SIZE -1 downto 0);
    FLUSH          : IN std_logic;
    -- MEM Control Signals
    DRAM_WE            : out std_logic;  -- Data RAM Write Enable
    LMD_LATCH_EN       : out std_logic;  -- LMD Register Latch Enable
    JUMP_EN            : out std_logic;  -- JUMP Enable Signal for PC input MUX
    PC_LATCH_EN        : out std_logic;  -- Program Counte Latch Enable
    RF_WE_MEM          : out std_logic;  -- for managing stalls

    -- WB Control signals
    WB_MUX_SEL         : out std_logic;  -- Write Back MUX Sel
    RF_WE_WB              : out std_logic);  -- Register File Write Enable



end dlx_cu;

architecture dlx_cu_hw of dlx_cu is

  type mem_array is array (integer range 0 to MICROCODE_MEM_SIZE - 1) of std_logic_vector(CW_SIZE
      - 1 downto 0);

  constant cw_mem : mem_array := (
    0  => "111100110000101",               -- R-Type (ADD, SUB, AND, OR, XOR, SLL, SRL, SNE, SLE,
       SGE, SRA, ADDU, SUBU, SEQ, SLT, SGT, SLTU, SGTU, SLEU, SGEU)
      1  => "000000000000000",                -- Unused
      2  => "111011010001100",                -- J    (MUXA=NPC, MUXB=Imm, JUMP_EN=1, PC_EN=1,
          RF_WE=0)
```

```
      3  => "1100011010001101",              -- JAL  (as J, but RF_WE=1 in order to save  PC+4
          in R31) ----da rivedere
      4  => "1110011011001100",              -- BEQZ (RegA, MUXA=NPC, MUXB=Imm, EQ_COND=1, PC_EN
          =1)
      5  => "1110011010001100",              -- BNEZ (as CW of BEQZ)
      18 => "1110010010001100",              -- JR   (RegA, MUXA=RegA, MUXB=Imm=0, JUMP_EN=1)
      19 => "1110010010001101",              -- JALR (as JR, plus RF_WE=1: R31 = PC + 4)
      6  => "0000000000000000",              -- Unused
      7  => "0000000000000000",              -- Unused
      8  => "1110010010000101",              -- ADDI (I-Type Aritmetic)
  9  => "1110010010000101",                -- ADDUI (I-Type Unsigned Add)
      10 => "1110010010000101",              -- SUBI
      11 => "1110010010000101",              -- SUBUI (I-Type Unsigned Sub)
      12 => "1110010010000101",              -- ANDI
      13 => "1110010010000101",              -- ORI
      14 => "1110010010000101",              -- XORI
      15 => "1110010010000101",              -- LHI  (RegIMM, MUXB=Imm, RF_WE=1: R[rd] = Imm <<
          16)
      16 => "0000000000000000",              -- Unused
      17 => "0000000000000000",              -- Unused
      20 => "1110010010000101",              -- SLLI
      21 => "0000000000000000",              -- NOP / Unused
      22 => "1110010010000101",              -- SRLI
      23 => "1110010010000101",              -- SRAI (I-Type Arithmetic Right Shift)
      24 => "1110010010000101",              -- SEQI (I-Type Set on Equal)
      25 => "1110010010000101",              -- SNEI
      26 => "1110010010000101",              -- SLTI (I-Type Set on Less Than)
      27 => "1110010010000101",              -- SGTI (I-Type Set on Greater Than)
      28 => "1110010010000101",              -- SLEI
      29 => "1110010010000101",              -- SGEI
      30 => "0000000000000000",              -- Unused
      31 => "0000000000000000",              -- Unused
      32 => "0000000000000000",              -- Unused
      33 => "0000000000000000",              -- Unused
      34 => "0000000000000000",              -- Unused
      35 => "1110010010010111",              -- LW   (Addr=RegA+Imm, DRAM_WE=0, LMD_EN=1, WB_MUX
          =1, RF_WE=1)
      36 => "0000000000000000",              -- Unused
      37 => "0000000000000000",              -- Unused
      38 => "0000000000000000",              -- Unused
      39 => "0000000000000000",              -- Unused
      40 => "0000000000000000",              -- Unused
      41 => "0000000000000000",              -- Unused
      42 => "0000000000000000",              -- Unused
      43 => "1111100010100100",              -- SW   (Addr=RegA+Imm, RegB attivo, DRAM_WE=1,
          RF_WE=0)
      44 to 57 => "0000000000000000",        -- Unused
      58 => "1110010010000101",              -- SLTUI (I-Type Set on Less Than Unsigned)
      59 => "1110010010000101",              -- SGTUI (I-Type Set on Greater Than Unsigned)
      60 => "1110010010000101",              -- SLEUI (I-Type Set on Less or Equal Unsigned)
      61 => "1110010010000101",              -- SGEUI (I-Type Set on Greater or Equal Unsigned)
      62 => "0000000000000000",              -- Remaining positions
  63 => "0000000000000000"
);

signal IR_opcode : std_logic_vector(OP_CODE_SIZE -1 downto 0);  -- OpCode part of IR
signal IR_func : std_logic_vector(FUNC_SIZE - 1 downto 0);   -- Func part of IR when Rtype
signal cw   : std_logic_vector(CW_SIZE - 1 downto 0); -- full control word read from cw_mem


-- control word is shifted to the correct stage
signal cw1 : std_logic_vector(CW_SIZE -1 downto 0); -- first stage
signal cw2 : std_logic_vector(CW_SIZE - 1 - 2 downto 0); -- second stage
signal cw3 : std_logic_vector(CW_SIZE - 1 - 5 downto 0); -- third stage
signal flush_d : std_logic;
signal aluOpcode_i: aluOp := NOP; -- ALUOP defined in package
signal aluOpcode: aluOp := NOP;



signal stall_eff : std_logic;
```

```
begin  -- dlx_cu_rtl

  IR_opcode(5 downto 0) <= IR_IN(31 downto 26);
  IR_func(10 downto 0)  <= IR_IN(FUNC_SIZE - 1 downto 0);

  cw <= cw_mem(conv_integer(IR_opcode));


  -- stage one control signals
  IR_LATCH_EN  <= not stall_eff;            --fetch always enabled
  NPC_LATCH_EN <= not stall_eff;

  -- stage two control signals
  RegA_LATCH_EN   <= cw(CW_SIZE - 3);
  RegB_LATCH_EN   <= cw(CW_SIZE - 4);
  RegIMM_LATCH_EN <= cw(CW_SIZE - 5);

  -- stage three control signals
  MUXA_SEL      <= cw1(CW_SIZE - 6);
  MUXB_SEL      <= cw1(CW_SIZE - 7);
  ALU_OUTREG_EN <= cw1(CW_SIZE - 8);
  EQ_COND       <= cw1(CW_SIZE - 9);
  RF_WE_EX      <= cw1(CW_SIZE - 15);
  -- stage four control signals
  DRAM_WE       <= cw2(CW_SIZE - 10);
  LMD_LATCH_EN  <= cw2(CW_SIZE - 11);
  JUMP_EN       <= cw1(CW_SIZE - 12);
  PC_LATCH_EN   <= not stall_eff;
  RF_WE_MEM     <= cw2(CW_SIZE - 15);
  -- stage five control signals
  WB_MUX_SEL    <= cw3(CW_SIZE - 14);
  RF_WE_WB      <= cw3(CW_SIZE - 15);


  stall_eff <= STALL and not FLUSH and not flush_d;

  -- process to pipeline control words
  CW_PIPE: process (Clk, Rst)
  begin  -- process Clk

    if Rst = '0' then                   -- asynchronous reset (active low)
      cw1 <= (others => '0');
      cw2 <= (others => '0');
      cw3 <= (others => '0');
      flush_d <= '0';
      aluOpcode <= NOP;
    elsif Clk'event and Clk = '1' then  -- rising clock edge
      if FLUSH = '1' or flush_d = '1' or stall_eff = '1' then
        cw1 <= (others => '0');
        cw2 <= cw1(CW_SIZE - 1 - 2 downto 0);
    cw3 <= cw2(CW_SIZE - 1 - 5 downto 0);
       else
        cw1 <= cw;
        cw2 <= cw1(CW_SIZE - 1 - 2 downto 0);
        cw3 <= cw2(CW_SIZE - 1 - 5 downto 0);
      end if;
      aluOpcode <= aluOpcode_i;
      flush_d <= FLUSH;
    end if;
  end process CW_PIPE;

  ALU_OPCODE <= aluOpcode
;       --ALU command instruction assigned here

  -- purpose: Generation of ALU OpCode
  -- type   : combinational
  -- inputs : IR_i
  -- outputs: aluOpcode

   ALU_OP_CODE_P : process (IR_opcode, IR_func)
   begin  -- process ALU_OP_CODE_P

    case conv_integer(unsigned(IR_opcode)) is
```

```
            -- case of R type requires analysis of FUNC
        when 0 =>
            case conv_integer(unsigned(IR_func)) is         --func code converted to integer
                when 4  => aluOpcode_i <= LLS; -- sll according to instruction set coding
                when 6  => aluOpcode_i <= LRS; -- srl
                when 7  => aluOpcode_i <= ARS; -- sra (Arithmetic Right Shift)
                when 32 => aluOpcode_i <= ADDS;
                when 33 => aluOpcode_i <= ADDS; -- addu (Unsigned Add, same bitwise operation)
                when 34 => aluOpcode_i <= SUBS;
                when 35 => aluOpcode_i <= SUBS; -- subu (Unsigned Sub, same bitwise operation)
                when 36 => aluOpcode_i <= ANDS;
                when 37 => aluOpcode_i <= ORS;
                when 38 => aluOpcode_i <= XORS;
                when 40 => aluOpcode_i <= SEQS; -- seq (Set on Equal)
                when 41 => aluOpcode_i <= SNES;
                when 42 => aluOpcode_i <= SLTS; -- slt (Set on Less Than, Signed)
                when 43 => aluOpcode_i <= SGTS; -- sgt (Set on Greater Than, Signed)
                when 44 => aluOpcode_i <= SLES;
                when 45 => aluOpcode_i <= SGES;
                when 58 => aluOpcode_i <= SLTUS; -- sltu (Set on Less Than, Unsigned)
                when 59 => aluOpcode_i <= SGTUS; -- sgtu (Set on Greater Than, Unsigned)
                when 60 => aluOpcode_i <= SLEUS; -- sleu (Set on Less or Equal, Unsigned)
                when 61 => aluOpcode_i <= SGEUS; -- sgeu (Set on Greater or Equal, Unsigned)
                when others => aluOpcode_i <= NOP;
        end case;

        when 2  => aluOpcode_i <= ADDS; -- j
        when 3  => aluOpcode_i <= ADDS; -- jal
        when 4  => aluOpcode_i <= ADDS;
        when 5  => aluOpcode_i <= ADDS;
        when 18 => aluOpcode_i <= ADDS; -- jr: target = R[RA] + 0
        when 19 => aluOpcode_i <= ADDS; -- jalr
        when 15 => aluOpcode_i <= LHIS; -- lhi
        when 8  => aluOpcode_i <= ADDS; -- addi
        when 9  => aluOpcode_i <= ADDS; -- addui (I-Type Unsigned Add)
        when 10 => aluOpcode_i <= SUBS;
        when 11 => aluOpcode_i <= SUBS; -- subui (I-Type Unsigned Sub)
        when 12 => aluOpcode_i <= ANDS;
        when 13 => aluOpcode_i <= ORS;
        when 14 => aluOpcode_i <= XORS;
        when 20 => aluOpcode_i <= LLS;
        when 22 => aluOpcode_i <= LRS;
        when 23 => aluOpcode_i <= ARS;  -- srai (I-Type Arithmetic Right Shift)
        when 24 => aluOpcode_i <= SEQS; -- seqi (I-Type Set on Equal)
        when 25 => aluOpcode_i <= SNES;
        when 26 => aluOpcode_i <= SLTS; -- slti (I-Type Set on Less Than, Signed)
        when 27 => aluOpcode_i <= SGTS; -- sgti (I-Type Set on Greater Than, Signed)
        when 28 => aluOpcode_i <= SLES;
        when 29 => aluOpcode_i <= SGES;
        when 35 => aluOpcode_i <= ADDS;
        when 43 => aluOpcode_i <= ADDS;
        when 58 => aluOpcode_i <= SLTUS; -- sltui (I-Type Set on Less Than, Unsigned)
        when 59 => aluOpcode_i <= SGTUS; -- sgtui (I-Type Set on Greater Than, Unsigned)
        when 60 => aluOpcode_i <= SLEUS; -- sleui (I-Type Set on Less or Equal, Unsigned)
        when 61 => aluOpcode_i <= SGEUS; -- sgeui (I-Type Set on Greater or Equal, Unsigned)
        when others => aluOpcode_i <= NOP;
     end case;
    end process ALU_OP_CODE_P;


end dlx_cu_hw;
```

E.5 Datapath

```
library ieee;
use ieee.std_logic_1164.all;
use WORK.myTypes.all;


entity DataPath is

generic (N : integer := numBit);

port(
    Clk                : IN  std_logic;  -- Clock
    Rst                : IN  std_logic;  -- Reset:Active-Low
    DIN        : IN std_logic_vector(N -1 downto 0); -- DRAM data
    IR         : IN std_logic_vector(N -1 downto 0);   --instruction register content to be
         decoded
    NPC_IN         : IN std_logic_vector(N -1 downto 0);
    --fetch phase done in the DLX file
    --decode
    RegA_LATCH_EN      : IN std_logic;  -- Register A Latch Enable
        RegB_LATCH_EN      : IN std_logic;  -- Register B Latch Enable
        RegIMM_LATCH_EN    : IN std_logic;  -- Immediate Register Latch Enable
    --execute
    MUXA_SEL           : IN std_logic;  -- MUX-A Sel
        MUXB_SEL           : IN std_logic;  -- MUX-B Sel
        ALU_OUTREG_EN      : IN std_logic;  -- ALU Output Register Enable
    EQ_COND            : IN std_logic;  -- Branch if (not) Equal to Zero
    ALU_OPCODE         : IN aluOp; -- choose between implicit or exlicit coding, like
         std_logic_vector(ALU_OPC_SIZE -1 downto 0);
    RF_WE_EX       : IN std_logic;
        --memory
        LMD_LATCH_EN       : IN std_logic;  -- LMD Register Latch Enable
        JUMP_EN            : IN std_logic;  -- JUMP Enable Signal for PC input MUX
        RF_WE_MEM      : IN std_logic;
    --writeback
    WB_MUX_SEL         : IN std_logic;  -- Write Back MUX Sel
        RF_WE              : IN std_logic;  -- Register File Write Enable



    DADDR          : OUT std_logic_vector(N -1 downto 0); --DRAM address
    DOUT           : OUT std_logic_vector(N -1 downto 0); --DRAM data
    PC_OUT         : OUT std_logic_vector(N -1 downto 0); --Next PC value
    FLUSH          : OUT std_logic;          --flush for CU for correct branch prediction
    STALL          : OUT std_logic
);




end entity;


architecture behavior of Datapath is

    component ALU is
      generic (N : integer := numBit);
      port (
        FUNC   : IN  aluOp;
        DATA1  : IN  std_logic_vector(N-1 downto 0);
        DATA2  : IN  std_logic_vector(N-1 downto 0);
        OUTALU : OUT std_logic_vector(N-1 downto 0)
      );
    end component;

    component register_file is
        generic (
        BIT_WIDTH    : natural := 32;
        ADDR_WIDTH   : natural := 5;
        REG_NUMBER   : natural := 32
        );
```

```
    port (
    CLK     : IN  std_logic;
    RESET   : IN  std_logic;
    ENABLE  : IN  std_logic;
    WR      : IN  std_logic;
    ADD_WR  : IN  std_logic_vector(ADDR_WIDTH - 1 downto 0);
    ADD_RD1 : IN  std_logic_vector(ADDR_WIDTH - 1 downto 0);
    ADD_RD2 : IN  std_logic_vector(ADDR_WIDTH - 1 downto 0);
    DATAIN  : IN  std_logic_vector(BIT_WIDTH - 1 downto 0);
    OUT1    : OUT std_logic_vector(BIT_WIDTH - 1 downto 0);
    OUT2    : OUT std_logic_vector(BIT_WIDTH - 1 downto 0)
    );
end component;

component REG is
    Generic(numBit: integer);
    Port (  D:  In  std_logic_vector(numBit-1 downto 0);
        EN:     In  std_logic;
        CK: In  std_logic;
        RESET:  In  std_logic;
        Q:  Out std_logic_vector(numBit-1 downto 0));
end component;


component MUX21_GENERIC is
    Generic (NBIT: integer:= 16); -- parameter for the number of bits

    Port (  A:  In  std_logic_vector(NBIT-1 downto 0);
        B:  In  std_logic_vector(NBIT-1 downto 0);
        SEL:    In  std_logic;
        Y:  Out std_logic_vector(NBIT-1 downto 0));

end component;



--fields extraction from IR instruction
signal opcode      : std_logic_vector(5 downto 0);
signal rs1     : std_logic_vector(4 downto 0);
signal rs2     : std_logic_vector(4 downto 0);
signal rd_i        : std_logic_vector(4 downto 0);
signal rd_r        : std_logic_vector(4 downto 0);
signal immediate_i : std_logic_vector(15 downto 0);
signal immediate_j : std_logic_vector(25 downto 0);
--used in decode stage
signal wr_addr     :std_logic_vector(4 downto 0);
signal wr_data     :std_logic_vector(numBit-1 downto 0);
signal regrd_ex    : std_logic_vector(4 downto 0);
signal regrd_mem   : std_logic_vector(4 downto 0);
signal regrd_wb    : std_logic_vector(4 downto 0);
signal sel_muxRD   : std_logic;
signal sel_muxIMM  : std_logic;
signal rf_out1     : std_logic_vector(numBit-1 downto 0);
signal rf_out2     : std_logic_vector(numBit-1 downto 0);
signal sign_ext    : std_logic_vector(numBit-1 downto 0);
signal regA_out    : std_logic_vector(numBit-1 downto 0);
signal regB_out    : std_logic_vector(numBit-1 downto 0);
signal regIMM_out  : std_logic_vector(numBit-1 downto 0);
signal regA_zero   : std_logic;
signal take_branch : std_logic;
signal is_jump     : std_logic;
signal npc_id      : std_logic_vector(numBit-1 downto 0);          --NPC propagated to EX for
     branch target computation
signal is_jal      : std_logic;
signal stall_int   : std_logic;
signal npc_en      : std_logic;
signal is_load     : std_logic;
signal is_load_ex  : std_logic;
--execute
signal muxA_out: std_logic_vector(numBit-1 downto 0);
signal muxB_out: std_logic_vector(numBit-1 downto 0);
signal alu_out: std_logic_vector(numBit-1 downto 0);
signal regAlu_out: std_logic_vector(numBit-1 downto 0);
signal is_jump_ex : std_logic;
```

```
    signal cond       : std_logic;
    signal cond_mem   : std_logic;
    signal npc_ex      : std_logic_vector(numBit-1 downto 0);
    signal is_jal_ex  : std_logic;
    signal rs1_ex, rs2_ex : std_logic_vector(4 downto 0);
    signal fwdA, fwdB     : std_logic_vector(numBit-1 downto 0);
    --memory
    signal regB_mem: std_logic_vector(numBit-1 downto 0);
    signal LMD_out: std_logic_vector(numBit-1 downto 0);
    signal is_jal_mem      : std_logic;
    signal npc_mem     : std_logic_vector(numBit-1 downto 0);


    --writeback
    signal imm_i_ext : std_logic_vector(numBit-1 downto 0);
    signal imm_j_ext : std_logic_vector(numBit-1 downto 0);
    signal fill_i    : std_logic_vector(15 downto 0);
    signal fill_j    : std_logic_vector(5 downto 0);
    signal regAlu_wb : std_logic_vector(numBit-1 downto 0);
    signal is_jal_wb : std_logic;
    signal npc_wb    : std_logic_vector(numBit-1 downto 0);
    signal wb_out     : std_logic_vector(numBit-1 downto 0);
    signal wr_addr_wb : std_logic_vector(4 downto 0);
    signal wr_addr_to_ex : std_logic_vector(4 downto 0);




begin

    opcode      <= IR(numBit-1 downto 26);
    rs1         <= IR(25 downto 21);
    rs2         <= IR(20 downto 16);
    rd_i        <= IR(20 downto 16);        --taking infos on IR
    rd_r        <= IR(15 downto 11);
    immediate_i     <= IR(15 downto 0);
    immediate_j     <= IR(25 downto 0);
    --decode
    sel_muxRD   <= '1' when opcode="000000" else '0';
    sel_muxIMM      <= '1' when (opcode=ITYPE_J) or (opcode=ITYPE_JAL) else '0';   -- j and jal 0
        x02 and 0x03

    fill_i <= (others => '0') when (opcode = ITYPE_ANDI  or opcode = ITYPE_ORI   or opcode =
        ITYPE_XORI  or
                                opcode = ITYPE_ADDUI or opcode = ITYPE_SUBUI or
                                opcode = ITYPE_SLTUI or opcode = ITYPE_SGTUI or
                                opcode = ITYPE_SLEUI or opcode = ITYPE_SGEUI)
            else (others => immediate_i(15));                                                   --in
                order to not lose sign MSB value is taken
    fill_j      <= (others => immediate_j(25));    --in order to not lose sign MSB value is taken
    imm_i_ext   <= fill_i & immediate_i;       --joint to create 32 bit value
    imm_j_ext       <= fill_j & immediate_j;       --joint to create 32 bit value

    is_load <= '1' when opcode = ITYPE_LW else '0';

    regA_zero <= '1' when fwdA = (fwdA'range => '0') else '0';  --check condition
    is_jump <= '1' when (opcode = ITYPE_J) or (opcode = ITYPE_JAL) or (opcode = ITYPE_JR) or (
        opcode = ITYPE_JALR) else '0';  -- unconditional jumps
    cond        <= '1' when is_jump_ex = '1' or EQ_COND = regA_zero else '0';   -- EX
    take_branch <= '1' when JUMP_EN = '1' and cond = '1' else '0';           -- MEM
    FLUSH <= take_branch;
    is_jal <= '1' when (opcode = ITYPE_JAL) or (opcode = ITYPE_JALR) else '0';  -- write PC + 4
        into r31

    stall_int <= '1' when is_load_ex = '1' and
            ((rs1 /= "00000" and rs1 = regrd_ex) or     --stalls when problems happens on rs1 or
                rs2
            (rs2 /= "00000" and rs2 = regrd_ex))        --if rs1/rs2 reads r0 does not make
                problems                                    --hazards occurs when:
         else '0';                      --1) instruction wants to write in the register file
                                    --2) instruction wants to write in the same register which

                                    --instruction in decode stage wants to read
```

```
wr_addr_to_ex <= "00000" when stall_int = '1' else wr_addr;

STALL  <= stall_int;
npc_en <= not stall_int;



fwdA <= regAlu_out when (rs1_ex /= "00000" and rs1_ex = regrd_mem  and RF_WE_MEM = '1') else
    wr_data    when (rs1_ex /= "00000" and rs1_ex = wr_addr_wb and RF_WE     = '1') else
    regA_out;

fwdB <= regAlu_out when (rs2_ex /= "00000" and rs2_ex = regrd_mem  and RF_WE_MEM = '1') else
    wr_data    when (rs2_ex /= "00000" and rs2_ex = wr_addr_wb and RF_WE     = '1') else
    regB_out;



COND_PIPE: process(Clk, Rst)
begin
    if Rst = '0' then
    is_jump_ex  <= '0';
    cond_mem    <= '0';
    is_jal_ex   <= '0';
    is_jal_mem  <= '0';
    is_jal_wb   <= '0';
    is_load_ex  <= '0';

    elsif rising_edge(Clk) then
    is_jump_ex <= is_jump;      -- ID -> EX
    cond_mem   <= cond;         -- EX -> MEM
    is_jal_ex  <= is_jal;
    is_jal_mem <= is_jal_ex;
    is_jal_wb  <= is_jal_mem;
    is_load_ex <= is_load;

    end if;
end process;


REG_FILE: register_file

port map(
    CLK     =>Clk,
    RESET   =>Rst,
    ENABLE  => '1',
    WR      => RF_WE,
    ADD_WR  => wr_addr_wb,
    ADD_RD1 => rs1,
    ADD_RD2 => rs2,
    DATAIN  => wr_data,
    OUT1    => rf_out1,
    OUT2    => rf_out2

);

REGA:  REG
    generic map (numBit => 32)
    port map(
        D     => rf_out1,
        EN    => RegA_LATCH_EN,
        CK    => Clk,
        RESET => Rst,
        Q     => regA_out
        );
REGB:  REG
    generic map (numBit => 32)
    port map(
        D     => rf_out2,
        EN    => RegB_LATCH_EN,
        CK    => Clk,
        RESET => Rst,
        Q     => regB_out
        );
```

```
MUXIMM:  MUX21_GENERIC
    generic map (NBIT => 32)
    port map(
        A   =>  imm_j_ext,
        B   =>  imm_i_ext,
        SEL =>  sel_muxIMM,
        Y   =>  sign_ext
            );

REGI:  REG
    generic map (numBit => 32)
    port map (
        D     => sign_ext,      --immediate from IR extended to 32 bits
        EN    => RegIMM_LATCH_EN,
        CK    => Clk,
        RESET => Rst,
        Q     => regIMM_out
        );
MUXA:  MUX21_GENERIC
    generic map (NBIT => 32)
    port map(
        A   =>  npc_ex,
        B   =>  fwdA,
        SEL =>  MUXA_SEL,
        Y   =>  muxA_out
            );
MUXB:  MUX21_GENERIC
    generic map (NBIT => 32)
    port map(
        A   =>  fwdB,
        B   =>  regIMM_out,
        SEL =>  MUXB_SEL,
        Y   =>  muxB_out
            );
MUXRD:  MUX21_GENERIC
    generic map (NBIT => 5)
    port map(
        A   => rd_r,
        B   => rd_i,
        SEL => sel_muxRD,
        Y   => wr_addr
            );

ALU_i : ALU
        generic map (N => 32)
        port map(
            FUNC  => ALU_OPCODE,
            DATA1 =>muxA_out,
            DATA2 =>muxB_out ,
            OUTALU => alu_out
    );

REGALU:  REG
    generic map (numBit => 32)
    port map(
        D     => alu_out,
        EN    =>ALU_OUTREG_EN,
        CK    => Clk,
        RESET => Rst,
        Q     => regAlu_out
        );

MUXJUMP:  MUX21_GENERIC
    generic map (NBIT => 32)
    port map(
        A   => alu_out,
        B   => NPC_IN,
        SEL => take_branch,
        Y   => PC_OUT
            );

REGALUWB:  REG                  --used in order to save the n of the ALU result
        generic map (numBit =>32)
```

```
        port map(
            D     => regAlu_out ,
            EN    => '1',
            CK    => Clk,
            RESET => Rst,
            Q     => regAlu_wb
            );

REGBPROP:  REG                  --used in order to save the value to be stored in the EX stage
    , and use B REG for other instructions
    generic map (numBit => 32)
    port map(
        D     => fwdB,
        EN    => '1',
        CK    => Clk,
        RESET => Rst,
        Q     => regB_mem
        );
REGLMD:  REG                    --used in order to save the data from DRAM during load
    instructions
    generic map (numBit => 32)
    port map(
        D     => DIN,
        EN    => LMD_LATCH_EN,
        CK    => Clk,
        RESET => Rst,
        Q     => LMD_out
        );

MUXWB:  MUX21_GENERIC
    generic map (NBIT => 32)
    port map(
        A   => LMD_out,
        B   => regAlu_wb,
        SEL => WB_MUX_SEL,
        Y   => wb_out
            );


REGEX:  REG                 --used in order to save the n of reg of the destination register
        generic map (numBit => 5)
        port map(
            D     => regrd_ex,
            EN    => '1',
            CK    => Clk,
            RESET => Rst,
            Q     => regrd_mem
            );
REGMEM:  REG                    --used in order to save the n of reg of the destination
    register
        generic map (numBit => 5)
        port map(
            D     => regrd_mem,
            EN    => '1',
            CK    => Clk,
            RESET => Rst,
            Q     => regrd_wb
            );

REGWB:  REG                 --used in order to save the n of reg of the destination register
        generic map (numBit => 5)
        port map(
            D     => wr_addr_to_ex,
            EN    => '1',
            CK    => Clk,
            RESET => Rst,
            Q     =>regrd_ex
            );

REGNPC_ID: REG
    generic map (numBit => 32)
    port map(
    D     => NPC_IN,
    EN    => npc_en,
```

```
    CK    => Clk,
    RESET => Rst,
    Q     => npc_id
    );

REGNPC_EX: REG
    generic map (numBit => 32)
    port map(
    D     => npc_id,
    EN    => '1',
    CK    => Clk,
    RESET => Rst,
    Q     => npc_ex
    );

REGNPC_MEM: REG
    generic map (numBit => 32)
    port map(
    D     => npc_ex,
    EN    => is_jal_ex,
    CK    => Clk,
    RESET => Rst,
    Q     => npc_mem
    );

REGNPC_WB: REG
    generic map (numBit => 32)
    port map(
    D     => npc_mem,
    EN    => is_jal_mem,
    CK    => Clk,
    RESET => Rst,
    Q     => npc_wb
    );

REGRS1:  REG
        generic map (numBit => 5)
        port map(
            D     => rs1,
            EN    => npc_en,   --not stall_int
            CK    => Clk,
            RESET => Rst,
            Q     => rs1_ex
            );
REGRS2:  REG
        generic map (numBit => 5)
        port map(
            D     => rs2,
            EN    => npc_en,
            CK    => Clk,
            RESET => Rst,
            Q     => rs2_ex
            );

MUXJAL_DATA:  MUX21_GENERIC
    generic map (NBIT => 32)
        port map(
    A   => npc_wb,
    B   => wb_out,
    SEL => is_jal_wb,
    Y   => wr_data
    );

MUXJAL_ADDR: MUX21_GENERIC
    generic map (NBIT => 5)
    port map(
    A   => "11111",
    B   => regrd_wb,
    SEL => is_jal_wb,
    Y   => wr_addr_wb
    );
    --outputs for DRAM that will be used in the DLX file

DADDR <= regAlu_out;
```

```
    DOUT <= regB_mem;

end architecture;
```

E.6 Register file

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;
use WORK.all;

entity register_file is
    generic (
        BIT_WIDTH    : natural := 32;
        ADDR_WIDTH   : natural := 5;
        REG_NUMBER   : natural := 32
    );
    port (
        CLK     : IN  std_logic;
        RESET   : IN  std_logic;
        ENABLE  : IN  std_logic;
        --RD1     : IN  std_logic;  --both removed from lab file since CU does not sent these
            signals
        --RD2     : IN  std_logic;
        WR      : IN  std_logic;
        ADD_WR  : IN  std_logic_vector(ADDR_WIDTH - 1 downto 0);
        ADD_RD1 : IN  std_logic_vector(ADDR_WIDTH - 1 downto 0);
        ADD_RD2 : IN  std_logic_vector(ADDR_WIDTH - 1 downto 0);
        DATAIN  : IN  std_logic_vector(BIT_WIDTH - 1 downto 0);
        OUT1    : OUT std_logic_vector(BIT_WIDTH - 1 downto 0);
        OUT2    : OUT std_logic_vector(BIT_WIDTH - 1 downto 0)
    );
end register_file;

architecture A of register_file is
    type REG_ARRAY is array(0 to REG_NUMBER - 1) of std_logic_vector(BIT_WIDTH - 1 downto 0);
    signal REGISTERS : REG_ARRAY;       -- is a matrix of 32 registers 32 bits each
begin
    OUT1 <= (others => '0') when ADD_RD1 = "00000" else
            DATAIN          when (WR = '1' and ADD_RD1 = ADD_WR) else   --combinatoral reading in
                order to have OUT1 and OUT2                                --ready for the next
                clock cycle
            REGISTERS(to_integer(unsigned(ADD_RD1)));           --otherwise could keep one more
                clock cycle with sequential reading

    OUT2 <= (others => '0') when ADD_RD2 = "00000" else         --FORWARDING: data in output
        either with CU write command enabled or WB address
                                    -- from ID phase is uqual to  the input address of RF
            DATAIN          when (WR = '1' and ADD_RD2 = ADD_WR) else
            REGISTERS(to_integer(unsigned(ADD_RD2)));



    process(CLK)
    begin
        if rising_edge(CLK) then
            if RESET = '0' then                         -- synchronous reset
                REGISTERS <= (others => (others => '0'));           -- double others function
                    since we are working on an a matrix
            elsif ENABLE = '1' then
                if WR = '1' then
                    REGISTERS(to_integer(unsigned(ADD_WR))) <= DATAIN;  --addresses are first used
                        as unsigned binary values,
                                --then converted as integers to choose the correct register
                end if;
           end if;
    end if;
```

```
    end process;

end A;
```

E.7 ALU

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
use IEEE.std_logic_arith.all;

use WORK.myTypes.all;

entity ALU is
  generic (N : integer := numBit);
  port (
    FUNC   : IN  aluOp;
    DATA1  : IN  std_logic_vector(N-1 downto 0);
    DATA2  : IN  std_logic_vector(N-1 downto 0);
    OUTALU : OUT std_logic_vector(N-1 downto 0)
  );
end ALU;

architecture BEHAVIOR of ALU is

  -- P4 adder of lab 6, used for both addition and subtraction:
  -- A - B = A + not(B) + 1
  component P4_ADDER is
    generic (NBIT : integer := 32);
    port (A    : in  std_logic_vector(NBIT-1 downto 0);
          B    : in  std_logic_vector(NBIT-1 downto 0);
          Cin  : in  std_logic;
          S    : out std_logic_vector(NBIT-1 downto 0);
          Cout : out std_logic);
  end component;

  signal b_p4, sum_p4    : std_logic_vector(N-1 downto 0);
  signal cin_p4, cout_p4 : std_logic;
begin

  cin_p4 <= '1' when FUNC = SUBS else '0';          -- +1 for the subtraction
  b_p4   <= not DATA2 when FUNC = SUBS else DATA2;    -- A - B = A + not(B) + 1

  ADDER: P4_ADDER
    generic map (NBIT => N)
    port map (A => DATA1, B => b_p4, Cin => cin_p4, S => sum_p4, Cout => cout_p4);

-- the ALU functions are defined externally inside alu_types file

  PROCESS_ALU: process (FUNC, DATA1, DATA2, sum_p4) -- the sensitivity list includes func (
      operation type) and the 2 operands
    variable tmp_mult : std_logic_vector((2*N)-1 downto 0); -- moltiplication result (2N bit)
  begin
    case FUNC is

    when NOP =>
            OUTALU <= (others => '0');

        when ADDS =>
            OUTALU <= sum_p4;          -- P4 adder

        when SUBS =>
            OUTALU <= sum_p4;          -- P4 adder with not(DATA2) and Cin = 1

        when ANDS =>
            OUTALU <= DATA1 and DATA2;

        when ORS =>
            OUTALU <= DATA1 or DATA2;
```

```
    when XORS =>
        OUTALU <= DATA1 xor DATA2;

when LLS =>
        OUTALU <= to_StdLogicVector((to_bitvector(DATA1)) sll (conv_integer(DATA2( 4 downto 0)
            )));

when LRS =>
        OUTALU <= to_StdLogicVector((to_bitvector(DATA1)) srl (conv_integer(DATA2( 4 downto 0)
            )));

when ARS =>
        OUTALU <= to_StdLogicVector((to_bitvector(DATA1)) sra (conv_integer(DATA2(4 downto 0))
            ));

when SGES =>

    if(signed(DATA1)>=signed(DATA2)) then

        OUTALU <= (others => '0'); OUTALU(0) <= '1';

    else
        OUTALU<=(others=>'0');
    end if;

when SLES =>
    if(signed(DATA1)<=signed(DATA2)) then

        OUTALU <= (others => '0'); OUTALU(0) <= '1';

    else
        OUTALU<=(others=>'0');
    end if;

when SNES =>
    if (signed(DATA1) /= signed(DATA2)) then
        OUTALU <= (others => '0'); OUTALU(0) <= '1';
    else
        OUTALU <= (others => '0');
    end if;

when SEQS =>
            if (DATA1 = DATA2) then
                OUTALU <= (others => '0'); OUTALU(0) <= '1';
            else
                OUTALU <= (others => '0');
            end if;

when SLTS =>
            if (signed(DATA1) < signed(DATA2)) then
                OUTALU <= (others => '0'); OUTALU(0) <= '1';
            else
                OUTALU <= (others => '0');
            end if;

when SGTS =>
            if (signed(DATA1) > signed(DATA2)) then
            OUTALU <= (others => '0'); OUTALU(0) <= '1';
            else
                OUTALU <= (others => '0');
            end if;

 when SGEUS =>
            if (unsigned(DATA1) >= unsigned(DATA2)) then
                OUTALU <= (others => '0'); OUTALU(0) <= '1';
            else
                OUTALU <= (others => '0');
            end if;

 when SLEUS =>
            if (unsigned(DATA1) <= unsigned(DATA2)) then
                OUTALU <= (others => '0'); OUTALU(0) <= '1';
            else
```

```
                    OUTALU <= (others => '0');
                end if;

     when SLTUS =>
                if (unsigned(DATA1) < unsigned(DATA2)) then
                    OUTALU <= (others => '0'); OUTALU(0) <= '1';
                else
                    OUTALU <= (others => '0');
                end if;

    when SGTUS =>
                if (unsigned(DATA1) > unsigned(DATA2)) then
                    OUTALU <= (others => '0'); OUTALU(0) <= '1';
                else
                    OUTALU <= (others => '0');
                end if;

        when LHIS =>                          -- lhi: immediate in the upper half
                OUTALU(N-1 downto 16) <= DATA2(15 downto 0);
                OUTALU(15 downto 0)   <= (others => '0');
        when others =>
            OUTALU <= (others => '0');

    end case;
  end process PROCESS_ALU;

end BEHAVIOR;

configuration CFG_ALU_BEHAVIORAL of ALU is
  for BEHAVIOR
  end for;
end CFG_ALU_BEHAVIORAL;
```

E.8 P4 adder

E.8.1 P4 adder

```
library ieee;
use ieee.std_logic_1164.all;

entity P4_ADDER is
 generic (
  NBIT : integer := 32
 );
 port (
  A    : in  std_logic_vector(NBIT-1 downto 0);
  B    : in  std_logic_vector(NBIT-1 downto 0);
  Cin  : in  std_logic;
  S    : out std_logic_vector(NBIT-1 downto 0);
  Cout : out std_logic
  );
end P4_ADDER;

architecture STRUCTURAL of P4_ADDER is
 constant NBIT_PER_BLOCK : integer := 4;     -- Setting block size as a architecture internal
     constant
 constant NUM_BLOCKS     : integer := NBIT / NBIT_PER_BLOCK;

 component CARRY_GENERATOR        -- Tree Carry Generator
  generic ( NBIT : integer; NBIT_PER_BLOCK : integer );
  port (
   A   : in  std_logic_vector(NBIT-1 downto 0);
   B   : in  std_logic_vector(NBIT-1 downto 0);
   Cin : in  std_logic;
   Co  : out std_logic_vector((NBIT/NBIT_PER_BLOCK)-1 downto 0)
   );
 end component;

 component SUM_GENERATOR        -- Select Carry Adder
```

```
  generic ( NBIT_PER_BLOCK : integer; NBLOCKS : integer );
  port (
   A  : in  std_logic_vector(NBIT_PER_BLOCK * NBLOCKS - 1 downto 0);
   B  : in  std_logic_vector(NBIT_PER_BLOCK * NBLOCKS - 1 downto 0);
   Ci : in  std_logic_vector(NBLOCKS - 1 downto 0);
   S  : out std_logic_vector(NBIT_PER_BLOCK * NBLOCKS - 1 downto 0)
   );
 end component;

 signal carry_out_tree : std_logic_vector(NUM_BLOCKS-1 downto 0); -- Internal signal to connect
     carry generator and sum generator
 signal carry_in_sum   : std_logic_vector(NUM_BLOCKS-1 downto 0);

 begin
 carry_in_sum(0) <= Cin;       -- connecting carry (C0 to Cin; C4,8,12,etc. to the respective
     source)
 carry_in_sum(NUM_BLOCKS-1 downto 1) <= carry_out_tree(NUM_BLOCKS-2 downto 0);

 Cout <= carry_out_tree(NUM_BLOCKS-1);     -- the last carry out is given as output

 CG: CARRY_GENERATOR       -- Instance of the carry generator
  generic map (
  NBIT => NBIT,
  NBIT_PER_BLOCK => NBIT_PER_BLOCK
  )
 port map (
  A => A,
  B => B,
  Cin => Cin,
  Co => carry_out_tree
  );

 SG: SUM_GENERATOR       -- Instance of the sum generator
  generic map (
   NBIT_PER_BLOCK => NBIT_PER_BLOCK,
   NBLOCKS => NUM_BLOCKS
   )
  port map (
   A => A,
   B => B,
   Ci => carry_in_sum,
   S => S
   );

end STRUCTURAL;
```

E.8.2 Carry generator

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity CARRY_GENERATOR is
    generic (
        NBIT           : integer := 32;
        NBIT_PER_BLOCK : integer := 4
    );
    port (
        A   : in  std_logic_vector(NBIT-1 downto 0);
        B   : in  std_logic_vector(NBIT-1 downto 0);
        Cin : in  std_logic;
        Co  : out std_logic_vector((NBIT/NBIT_PER_BLOCK)-1 downto 0)
    );
end CARRY_GENERATOR;

architecture STRUCTURAL of CARRY_GENERATOR is

    component PG_NET_BLOCK
        port ( A, B : in std_logic; p, g : out std_logic );
    end component;
```

```
    component G_BLOCK
        port ( Pik, Gik, Gkj : in std_logic; Gij : out std_logic );
    end component;

    component PG_BLOCK
        port ( Pik, Gik, Pkj, Gkj : in std_logic; Pij, Gij : out std_logic );
    end component;

    type SignalVector is array (NBIT downto 0) of std_logic_vector(NBIT downto 0);

    signal P_matrix : SignalVector;        -- Propagation Matrix
    signal G_matrix : SignalVector;        -- Generation Matrix

begin

    rows: for row in 0 to NBIT-1 generate

        row0: if row = 0 generate
            columns0: for column in 0 to NBIT generate
                gen_cin: if column = 0 generate
                    P_matrix(0)(0) <= '0';
                    G_matrix(0)(0) <= Cin;
                end generate;

                gen_pg: if column > 0 generate
                    PG_NET: PG_NET_BLOCK port map (
                        A => A(column-1), B => B(column-1),
                        p => P_matrix(0)(column), g => G_matrix(0)(column)
                    );
                end generate;
            end generate columns0;
        end generate row0;

        row_n: if row > 0 generate
            tree_active: if (2**(row-1) <= NBIT) generate
                columns: for column in 0 to NBIT generate
                    pass_cin: if column = 0 generate
                        P_matrix(row)(0) <= '0';
                        G_matrix(row)(0) <= G_matrix(row-1)(0);
                    end generate;

                    connect: if column > 0 generate
                        pass_wire: if column < 2**(row-1) generate
                            P_matrix(row)(column) <= P_matrix(row-1)(column);
                            G_matrix(row)(column) <= G_matrix(row-1)(column);
                        end generate;

                        do_block: if column >= 2**(row-1) generate
                            use_g: if (column - 2**(row-1)) = 0 generate
                                G_INST: G_BLOCK port map (
                                    Pik => P_matrix(row-1)(column),
                                    Gik => G_matrix(row-1)(column),
                                    Gkj => G_matrix(row-1)(0),
                                    Gij => G_matrix(row)(column)
                                );
                                P_matrix(row)(column) <= '0';
                            end generate;

                            use_pg: if (column - 2**(row-1)) > 0 generate
                                PG_INST: PG_BLOCK port map (
                                    Pik => P_matrix(row-1)(column),
                                    Gik => G_matrix(row-1)(column),
                                    Pkj => P_matrix(row-1)(column - 2**(row-1)),
                                    Gkj => G_matrix(row-1)(column - 2**(row-1)),
                                    Pij => P_matrix(row)(column),
                                    Gij => G_matrix(row)(column)
                                );
                            end generate;
                        end generate do_block;
                    end generate connect;
                end generate columns;
            end generate tree_active;tree_inactive: if (2**(row-1) > NBIT) generate
                copy_row: for column in 0 to NBIT generate
                    P_matrix(row)(column) <= P_matrix(row-1)(column);
```

```
                    G_matrix(row)(column) <= G_matrix(row-1)(column);
                end generate;
            end generate tree_inactive;
        end generate row_n;
    end generate rows;

    output_gen: for i in 0 to (NBIT/NBIT_PER_BLOCK)-1 generate
        Co(i) <= G_matrix(NBIT-1)(((i+1) * NBIT_PER_BLOCK));
    end generate output_gen;

end STRUCTURAL;
```

E.8.3 G, PG and PG-network blocks

```
library ieee;
use ieee.std_logic_1164.all;

entity G_BLOCK is
    port (
        Pik : in  std_logic;
        Gik : in  std_logic;
        Gkj : in  std_logic;
        Gij : out std_logic
        );
end G_BLOCK;

architecture BEHAVIORAL of G_BLOCK is
    begin
    Gij <= Gik or (Pik and Gkj);     -- G_i:j = G_i:k + (P_i:k * G_k-1:j)
end BEHAVIORAL;
```

```
library ieee;
use ieee.std_logic_1164.all;

entity PG_BLOCK is
    port (
        Pik : in  std_logic;
        Gik : in  std_logic;
        Pkj : in  std_logic;
        Gkj : in  std_logic;
        Pij : out std_logic;
        Gij : out std_logic
    );
end PG_BLOCK;

architecture BEHAVIORAL of PG_BLOCK is
begin
    -- Formule:
    -- G_i:j = G_i:k + (P_i:k * G_k-1:j)
    -- P_i:j = P_i:k * P_k-1:j
    Gij <= Gik or (Pik and Gkj);
    Pij <= Pik and Pkj;
end BEHAVIORAL;
```

```
library ieee;
use ieee.std_logic_1164.all;

entity PG_NET_BLOCK is
    port (
        A : in  std_logic;
        B : in  std_logic;
        p : out std_logic;
        g : out std_logic
    );
end PG_NET_BLOCK;

architecture BEHAVIORAL of PG_NET_BLOCK is
begin
                -- Basic Logic:
    p <= A xor B;    -- p_i = a_i xor b_i
    g <= A and B;    -- g_i = a_i and b_i
end BEHAVIORAL;
```

E.8.4 Sum generator

```
library ieee;
use ieee.std_logic_1164.all;

entity SUM_GENERATOR is
    generic (
        NBIT_PER_BLOCK : integer := 4;
        NBLOCKS        : integer := 8
    );
    port (
        A  : in  std_logic_vector(NBIT_PER_BLOCK * NBLOCKS - 1 downto 0);
        B  : in  std_logic_vector(NBIT_PER_BLOCK * NBLOCKS - 1 downto 0);
        Ci : in  std_logic_vector(NBLOCKS - 1 downto 0);
        S  : out std_logic_vector(NBIT_PER_BLOCK * NBLOCKS - 1 downto 0)
    );
end SUM_GENERATOR;

architecture STRUCTURAL of SUM_GENERATOR is

                                                -- Base Block
    component CARRY_SELECT_BLOCK
        generic (
            NBIT : positive := 4
        );
        port (
            A   : in  std_logic_vector(NBIT - 1 downto 0);
            B   : in  std_logic_vector(NBIT - 1 downto 0);
            Ci  : in  std_logic;
            S   : out std_logic_vector(NBIT - 1 downto 0)
        );
    end component;

begin

                                                -- Creating NBLOCKS blocks
    CSB_ARRAY : for i in 0 to NBLOCKS - 1 generate

        CSB_INST : CARRY_SELECT_BLOCK                           -- Single Carry Block
            generic map (
                NBIT => NBIT_PER_BLOCK
            )
            port map (
                A  => A(((i + 1) * NBIT_PER_BLOCK) - 1 downto i * NBIT_PER_BLOCK),
                B  => B(((i + 1) * NBIT_PER_BLOCK) - 1 downto i * NBIT_PER_BLOCK),
                Ci => Ci(i),
                S  => S(((i + 1) * NBIT_PER_BLOCK) - 1 downto i * NBIT_PER_BLOCK)
            );

    end generate CSB_ARRAY;

end STRUCTURAL;
```

E.8.5 Carry-select block

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity CARRY_SELECT_BLOCK is
    generic (
        NBIT : positive := 4 -- default dimension of the block
    );
    port (
        A   : in  std_logic_vector(NBIT - 1 downto 0);
        B   : in  std_logic_vector(NBIT - 1 downto 0);
        Ci  : in  std_logic;    -- Real Carry in that act as the MUX selector
        S   : out std_logic_vector(NBIT - 1 downto 0)
    );
end CARRY_SELECT_BLOCK;

architecture STRUCTURAL of CARRY_SELECT_BLOCK is
    component RCA
        generic (
            NBIT  : positive
        );
        port (
            A  : in  std_logic_vector(NBIT - 1 downto 0);
            B  : in  std_logic_vector(NBIT - 1 downto 0);
            Ci : in  std_logic;
            S  : out std_logic_vector(NBIT - 1 downto 0);
            Co : out std_logic
        );
    end component;

    component MUX21_4BIT
        generic (N : positive := 4);
        port (
            A:  in  std_logic_vector(N - 1 downto 0);
            B:  in  std_logic_vector(N - 1 downto 0);
            S:  in  std_logic;
            Y:  out std_logic_vector(N - 1 downto 0)
        );
    end component;

    signal sum_low  : std_logic_vector(NBIT - 1 downto 0);
    signal sum_high : std_logic_vector(NBIT - 1 downto 0);
begin

    RCA_LOW: RCA    -- RCA with Carry-in = '0'
        generic map (NBIT => NBIT)
        port map (
            A  => A,
            B  => B,
            Ci => '0',
            S  => sum_low,
            Co => open
        );

    RCA_HIGH: RCA    -- RCA with Carry-in = '1'
        generic map (NBIT => NBIT)
        port map (
            A  => A,
            B  => B,
            Ci => '1',
            S  => sum_high,
            Co => open
        );

    MUX_INST: MUX21_4BIT   -- MUX: Map 'sum_high' on 'A' because MUX Y <= A when S='1'
        generic map (N => NBIT)
        port map (
            A => sum_high,
            B => sum_low,
            S => Ci,
            Y => S
```

```
        );
end STRUCTURAL;
```

E.8.6 Ripple-carry adder and full adder

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity RCA is
    generic (
         NBIT  :    positive
        );
    Port (  A:  In  std_logic_vector(NBIT - 1 downto 0);
        B:  In  std_logic_vector(NBIT - 1 downto 0);
        Ci: In  std_logic;
        S:  Out std_logic_vector(NBIT - 1 downto 0);
        Co: Out std_logic);
end RCA;

architecture STRUCTURAL of RCA is

  signal STMP : std_logic_vector(NBIT - 1 downto 0);
  signal CTMP : std_logic_vector(NBIT downto 0);

  component FA

  Port ( A: In  std_logic;
     B: In  std_logic;
     Ci:    In  std_logic;
     S: Out std_logic;
     Co:    Out std_logic);
  end component;

begin

  CTMP(0) <= Ci;
  S <= STMP;
  Co <= CTMP(NBIT);

  ADDER1: for I in 1 to NBIT generate                    -- FA instance (considering delays) for
      each stage of the RCA
    FAI : FA

      Port Map (A(I-1), B(I-1), CTMP(I-1), STMP(I-1), CTMP(I));
  end generate;

end STRUCTURAL;


architecture BEHAVIORAL of RCA is
    signal c_tmp: std_logic_vector(NBIT downto 0);
    begin
    c_tmp(0)<=Ci;                              -- the first carry (carry-in) is initialized at the
         value 0
                                    -- because the first stage of the RCA is a half-adder

    ADDER_BITS: for i in 0 to NBIT - 1  generate
            S(i) <= A(i) xor B(i) xor c_tmp(i);
            c_tmp(i+1) <= (A(i) and B(i)) or (B(i) and c_tmp(i)) or (A(i) and c_tmp(i)) ;
            end generate;

        Co<=c_tmp(NBIT);                       -- the last carryout exceeds by 1 the original nbit
             index range (temporary signal for the carry)

end BEHAVIORAL;

configuration CFG_RCA_STRUCTURAL of RCA is              -- definition of str/beh configurations
     for the RCA
  for STRUCTURAL
    for ADDER1
```

```
      for all : FA
        use configuration WORK.CFG_FA_BEHAVIORAL;
      end for;
    end for;
  end for;
end CFG_RCA_STRUCTURAL;

configuration CFG_RCA_BEHAVIORAL of RCA is
  for BEHAVIORAL
  end for;
end CFG_RCA_BEHAVIORAL;
```

```
--------------------------------------------------------------------------------
-- FA: 1-bit full adder.
-- Basic cell of the ripple-carry adders (RCA) used in the carry-select blocks
-- of the P4 adder. It adds the two input bits and the carry-in.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity FA is

    Port (
        A:  In  std_logic;   -- first operand bit
        B:  In  std_logic;   -- second operand bit
        Ci: In  std_logic;   -- carry-in
        S:  Out std_logic;   -- sum bit
        Co: Out std_logic);  -- carry-out
end FA;

architecture BEHAVIORAL of FA is
    begin
    S <= A xor B xor Ci ;                          -- sum: odd number of ones
    Co <= (A and B) or (B and Ci) or (A and Ci) ;  -- carry: at least two ones
end BEHAVIORAL;

-- configuration used by the RCA (CFG_RCA_STRUCTURAL) to bind its full adders
configuration CFG_FA_BEHAVIORAL of FA is
    for BEHAVIORAL
    end for;
end CFG_FA_BEHAVIORAL;
```

E.8.7 4-bit multiplexer

```
--------------------------------------------------------------------------------
-- MUX21_4BIT: 2-to-1 multiplexer, N bits wide (4 by default).
-- Used in each carry-select block of the P4 adder to choose between the two
-- precomputed sums, once the carry generator provides the real carry-in.
--------------------------------------------------------------------------------
library IEEE;
use IEEE.std_logic_1164.all;

ENTITY MUX21_4BIT IS
    generic (N : positive := 4);   -- width of the inputs and of the output
    port (
        A:  in  std_logic_vector(N - 1 downto 0);   -- selected when S = '1' (sum with carry-in 1)
        B:  in  std_logic_vector(N - 1 downto 0);   -- selected when S = '0' (sum with carry-in 0)
        S:  in  std_logic;                          -- select: the carry-in of the block
        Y:  out std_logic_vector(N - 1 downto 0));  -- selected sum
end MUX21_4BIT;


architecture BEHAVIORAL of MUX21_4BIT is
begin
    Y <= A when S='1' else B;   -- select A when S is high, B otherwise
end BEHAVIORAL;
```

E.9 Multiplexer

```
library IEEE;
use IEEE.std_logic_1164.all;

entity MUX21_GENERIC is
  Generic (NBIT: integer:= 16; -- parameter for the number of bits
           DELAY_MUX: time := 3 ns);
    Port (  A:  In  std_logic_vector(NBIT-1 downto 0);
        B:  In  std_logic_vector(NBIT-1 downto 0);
        SEL:    In  std_logic;
        Y:  Out std_logic_vector(NBIT-1 downto 0));
end MUX21_GENERIC;

architecture behavioural of MUX21_GENERIC is -- no components used in the behavioral arch
begin


                  Y <= A when SEL = '1' else B;

end behavioural;


architecture structural of MUX21_GENERIC is

  signal s_sel : std_logic; -- mux selector signal
  signal s_and_a, s_and_b: std_logic_vector(NBIT-1 downto 0); -- mux input signals

  begin
  s_sel<=SEL;
  str_mux_gen :  for i in 0 to NBIT-1 generate -- mux syntax for nbits configuration

      s_and_a(i)<=A(i) and s_sel;
      s_and_b(i)<=B(i) and (not(s_sel));

      Y(i)<= s_and_a(i) or s_and_b(i);

 end generate str_mux_gen;

end structural;


-- configuration defininitions for the testbench

configuration CFG_MUX21_BEHAVIOURAL of MUX21_GENERIC is
    for behavioural
    end for;
end CFG_MUX21_BEHAVIOURAL;

configuration CFG_MUX21_STRUCTURAL of MUX21_GENERIC is
    for structural
    end for;
end CFG_MUX21_STRUCTURAL;
```

E.10 Register

```
library IEEE;
use IEEE.std_logic_1164.all;
use WORK.myTypes.all;


entity REG is
    Generic(numBit: integer);
    Port (  D:  In  std_logic_vector(numBit-1 downto 0);
        EN:     In  std_logic;
        CK: In  std_logic;
        RESET:  In  std_logic;
        Q:  Out std_logic_vector(numBit-1 downto 0));
end REG;


architecture SYNCH of REG is -- register with syncronous reset

begin
    REGSYNCH: process(CK)
    begin
      if CK'event and CK='1' then -- positive edge triggered:
        if RESET='0' then -- active low reset
          Q <= (others=>'0');
        else
          Q <= D; -- input is written on output
        end if;
      end if;
    end process;

end SYNCH;

architecture ASYNCH of REG is -- register with asyncronous reset

begin

    REGASYNCH: process(CK,RESET)
    begin
      if RESET='0' then
        Q <= (others=>'0');
      elsif CK'event and CK='1' then -- positive edge triggered:
        if EN='1' then
           Q <= D;
        end if;
      end if;
    end process;

end ASYNCH;


configuration CFG_REG_1 of REG is
    for SYNCH
    end for;
end CFG_REG_1 ;


configuration CFG_REG_2 of REG is
    for ASYNCH
    end for;
end CFG_REG_2 ;
```

E.11 Instruction memory

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use std.textio.all;
use ieee.std_logic_textio.all;


-- Instruction memory for DLX
-- Memory filled by a process which reads from a file
-- file name is "test.asm.mem"
entity IRAM is
  generic (
    RAM_DEPTH : integer := 48;
    I_SIZE : integer := 32);
  port (
    Rst  : in  std_logic;
    Addr : in  std_logic_vector(I_SIZE - 1 downto 0);
    Dout : out std_logic_vector(I_SIZE - 1 downto 0)
    );

end IRAM;

architecture IRam_Bhe of IRAM is

  type RAMtype is array (0 to RAM_DEPTH - 1) of integer;-- std_logic_vector(I_SIZE - 1 downto 0);

  signal IRAM_mem : RAMtype;

begin  -- IRam_Bhe

   Dout <= conv_std_logic_vector(IRAM_mem(conv_integer(unsigned(Addr(I_SIZE-1 downto 2))) mod
        RAM_DEPTH), I_SIZE);

  -- purpose: This process is in charge of filling the Instruction RAM with the firmware
  -- type   : combinational
  -- inputs : Rst
  -- outputs: IRAM_mem
  FILL_MEM_P: process (Rst)
    file mem_fp: text;
    variable file_line : line;
    variable index : integer := 0;
    variable tmp_data_u : std_logic_vector(I_SIZE-1 downto 0);
  begin  -- process FILL_MEM_P
    if (Rst = '0') then
      index := 0;
      file_open(mem_fp,"test.asm.mem",READ_MODE);
      while (not endfile(mem_fp)) loop
        readline(mem_fp,file_line);
        hread(file_line,tmp_data_u);
        IRAM_mem(index) <= conv_integer(unsigned(tmp_data_u));
        index := index + 1;
      end loop;
    file_close(mem_fp);
    end if;
  end process FILL_MEM_P;

end IRam_Bhe;
```

E.12 Data memory

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use work.myTypes.all;

-- Data memory for DLX
-- Memory filled by data by ALU output register


entity DRAM is
  generic (
    RAM_DEPTH : integer := numBit*2;
    D_SIZE : integer := numBit);
  port (
    Clk     : in std_logic;
    Rst     : in  std_logic;
    DRAM_WE     : in  std_logic;
    Din     : in std_logic_vector(D_SIZE - 1 downto 0);
    addr    : in  std_logic_vector(D_SIZE - 1 downto 0);
    Dout    : out std_logic_vector(D_SIZE - 1 downto 0)
    );

end DRAM;

architecture behaviour of DRAM is

  type RAMtype is array (0 to RAM_DEPTH - 1) of integer;-- std_logic_vector(D_SIZE - 1 downto 0);

  signal DRAM_mem : RAMtype;

begin  -- DRam_Bhe

  --reading process, combinatorial behaviour

  Dout <= conv_std_logic_vector(DRAM_mem(conv_integer(unsigned(addr(D_SIZE-1 downto 2))) mod
       RAM_DEPTH), D_SIZE);   --data converted and resized to 32 bits


  --writing process, sequential behaviour

  FILL_MEM_P: process (Clk,Rst)


begin
        if rising_edge(Clk) then
      if (Rst = '0') then
        DRAM_mem <= (others => 0);
        elsif DRAM_WE = '1' then
         DRAM_mem(conv_integer(unsigned(addr(D_SIZE-1 downto 2))) mod RAM_DEPTH) <= conv_integer(
              unsigned(Din));
        --removing the last 2 LSB since we are not storing bytes but entire words
      end if;
        end if;
end process FILL_MEM_P;

end behaviour;
```

E.13 Testbenches

E.13.1 Processor

```
library IEEE;

use IEEE.std_logic_1164.all;
use WORK.all;

entity tb_dlx is
end tb_dlx;

architecture TEST of tb_dlx is


    constant SIZE_IR      : integer := 32;        -- Instruction Register Size
    constant SIZE_PC      : integer := 32;        -- Program Counter Size
    constant SIZE_ALU_OPC : integer := 6;         -- ALU Op Code Word Size in case explicit coding
        is used
    signal Clock: std_logic := '0';
    signal Reset: std_logic := '1';

    component DLX
       generic (
       IR_SIZE      : integer := 32;        -- Instruction Register Size
       PC_SIZE      : integer := 32        -- Program Counter Size
       );      -- ALU_OPC_SIZE is explicit ALU Op Code Word Size
       port (
       Clk : in std_logic;
       Rst : in std_logic);                -- Active Low
    end component;
begin


        -- instance of DLX
    U1: DLX
        Generic Map (SIZE_IR, SIZE_PC) -- SIZE_ALU_OPC)
    Port Map (Clock, Reset);


        PCLOCK : process(Clock)
    begin
        Clock <= not(Clock) after 0.5 ns;
    end process;

    Reset <= '0', '1' after 2 ns;


end TEST;

--------------------------------

configuration CFG_TB of tb_dlx  is
    for TEST
    end for;
end CFG_TB;
```

E.13.2 ALU

```
--------------------------------------------------------------------------------
-- TB_ALU
-- Testbench of the ALU (a.b.b-ALU_v1.vhd).
-- Every operation of aluOp (20, lhi included) is applied for 5 ns; the expected output is
-- written in the comment next to each step, to be checked on the waveform.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;
use work.myTypes.all;

entity TB_ALU is
end TB_ALU;

architecture test of TB_ALU is

  component ALU is
    generic (N : integer := numBit);
    port (
      FUNC   : in  aluOp;
      DATA1  : in  std_logic_vector(N-1 downto 0);
      DATA2  : in  std_logic_vector(N-1 downto 0);
      OUTALU : out std_logic_vector(N-1 downto 0));
  end component;

  signal FUNC   : aluOp := NOP;
  signal DATA1  : std_logic_vector(31 downto 0) := (others => '0');
  signal DATA2  : std_logic_vector(31 downto 0) := (others => '0');
  signal OUTALU : std_logic_vector(31 downto 0);

begin

  DUT: ALU
    generic map (N => 32)
    port map (FUNC => FUNC, DATA1 => DATA1, DATA2 => DATA2, OUTALU => OUTALU);

  stimuli: process
  begin
    -- arithmetic and logic, operands 12 and 5
    DATA1 <= x"0000000C"; DATA2 <= x"00000005";
    FUNC <= ADDS;  wait for 5 ns;   -- 00000011  (12 + 5)
    FUNC <= SUBS;  wait for 5 ns;   -- 00000007  (12 - 5)
    FUNC <= ANDS;  wait for 5 ns;   -- 00000004
    FUNC <= ORS;   wait for 5 ns;   -- 0000000D
    FUNC <= XORS;  wait for 5 ns;   -- 00000009
    FUNC <= LLS;   wait for 5 ns;   -- 00000180  (12 << 5)

    -- negative result
    DATA1 <= x"00000005"; DATA2 <= x"0000000C";
    FUNC <= SUBS;  wait for 5 ns;   -- FFFFFFF9  (5 - 12 = -7)

    -- right shifts of -3 by 1: logical vs arithmetic
    DATA1 <= x"FFFFFFFD"; DATA2 <= x"00000001";
    FUNC <= LRS;   wait for 5 ns;   -- 7FFFFFFE  (logical)
    FUNC <= ARS;   wait for 5 ns;   -- FFFFFFFE  (arithmetic, sign kept)

    -- comparisons between -3 and 5: signed vs unsigned
    DATA1 <= x"FFFFFFFD"; DATA2 <= x"00000005";
    FUNC <= SLTS;  wait for 5 ns;   -- 00000001  (-3 < 5, signed)
    FUNC <= SGTS;  wait for 5 ns;   -- 00000000
    FUNC <= SGES;  wait for 5 ns;   -- 00000000
    FUNC <= SNES;  wait for 5 ns;   -- 00000001
    FUNC <= SLTUS; wait for 5 ns;   -- 00000000  (0xFFFFFFFD is large, unsigned)
    FUNC <= SGTUS; wait for 5 ns;   -- 00000001
    FUNC <= SGEUS; wait for 5 ns;   -- 00000001

    -- equal operands
    DATA1 <= x"00000005"; DATA2 <= x"00000005";
    FUNC <= SEQS;  wait for 5 ns;   -- 00000001
    FUNC <= SLES;  wait for 5 ns;   -- 00000001
    FUNC <= SLEUS; wait for 5 ns;   -- 00000001
```

```
    -- lhi: the immediate (low half of DATA2) goes to the upper half
    DATA1 <= x"00000000"; DATA2 <= x"00001234";
    FUNC <= LHIS;  wait for 5 ns;    -- 12340000

    -- no operation
    FUNC <= NOP;   wait for 5 ns;    -- 00000000

    wait;
  end process;

end test;
```

E.13.3 P4 adder

```
--------------------------------------------------------------------------------
-- TB_P4_ADDER
-- Testbench of the P4 adder (a.b.b.a-p4_adder.vhd), used by the ALU for both
-- addition and subtraction (A - B = A + not(B) + 1, with Cin = '1').
-- Each step lasts 10 ns; the expected S and Cout are written in the comments,
-- to be checked on the waveform. The operands are chosen so that the carry
-- has to cross the boundaries of the 4-bit blocks of the carry generator.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity TB_P4_ADDER is
end TB_P4_ADDER;

architecture test of TB_P4_ADDER is

  component P4_ADDER is
    generic (NBIT : integer := 32);
    port (A    : in  std_logic_vector(NBIT-1 downto 0);
          B    : in  std_logic_vector(NBIT-1 downto 0);
          Cin  : in  std_logic;
          S    : out std_logic_vector(NBIT-1 downto 0);
          Cout : out std_logic);
  end component;

  signal A, B, S  : std_logic_vector(31 downto 0) := (others => '0');
  signal Cin      : std_logic := '0';
  signal Cout     : std_logic;

begin

  DUT: P4_ADDER
    generic map (NBIT => 32)
    port map (A => A, B => B, Cin => Cin, S => S, Cout => Cout);

  stimuli: process
  begin
    -- simple addition
    A <= x"0000000C"; B <= x"00000005"; Cin <= '0'; wait for 10 ns;
                                    -- S = 00000011, Cout = 0   (12 + 5)

    -- subtraction as done by the ALU: B inverted, Cin = 1
    A <= x"0000000C"; B <= x"FFFFFFFA"; Cin <= '1'; wait for 10 ns;
                                    -- S = 00000007, Cout = 1   (12 - 5)
    A <= x"00000005"; B <= x"FFFFFFF3"; Cin <= '1'; wait for 10 ns;
                                    -- S = FFFFFFF9, Cout = 0   (5 - 12 = -7)

    -- carry input alone
    A <= x"00000000"; B <= x"00000000"; Cin <= '1'; wait for 10 ns;
                                    -- S = 00000001, Cout = 0

    -- carry crossing one block boundary and several blocks
    A <= x"0000000F"; B <= x"00000001"; Cin <= '0'; wait for 10 ns;
                                    -- S = 00000010, Cout = 0
    A <= x"0000FFFF"; B <= x"00000001"; Cin <= '0'; wait for 10 ns;
                                    -- S = 00010000, Cout = 0
```

```
    -- carry through all the 32 bits
    A <= x"FFFFFFFF"; B <= x"00000001"; Cin <= '0'; wait for 10 ns;
                                    -- S = 00000000, Cout = 1
    A <= x"7FFFFFFF"; B <= x"00000001"; Cin <= '0'; wait for 10 ns;
                                    -- S = 80000000, Cout = 0   (signed overflow)

    -- generic operands, no carry between blocks
    A <= x"12345678"; B <= x"87654321"; Cin <= '0'; wait for 10 ns;
                                    -- S = 99999999, Cout = 0

    -- generic operands with carries in every block
    A <= x"89ABCDEF"; B <= x"76543211"; Cin <= '0'; wait for 10 ns;
                                    -- S = 00000000, Cout = 1
    wait;
  end process;

end test;
```

E.13.4 Register file

```
--------------------------------------------------------------------------------
-- TB_register_file
-- Testbench of the register file (a.b.a-register_file_v1.vhd).
-- The expected values are written in the comments, to be checked on the
-- waveform. Clock period 2 ns, reset active low.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity TB_register_file is
end TB_register_file;

architecture test of TB_register_file is

  component register_file is
    generic (
      BIT_WIDTH  : natural := 32;
      ADDR_WIDTH : natural := 5;
      REG_NUMBER : natural := 32);
    port (
      CLK     : in  std_logic;
      RESET   : in  std_logic;
      ENABLE  : in  std_logic;
      WR      : in  std_logic;
      ADD_WR  : in  std_logic_vector(ADDR_WIDTH - 1 downto 0);
      ADD_RD1 : in  std_logic_vector(ADDR_WIDTH - 1 downto 0);
      ADD_RD2 : in  std_logic_vector(ADDR_WIDTH - 1 downto 0);
      DATAIN  : in  std_logic_vector(BIT_WIDTH - 1 downto 0);
      OUT1    : out std_logic_vector(BIT_WIDTH - 1 downto 0);
      OUT2    : out std_logic_vector(BIT_WIDTH - 1 downto 0));
  end component;

  signal CLK     : std_logic := '0';
  signal RESET   : std_logic := '0';
  signal ENABLE  : std_logic := '1';
  signal WR      : std_logic := '0';
  signal ADD_WR  : std_logic_vector(4 downto 0)  := "00000";
  signal ADD_RD1 : std_logic_vector(4 downto 0)  := "00000";
  signal ADD_RD2 : std_logic_vector(4 downto 0)  := "00000";
  signal DATAIN  : std_logic_vector(31 downto 0) := (others => '0');
  signal OUT1    : std_logic_vector(31 downto 0);
  signal OUT2    : std_logic_vector(31 downto 0);

begin

  DUT: register_file
    generic map (BIT_WIDTH => 32, ADDR_WIDTH => 5, REG_NUMBER => 32)
    port map (CLK => CLK, RESET => RESET, ENABLE => ENABLE, WR => WR,
              ADD_WR => ADD_WR, ADD_RD1 => ADD_RD1, ADD_RD2 => ADD_RD2,
              DATAIN => DATAIN, OUT1 => OUT1, OUT2 => OUT2);
```

```
  CLK <= not CLK after 1 ns;            -- 2 ns period

  stimuli: process
  begin
    -- reset: all registers cleared
    RESET <= '0';
    wait for 5 ns;
    RESET <= '1';

    -- write r5 and r31 (each write happens on the next rising edge)
    WR <= '1'; ADD_WR <= "00101"; DATAIN <= x"12345678"; wait for 2 ns;
    WR <= '1'; ADD_WR <= "11111"; DATAIN <= x"CAFE0031"; wait for 2 ns;
    WR <= '0';

    -- read them back on the two ports
    ADD_RD1 <= "00101"; ADD_RD2 <= "11111"; wait for 4 ns;
    -- OUT1 = 12345678, OUT2 = CAFE0031

    -- write r0: it must still read zero
    WR <= '1'; ADD_WR <= "00000"; DATAIN <= x"FFFFFFFF"; wait for 2 ns;
    WR <= '0';
    ADD_RD1 <= "00000"; wait for 4 ns;
    -- OUT1 = 00000000

    -- bypass: read r7 in the same cycle in which it is written
    ADD_RD1 <= "00111";
    WR <= '1'; ADD_WR <= "00111"; DATAIN <= x"0000BEEF"; wait for 2 ns;
    -- OUT1 = 0000BEEF already before the rising edge
    WR <= '0'; wait for 4 ns;
    -- OUT1 = 0000BEEF, now read from the register

    -- WR = 0: no write and no bypass
    ADD_WR <= "00111"; DATAIN <= x"DEADDEAD"; wait for 4 ns;
    -- OUT1 stays 0000BEEF

    wait;
  end process;

end test;
```

E.13.5 Control unit

```
--------------------------------------------------------------------------------
-- TB_CU
-- Testbench of the hardwired control unit (a.a-CU_HW.vhd).
--
-- One instruction per clock cycle is applied to IR_IN, as the instruction
-- register would do. An instruction applied in cycle n drives:
--   ID  signals in cycle n   (RegA/RegB/RegIMM_LATCH_EN, from cw)
--   EX  signals in cycle n+1 (MUXA/MUXB_SEL, ALU_OUTREG_EN, EQ_COND,
--                             JUMP_EN, RF_WE_EX, ALU_OPCODE, from cw1)
--   MEM signals in cycle n+2 (DRAM_WE, LMD_LATCH_EN, RF_WE_MEM, from cw2)
--   WB  signals in cycle n+3 (WB_MUX_SEL, RF_WE_WB, from cw3)
-- Then a FLUSH and a STALL are applied, to show the bubbles.
-- Clock period 2 ns, reset active low.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;
use work.myTypes.all;

entity TB_CU is
end TB_CU;

architecture test of TB_CU is

  component dlx_cu is
    generic (
      MICROCODE_MEM_SIZE : integer := 64;
      FUNC_SIZE          : integer := 11;
      OP_CODE_SIZE       : integer := 6;
      IR_SIZE            : integer := 32;
```

```
      CW_SIZE            : integer := 15);
    port (
      Clk             : in  std_logic;
      Rst             : in  std_logic;
      STALL           : in  std_logic;
      IR_IN           : in  std_logic_vector(IR_SIZE - 1 downto 0);
      IR_LATCH_EN     : out std_logic;
      NPC_LATCH_EN    : out std_logic;
      RegA_LATCH_EN   : out std_logic;
      RegB_LATCH_EN   : out std_logic;
      RegIMM_LATCH_EN : out std_logic;
      MUXA_SEL        : out std_logic;
      MUXB_SEL        : out std_logic;
      ALU_OUTREG_EN   : out std_logic;
      EQ_COND         : out std_logic;
      RF_WE_EX        : out std_logic;
      ALU_OPCODE      : out aluOp;
      FLUSH           : in  std_logic;
      DRAM_WE         : out std_logic;
      LMD_LATCH_EN    : out std_logic;
      JUMP_EN         : out std_logic;
      PC_LATCH_EN     : out std_logic;
      RF_WE_MEM       : out std_logic;
      WB_MUX_SEL      : out std_logic;
      RF_WE_WB        : out std_logic);
  end component;

  signal Clk, Rst, STALL, FLUSH : std_logic := '0';
  signal IR_IN : std_logic_vector(31 downto 0) := (others => '0');
  signal IR_LATCH_EN, NPC_LATCH_EN, PC_LATCH_EN : std_logic;
  signal RegA_LATCH_EN, RegB_LATCH_EN, RegIMM_LATCH_EN : std_logic;
  signal MUXA_SEL, MUXB_SEL, ALU_OUTREG_EN, EQ_COND, JUMP_EN, RF_WE_EX : std_logic;
  signal ALU_OPCODE : aluOp;
  signal DRAM_WE, LMD_LATCH_EN, RF_WE_MEM : std_logic;
  signal WB_MUX_SEL, RF_WE_WB : std_logic;

  -- instruction words (hexadecimal encoding of the DLX formats)
  constant I_ADDI : std_logic_vector(31 downto 0) := x"2001000C";  -- addi r1, r0, #12
  constant I_ADD  : std_logic_vector(31 downto 0) := x"00221820";  -- add  r3, r1, r2
  constant I_SUB  : std_logic_vector(31 downto 0) := x"00221822";  -- sub  r3, r1, r2
  constant I_SLT  : std_logic_vector(31 downto 0) := x"0022182A";  -- slt  r3, r1, r2
  constant I_LW   : std_logic_vector(31 downto 0) := x"8C040000";  -- lw   r4, 0(r0)
  constant I_SW   : std_logic_vector(31 downto 0) := x"AC010000";  -- sw   0(r0), r1
  constant I_BEQZ : std_logic_vector(31 downto 0) := x"10200008";  -- beqz r1, +8
  constant I_J    : std_logic_vector(31 downto 0) := x"08000010";  -- j    +16
  constant I_JAL  : std_logic_vector(31 downto 0) := x"0C000010";  -- jal  +16
  constant I_JR   : std_logic_vector(31 downto 0) := x"4BE00000";  -- jr   r31
  constant I_JALR : std_logic_vector(31 downto 0) := x"4D400000";  -- jalr r10
  constant I_LHI  : std_logic_vector(31 downto 0) := x"3C011234";  -- lhi  r1, #0x1234
  constant I_NOP  : std_logic_vector(31 downto 0) := x"54000000";  -- nop

begin

  DUT: dlx_cu
    port map (
      Clk => Clk, Rst => Rst, STALL => STALL, IR_IN => IR_IN,
      IR_LATCH_EN => IR_LATCH_EN, NPC_LATCH_EN => NPC_LATCH_EN,
      RegA_LATCH_EN => RegA_LATCH_EN, RegB_LATCH_EN => RegB_LATCH_EN,
      RegIMM_LATCH_EN => RegIMM_LATCH_EN, MUXA_SEL => MUXA_SEL,
      MUXB_SEL => MUXB_SEL, ALU_OUTREG_EN => ALU_OUTREG_EN, EQ_COND => EQ_COND,
      RF_WE_EX => RF_WE_EX, ALU_OPCODE => ALU_OPCODE, FLUSH => FLUSH,
      DRAM_WE => DRAM_WE, LMD_LATCH_EN => LMD_LATCH_EN, JUMP_EN => JUMP_EN,
      PC_LATCH_EN => PC_LATCH_EN, RF_WE_MEM => RF_WE_MEM,
      WB_MUX_SEL => WB_MUX_SEL, RF_WE_WB => RF_WE_WB);

  Clk <= not Clk after 1 ns;          -- 2 ns period

  stimuli: process
  begin
    Rst <= '0'; IR_IN <= I_NOP;
    wait for 5 ns;                    -- every control output at 0 during reset
    Rst <= '1';

    -- one instruction per cycle; what to observe:
```

```
    IR_IN <= I_ADDI; wait for 2 ns;   -- RegIMM enable; next cycle ALU_OPCODE = ADDS; WB: RF_WE_WB
         = 1
    IR_IN <= I_ADD;  wait for 2 ns;   -- RegA and RegB enables; EX: ADDS; WB: RF_WE_WB = 1
    IR_IN <= I_SUB;  wait for 2 ns;   -- EX: ALU_OPCODE = SUBS (decoded from func)
    IR_IN <= I_SLT;  wait for 2 ns;   -- EX: ALU_OPCODE = SLTS
    IR_IN <= I_LW;   wait for 2 ns;   -- MEM: LMD_LATCH_EN = 1; WB: WB_MUX_SEL = 1, RF_WE_WB = 1
    IR_IN <= I_SW;   wait for 2 ns;   -- MEM: DRAM_WE = 1; WB: RF_WE_WB = 0
    IR_IN <= I_BEQZ; wait for 2 ns;   -- EX: JUMP_EN = 1, EQ_COND = 1; no register written
    IR_IN <= I_J;    wait for 2 ns;   -- EX: JUMP_EN = 1, EQ_COND = 0
    IR_IN <= I_JAL;  wait for 2 ns;   -- EX: JUMP_EN = 1; WB: RF_WE_WB = 1 (return address in r31)
    IR_IN <= I_JR;   wait for 2 ns;   -- RegA enable; EX: JUMP_EN = 1, MUXA_SEL = 0 (register),
        ADDS
    IR_IN <= I_JALR; wait for 2 ns;   -- as jr; WB: RF_WE_WB = 1 (return address in r31)
    IR_IN <= I_LHI;  wait for 2 ns;   -- RegIMM enable; EX: ALU_OPCODE = LHIS; WB: RF_WE_WB = 1
    IR_IN <= I_NOP;  wait for 8 ns;   -- all control words drain to zero

    -- FLUSH: a branch is taken in EX; the next two control words become bubbles
    IR_IN <= I_ADD;  wait for 2 ns;   -- add enters ID normally
    FLUSH <= '1';
    IR_IN <= I_ADDI; wait for 2 ns;   -- FLUSH: cw1 = 0 in the next cycle instead of addi
    FLUSH <= '0';
    IR_IN <= I_SUB;  wait for 2 ns;   -- flush_d: cw1 = 0 again instead of sub
    IR_IN <= I_NOP;  wait for 8 ns;   -- only the add reaches WB with RF_WE_WB = 1

    -- STALL: load-use hazard detected in ID
    IR_IN <= I_ADD;  STALL <= '1';
    wait for 2 ns;                    -- IR_LATCH_EN = NPC_LATCH_EN = PC_LATCH_EN = 0;
                                      -- next cycle cw1 = 0 (bubble in EX)
    STALL <= '0';     wait for 2 ns;  -- enables back to 1; the add proceeds
    IR_IN <= I_NOP;  wait for 8 ns;

    wait;
  end process;

end test;
```

E.13.6 Register

```
--------------------------------------------------------------------------------
-- TB_REG
-- Testbench of the generic register with enable (a.b.d-register_v1.vhd).
-- Reset asynchronous, active low; load on the rising edge only when EN = '1'.
-- The enable is what keeps an instruction in its stage during a stall.
-- Clock period 2 ns.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity TB_REG is
end TB_REG;

architecture test of TB_REG is

  component REG is
    generic (numBit : integer);
    port (D     : in  std_logic_vector(numBit-1 downto 0);
          EN    : in  std_logic;
          CK    : in  std_logic;
          RESET : in  std_logic;
          Q     : out std_logic_vector(numBit-1 downto 0));
  end component;

  signal D, Q  : std_logic_vector(31 downto 0) := (others => '0');
  signal EN    : std_logic := '0';
  signal CK    : std_logic := '0';
  signal RESET : std_logic := '0';

begin

  DUT: REG
    generic map (numBit => 32)
    port map (D => D, EN => EN, CK => CK, RESET => RESET, Q => Q);

  CK <= not CK after 1 ns;            -- 2 ns period

  stimuli: process
  begin
    RESET <= '0'; D <= x"12345678"; EN <= '1';
    wait for 5 ns;                    -- Q = 00000000 while reset is active
    RESET <= '1';
    wait for 4 ns;                    -- Q = 12345678 from the next rising edge

    D <= x"CAFE0001";
    wait for 4 ns;                    -- Q = CAFE0001

    EN <= '0'; D <= x"DEADDEAD";
    wait for 6 ns;                    -- Q stays CAFE0001: register frozen (stall)

    EN <= '1';
    wait for 4 ns;                    -- Q = DEADDEAD

    RESET <= '0';
    wait for 1.5 ns;                  -- Q = 00000000 at once, without waiting for the clock
    RESET <= '1';
    wait;
  end process;

end test;
```

E.13.7 Multiplexer

```
--------------------------------------------------------------------------------
-- TB_MUX21
-- Testbench of the generic 2-to-1 multiplexer (a.b.c-mux21_v1.vhd).
-- SEL = '1' selects A, SEL = '0' selects B (convention used in the
-- datapath, e.g. MUXJUMP takes the jump target on A when the branch is taken).
-- The output follows the inputs after DELAY_MUX.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity TB_MUX21 is
end TB_MUX21;

architecture test of TB_MUX21 is

  component MUX21_GENERIC is
    generic (NBIT : integer := 16; DELAY_MUX : time := 3 ns);
    port (A   : in  std_logic_vector(NBIT-1 downto 0);
          B   : in  std_logic_vector(NBIT-1 downto 0);
          SEL : in  std_logic;
          Y   : out std_logic_vector(NBIT-1 downto 0));
  end component;

  signal A, B, Y : std_logic_vector(31 downto 0) := (others => '0');
  signal SEL     : std_logic := '0';

begin

  DUT: MUX21_GENERIC
    generic map (NBIT => 32)
    port map (A => A, B => B, SEL => SEL, Y => Y);

  stimuli: process
  begin
    A <= x"AAAAAAAA"; B <= x"55555555";
    SEL <= '1'; wait for 10 ns;    -- Y = AAAAAAAA
    SEL <= '0'; wait for 10 ns;    -- Y = 55555555
    B <= x"12345678";  wait for 10 ns;    -- Y = 12345678, follows B
    SEL <= '1'; wait for 10 ns;    -- Y = AAAAAAAA
    A <= x"0000BEEF";  wait for 10 ns;    -- Y = 0000BEEF, follows A
    wait;
  end process;

end test;
```

E.13.8 Instruction memory

```
--------------------------------------------------------------------------------
-- TB_IRAM
-- Testbench of the instruction memory (a.c-IRAM.vhd).
-- While Rst = '0' (active low) the memory loads the program from the file
-- test.asm.mem, in the simulation directory. The read is combinational and
-- the address is a byte address: Addr(31 downto 2) selects the word.
--
-- The expected values in the comments refer to test_all.asm, the last program
-- assembled by build.sh (check with:  head -3 test.asm.mem):
--   word 0: addi r1, r0, #12  -> 2001000C
--   word 1: addi r2, r0, #5   -> 20020005
--   word 2: add  r3, r1, r2   -> 00221820
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity TB_IRAM is
end TB_IRAM;

architecture test of TB_IRAM is
```

```
  component IRAM is
    generic (RAM_DEPTH : integer := 48;
             I_SIZE    : integer := 32);
    port (Rst  : in  std_logic;
          Addr : in  std_logic_vector(I_SIZE - 1 downto 0);
          Dout : out std_logic_vector(I_SIZE - 1 downto 0));
  end component;

  signal Rst  : std_logic := '0';
  signal Addr : std_logic_vector(31 downto 0) := (others => '0');
  signal Dout : std_logic_vector(31 downto 0);

begin

  DUT: IRAM
    port map (Rst => Rst, Addr => Addr, Dout => Dout);

  stimuli: process
  begin
    Rst <= '0'; wait for 5 ns;          -- program loaded from test.asm.mem
    Rst <= '1';

    Addr <= x"00000000"; wait for 5 ns; -- Dout = 2001000C  (word 0)
    Addr <= x"00000004"; wait for 5 ns; -- Dout = 20020005  (word 1)
    Addr <= x"00000008"; wait for 5 ns; -- Dout = 00221820  (word 2)

    -- the two least significant bits are discarded
    Addr <= x"00000005"; wait for 5 ns; -- Dout = 20020005, same word as address 4
    Addr <= x"00000007"; wait for 5 ns; -- Dout = 20020005

    -- addresses beyond the 48 words wrap around (modulo RAM_DEPTH)
    Addr <= x"000000C0"; wait for 5 ns; -- 192 = word 48 -> word 0: Dout = 2001000C

    -- a second reset reloads the program from the beginning
    Rst <= '0'; wait for 5 ns;
    Rst <= '1';
    Addr <= x"00000000"; wait for 5 ns; -- Dout = 2001000C again
    wait;
  end process;

end test;
```

E.13.9 Data memory

```
--------------------------------------------------------------------------------
-- TB_DRAM
-- Testbench of the data memory (a.d-DRAM.vhd).
-- Byte addresses, word organized: the two least significant bits of addr
-- are discarded (address 4 is the second word).
-- Writing is synchronous (rising edge, DRAM_WE = '1'), reading combinational.
-- Clock period 2 ns, reset active low.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;
use work.myTypes.all;

entity TB_DRAM is
end TB_DRAM;

architecture test of TB_DRAM is

  component DRAM is
    generic (RAM_DEPTH : integer := numBit*2;
             D_SIZE    : integer := numBit);
    port (Clk     : in  std_logic;
          Rst     : in  std_logic;
          DRAM_WE : in  std_logic;
          Din     : in  std_logic_vector(D_SIZE - 1 downto 0);
          addr    : in  std_logic_vector(D_SIZE - 1 downto 0);
          Dout    : out std_logic_vector(D_SIZE - 1 downto 0));
  end component;
```

```
  signal Clk     : std_logic := '0';
  signal Rst     : std_logic := '0';
  signal DRAM_WE : std_logic := '0';
  signal Din     : std_logic_vector(31 downto 0) := (others => '0');
  signal addr    : std_logic_vector(31 downto 0) := (others => '0');
  signal Dout    : std_logic_vector(31 downto 0);

begin

  DUT: DRAM
    port map (Clk => Clk, Rst => Rst, DRAM_WE => DRAM_WE,
              Din => Din, addr => addr, Dout => Dout);

  Clk <= not Clk after 1 ns;          -- 2 ns period

  stimuli: process
  begin
    Rst <= '0'; wait for 5 ns;
    Rst <= '1';

    -- three stores, as in test_all: 17, -3, 29 at addresses 0, 4, 8
    DRAM_WE <= '1';
    addr <= x"00000000"; Din <= x"00000011"; wait for 2 ns;
    addr <= x"00000004"; Din <= x"FFFFFFFD"; wait for 2 ns;
    addr <= x"00000008"; Din <= x"0000001D"; wait for 2 ns;
    DRAM_WE <= '0';

    -- loads
    addr <= x"00000000"; wait for 2 ns;   -- Dout = 00000011
    addr <= x"00000004"; wait for 2 ns;   -- Dout = FFFFFFFD, negative value kept
    addr <= x"00000008"; wait for 2 ns;   -- Dout = 0000001D
    addr <= x"00000006"; wait for 2 ns;   -- Dout = FFFFFFFD: same word as address 4

    -- DRAM_WE = '0': nothing is written
    addr <= x"00000000"; Din <= x"DEADDEAD"; wait for 4 ns;
                                          -- Dout stays 00000011
    wait;
  end process;

end test;
```